CODE4LIFE TECHNOLOGY

LƯU LẠI ĐỂ ÔN THI

CẨM NANG HĐH

HỆ ĐIỀU HÀNH CHO SINH VIÊN • 134 TRANG

```
def fifo(refs, n):
    mem, faults = [], 0
    for p in refs:
        if p not in mem:
            mem = (mem + [p])[-n:]
            faults += 1
    return faults
refs = [1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5]
print(fifo(refs, 3), fifo(refs, 4))
```

3 khung: 9 lỗi. 4 khung: 10 lỗi. Vì sao?

Tiến trình | Lập lịch | Bộ nhớ | Linux | Ôn thi

Code4Life Technology - Tào Việt Đức

Bản đồ cẩm nang

Đi từ nguyên lý, qua tiến trình, bộ nhớ và file, tới thực hành Linux và ôn thi.

1 TỔNG QUAN Trang 3

2 TIẾN TRÌNH VÀ LUỒNG Trang 14

3 LẬP LỊCH CPU Trang 26

4 ĐỒNG BỘ Trang 38

5 DEADLOCK Trang 51

6 QUẢN LÝ BỘ NHỚ Trang 57

7 FILE VÀ LƯU TRỮ Trang 77

8 ẢO HÓA VÀ CONTAINER Trang 96

9 THỰC HÀNH Trang 100

10 ÔN THI Trang 118

Dùng cẩm nang này thế nào?

Mỗi trang có lý thuyết ngắn, ví dụ chạy thật và lỗi hay gặp. Học đến đâu, gõ lại code đến đó.

Mục tiêu của bạn	Nên đọc
Qua môn Hệ điều hành	**Phần 1 đến 7, rồi phần 10**
Sắp thi cuối kỳ	**Phần 10, rồi phần 3, 5, 6**
Dùng Linux thành thạo	**Phần 9, rồi phần 7**
Phỏng vấn backend, DevOps	**Phần 2, 4, 6, 8**
Hiểu vì sao máy chậm	**Phần 6, rồi phần 9**

Dòng ghi trên kết quả	Ý nghĩa
KẾT QUẢ KHI CHẠY	Python, chạy được trên mọi máy
KẾT QUẢ CHẠY TRÊN LINUX	Cần Linux hoặc WSL, vì dùng fork, /proc
KẾT QUẢ BIÊN DỊCH VÀ CHẠY	Mã C, biên dịch bằng gcc

Nội dung được đối chiếu với OSTEP, tài liệu kernel Linux, tài liệu Microsoft và đề cương của một số trường đại học. Mỗi trường có đề cương riêng, hãy so lại với chương trình của bạn.

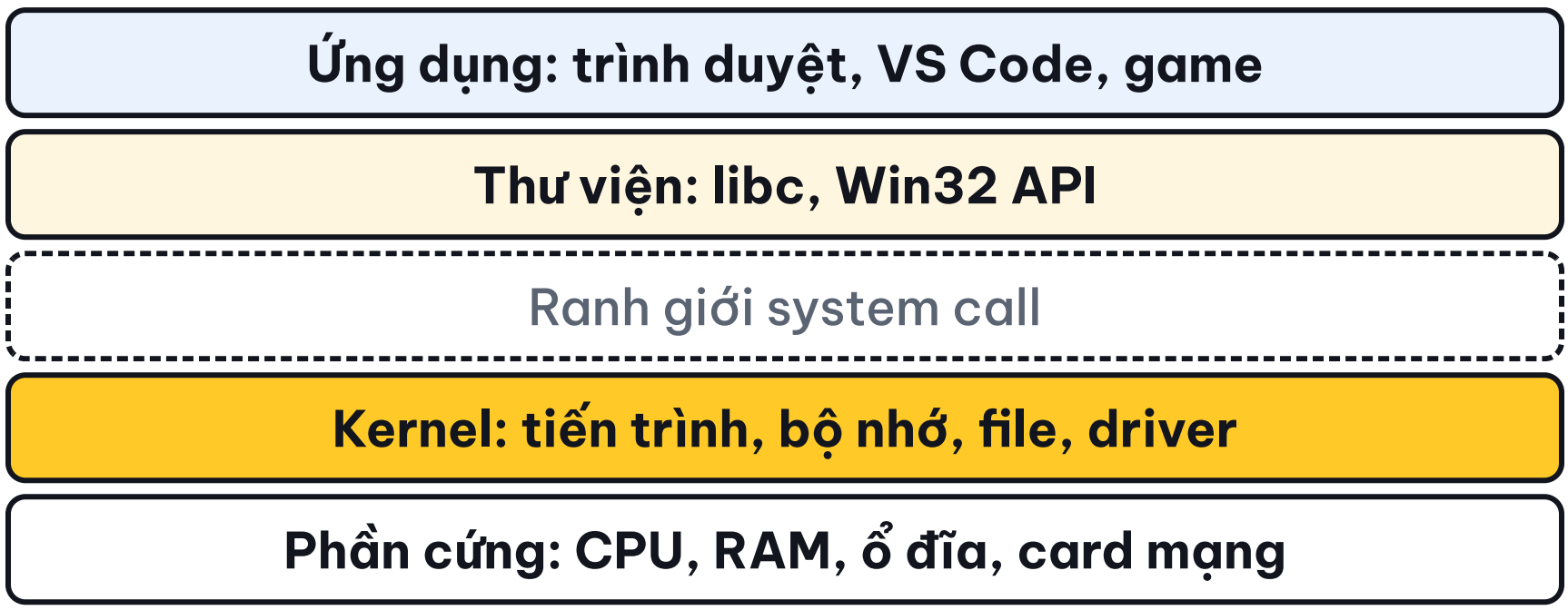
Một khái niệm, nhiều tên gọi

Tiếng Anh	Các tên hay gặp trong giáo trình Việt Nam
Process	Tiến trình, quá trình
Thread	Luồng, tiểu trình
Scheduling	Lập lịch, điều phối, định thời
Context switch	Chuyển ngữ cảnh
Race condition	Tranh chấp, tranh đoạt điều khiển
Critical section	Đoạn găng, miền găng, vùng tranh chấp
Semaphore	Đèn báo, cờ hiệu
Deadlock	Bế tắc, khóa chết, tắc nghẽn
Starvation	Đói tài nguyên, chết đói
Page, frame	Trang, khung trang
Page fault	Lỗi trang
Thrashing	Trì trệ, hiện tượng đổi trang liên tục
System call	Lời gọi hệ thống
Interrupt	Ngắt
Kernel	Nhân, hạt nhân

Đề thi có thể dùng tên khác với giáo trình bạn học. Hiểu bản chất thì đổi tên vẫn làm được.

1 Hệ điều hành là gì?

Phần mềm đứng giữa ứng dụng và phần cứng. Nó chia tài nguyên cho các chương trình và che đi sự phức tạp của máy.



Nhiệm vụ	Hệ điều hành làm gì
Quản lý tiến trình	Tạo, dừng, chia thời gian CPU
Quản lý bộ nhớ	Cấp RAM, tách vùng nhớ của từng chương trình
Quản lý file	Tổ chức dữ liệu trên ổ đĩa thành file, thư mục
Quản lý thiết bị	Làm việc với phần cứng qua driver
Bảo vệ	Chặn chương trình này phá chương trình khác

ẢO HÓA
Mỗi chương trình tưởng mình có CPU và RAM riêng.

ĐỒNG THỜI
Nhiều việc chạy cùng lúc mà không giẫm lên nhau.

Ý thứ ba là bền vững: dữ liệu phải còn nguyên sau khi tắt máy. Cả môn học xoay quanh ba ý này.

2 Vì sao tốc độ phần cứng quyết định mọi thứ?

CPU nhanh hơn ổ đĩa hàng triệu lần. Hệ điều hành phải tận dụng khoảng chờ đó để làm việc khác.

```
levels = [("Cache L1", 1), ("RAM", 100),
          ("SSD", 100_000), ("HDD", 10_000_000)]

def human(s):                          # nếu 1 ns dài bằng 1 giây
    return f"{s} giây" if s < 3600 else f"{s / 86400:.1f} ngày"

for name, ns in levels:
    print(f"{name:9} {ns:>11,} ns, phóng đại: {human(ns)}")
```

KẾT QUẢ KHI CHẠY

```
Cache L1             1 ns, phóng đại: 1 giây
RAM                100 ns, phóng đại: 100 giây
SSD            100,000 ns, phóng đại: 1.2 ngày
HDD         10,000,000 ns, phóng đại: 115.7 ngày
```

Tầng	Dung lượng thường gặp	Đặc điểm
Thanh ghi, cache	Vài KB đến vài chục MB	Nằm trong CPU
RAM	8 đến 64 GB	Mất dữ liệu khi tắt máy
SSD	256 GB đến vài TB	Không có bộ phận cơ khí
HDD	1 đến 20 TB	Rẻ, có đầu đọc phải di chuyển

- ✓ Chương trình đọc file thì CPU không ngồi chờ, mà chuyển sang chương trình khác.
- ✓ Dữ liệu vừa dùng được giữ lại ở tầng nhanh hơn. Đó là ý tưởng cache.
- ✓ Đổi từ HDD sang SSD là nâng cấp đáng tiền nhất cho máy cũ.

Các con số chỉ thể hiện bậc độ lớn, giá trị thật tùy từng máy.

3 Kernel mode và user mode

CPU có ít nhất hai chế độ. Chương trình thường chạy ở user mode, chỉ kernel mới được làm việc nguy hiểm.

Việc	User mode	Kernel mode
Tính toán thường	Được	Được
Đọc, ghi thiết bị	Không	Được
Đổi bảng trang	Không	Được
Tắt ngắt	Không	Được
Đọc RAM của tiến trình khác	Không	Được

VÌ SAO CẦN
Nếu ai cũng tắt được ngắt, một vòng lặp lỗi sẽ chiếm trọn CPU và treo cả máy.

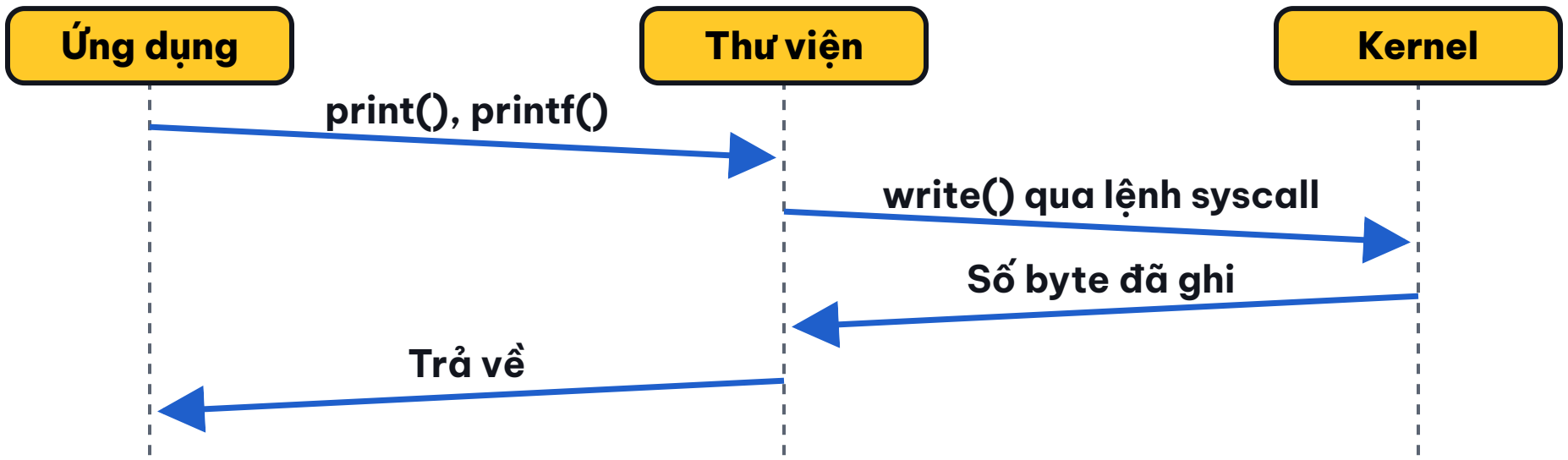
TRÊN X86
Kernel chạy ở ring 0, ứng dụng ở ring 3. Ring 1 và 2 hầu như không dùng.

Chương trình cố dùng lệnh đặc quyền ở user mode thì CPU báo ngoại lệ, và kernel thường kết thúc chương trình đó.

Driver chạy trong kernel mode nên driver lỗi có thể làm cả máy sập. Đó là một nguyên nhân hay gặp của màn hình xanh trên Windows.

4 System call

Cổng duy nhất để chương trình nhờ kernel làm việc. Mỗi lần print, mở file, gửi mạng đều đi qua đây.




```python
import os

flags = os.O_CREAT | os.O_WRONLY | os.O_TRUNC
fd = os.open("/tmp/hdh.txt", flags, 0o644)
print("Đã ghi:", os.write(fd, "xin chào\n".encode()), "byte")
os.close(fd)

fd = os.open("/tmp/hdh.txt", os.O_RDONLY)
print(os.read(fd, 100).decode().strip())
os.close(fd)
```

KẾT QUẢ CHẠY TRÊN LINUX

```
Đã ghi: 10 byte
xin chào
```

Nhóm	Linux	Windows
Tiến trình	fork, execve, wait, exit	CreateProcess, ExitProcess
File	open, read, write, close	CreateFile, ReadFile, WriteFile
Bộ nhớ	mmap, brk	VirtualAlloc

os.open, os.read gọi gần như trực tiếp system call cùng tên. Mỗi lần gọi đều phải vào kernel, nên đọc từng byte một rất chậm.

5 Ngắt, trap và ngoại lệ

Ba cách để CPU tạm dừng việc đang làm và nhảy vào kernel. Nhờ đó CPU không phải liên tục hỏi thiết bị.

Loại	Nguồn	Ví dụ
Ngắt phần cứng	Thiết bị, không đồng bộ	Bàn phím, timer, card mạng
Trap	Chương trình chủ động	System call
Ngoại lệ	Lệnh đang chạy bị lỗi	Chia cho 0, lỗi trang

1. **Lưu trạng thái**
   CPU cất con trỏ lệnh và các thanh ghi cần thiết.

2. **Tra bảng vector ngắt**
   Tìm địa chỉ hàm xử lý tương ứng.

3. **Chạy hàm xử lý**
   Kernel làm việc cần thiết ở kernel mode.

4. **Khôi phục**
   Quay lại chương trình cũ, hoặc chuyển sang tiến trình khác.

Ngắt timer là lý do một vòng lặp vô hạn không làm treo cả máy: cứ vài mili giây kernel lại được đánh thức và giành lại CPU.

6 Khởi động máy

Từ lúc bấm nút nguồn tới màn hình đăng nhập, từng lớp phần mềm nạp lớp tiếp theo.

`UEFI` → `Bootloader` → `Kernel` → `systemd` → `Đăng-nhập`

Bước	Việc xảy ra	Hỏng thì thấy gì
UEFI, BIOS	Kiểm tra phần cứng, tìm ổ khởi động	Không thấy ổ, báo lỗi RAM
Bootloader	GRUB hoặc Windows Boot Manager nạp kernel	Dừng ở màn hình grub
Kernel	Khởi tạo driver, gắn ổ gốc	Kernel panic, màn hình xanh
Tiến trình đầu tiên	Linux: systemd với PID 1	Dịch vụ không lên
Đăng nhập	Mở màn hình đăng nhập, phiên làm việc	Treo logo, màn hình đen

```
ps -p 1 -o pid=,comm=
systemctl --version | head -1 | cut -d' ' -f1
```

KẾT QUẢ CHẠY TRÊN LINUX

```
1 systemd
systemd
```

UEFI VÀ MBR

UEFI đi với bảng phân vùng GPT, thay cho BIOS đi với MBR. Cài song song Windows và Linux nên dùng UEFI cho cả hai.

SECURE BOOT

Chỉ cho bootloader đã ký chạy, chặn mã độc nạp trước hệ điều hành.

7 Kiến trúc kernel

Câu hỏi cốt lõi: dịch vụ nào chạy trong kernel, dịch vụ nào đưa ra ngoài?

Kiểu	Cách làm	Ví dụ
Nguyên khối	Mọi dịch vụ nằm chung trong kernel	Linux, FreeBSD
Vi nhân	Kernel chỉ giữ phần tối thiểu, driver chạy ở user mode	QNX, MINIX 3, seL4
Lai	Thiết kế theo vi nhân, nhưng nhiều dịch vụ vẫn ở kernel	Windows NT, XNU

NGUYÊN KHỐI
Nhanh vì gọi hàm trực tiếp. Lỗi một driver có thể kéo sập cả kernel.

VI NHÂN
Ổn định và cô lập tốt hơn. Tốn chi phí do truyền thông điệp giữa các phần.

Linux là kernel nguyên khối nhưng nạp được module khi đang chạy, nên driver không cần biên dịch sẵn trong kernel.

```
lsmod | head              # xem các module đang nạp
sudo modprobe ten_module # nạp thêm một module
uname -r                  # xem phiên bản kernel
```

Câu hỏi thi hay gặp: so sánh ưu, nhược điểm giữa nguyên khối và vi nhân, cho ví dụ từng loại.

8 Các hệ điều hành phổ biến

Khác nhau ở kernel, hệ thống file và nơi được dùng. Biết điểm chung thì học cái nào cũng nhanh.

Hệ điều hành	Kernel	Hệ file thường dùng	Mạnh ở
Windows	NT	NTFS	Máy cá nhân, doanh nghiệp
Ubuntu, Debian	Linux	ext4	Server, cloud, lập trình
macOS	XNU	APFS	Laptop, đồ họa, lập trình
Android	Linux	ext4, F2FS	Điện thoại
iOS	XNU	APFS	Điện thoại

LINUX CHỈ LÀ KERNEL
Ubuntu là bản phân phối: kernel cộng công cụ, trình quản lý gói và giao diện.

POSIX
Chuẩn chung về lời gọi và lệnh. Viết theo POSIX thì dễ chạy trên cả Linux và macOS.

- ✓ Sinh viên IT nên biết cả Windows và Linux. Server thực tế đa số chạy Linux.
- ✓ Windows dùng API riêng, không theo POSIX. WSL là cách dễ nhất để có Linux trên Windows.
- ✓ Android dùng kernel Linux nhưng phần trên khác hẳn một bản Linux trên máy tính.

9 Phân loại hệ điều hành

Hay ra ở chương mở đầu. Mỗi loại ra đời để giải quyết nhược điểm của loại trước.

Loại	Ý tưởng chính
Xử lý theo lô	Gom công việc, chạy lần lượt, không tương tác
Đa chương	Nhiều chương trình trong RAM, một cái chờ I/O thì chạy cái khác
Chia sẻ thời gian	Chia CPU thành lát nhỏ, người dùng thấy phản hồi ngay
Đa xử lý	Nhiều CPU dùng chung bộ nhớ
Phân tán	Nhiều máy phối hợp qua mạng
Thời gian thực	Đúng hạn quan trọng như đúng kết quả
Nhúng	Chạy trong thiết bị, tài nguyên hạn chế

Đa chương có lợi đến đâu? Nếu mỗi chương trình chờ I/O với tỉ lệ p, CPU bận khoảng $1 - p^n$.

```python
p = 0.8                                  # 80% thời gian chờ I/O
for n in (1, 2, 4, 8):
    print(f"{n} chương trình: CPU bận {1 - p ** n:.0%}")
```

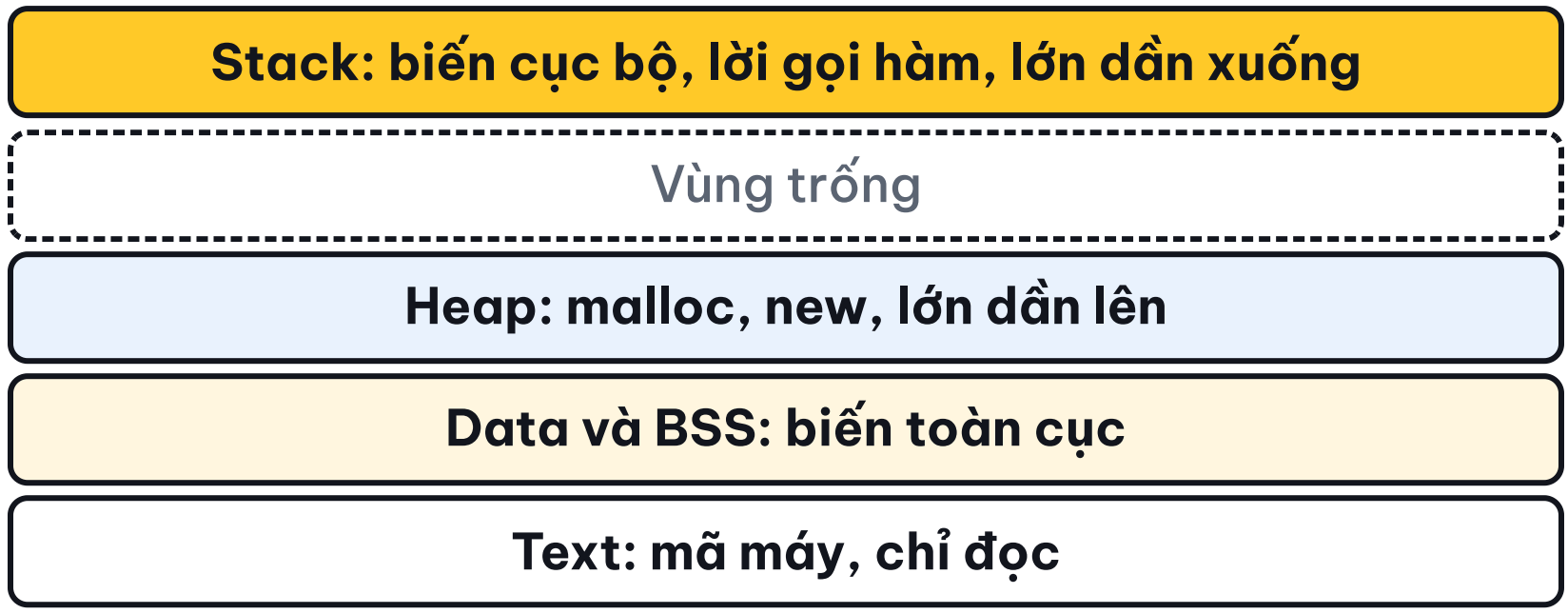
KẾT QUẢ KHI CHẠY

```
1 chương trình: CPU bận 20%
2 chương trình: CPU bận 36%
4 chương trình: CPU bận 59%
8 chương trình: CPU bận 83%
```

Mô hình xấp xỉ, giả sử các chương trình chờ I/O độc lập với nhau. Nó cho thấy vì sao thêm RAM để giữ nhiều chương trình lại giúp CPU bận rộn hơn.

10 Tiến trình là gì?

Chương trình là file nằm trên đĩa. Tiến trình là chương trình đang chạy, có bộ nhớ và trạng thái riêng.



Xem bố cục thật của một tiến trình Python, xếp theo địa chỉ từ thấp tới cao:

```python
first = {}
for line in open("/proc/self/maps"):
    f = line.split()
    if len(f) > 5:
        first.setdefault(f[5], int(f[0].split("-")[0], 16))

keep = ("python", "[heap]", "[stack]")
names = [k for k in first if any(w in k for w in keep)]
print(sorted(names, key=first.get))
```

KẾT QUẢ CHẠY TRÊN LINUX

```
['/usr/bin/python3.14', '[heap]', '[stack]']
```

MỘT CHƯƠNG TRÌNH, NHIỀU TIẾN TRÌNH

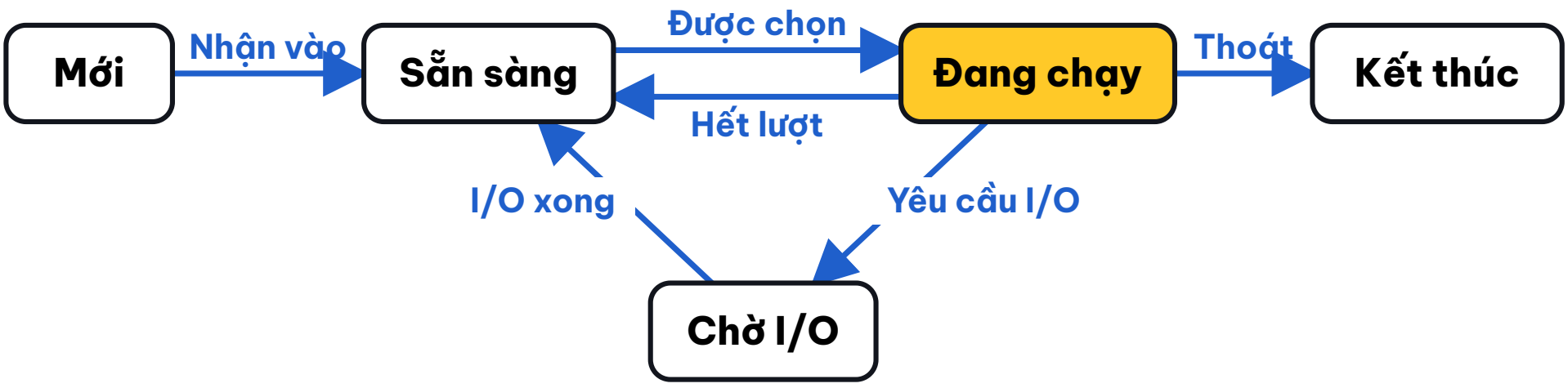
Mở hai cửa sổ VS Code là có nhiều tiến trình từ cùng một file.

CÁCH LY

Mỗi tiến trình có không gian địa chỉ riêng, nên lỗi của nó khó lan sang cái khác.

11 Trạng thái tiến trình và PCB

Mỗi tiến trình luôn ở một trạng thái. Kernel lưu mọi thông tin về nó trong khối điều khiển tiến trình (PCB).



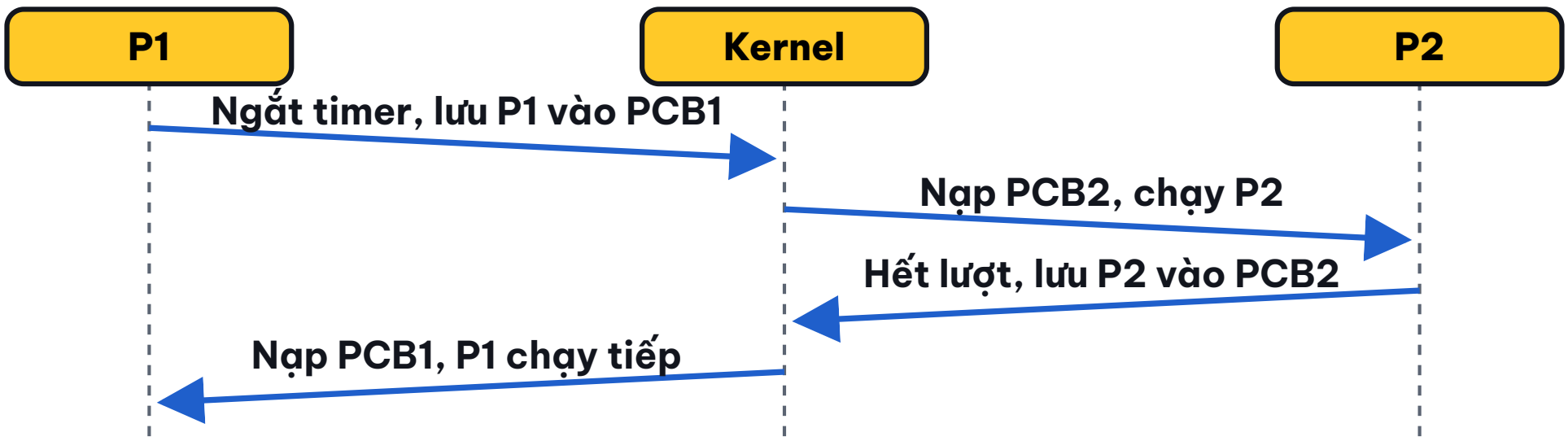
Trong PCB có	Dùng để
PID, tiến trình cha	Định danh
Trạng thái, độ ưu tiên	Lập lịch
Con trỏ lệnh, thanh ghi	Chạy tiếp đúng chỗ cũ
Bảng trang	Quản lý bộ nhớ
Bảng file đang mở	Quản lý I/O

Mã trong ps	Nghĩa trên Linux
R	Đang chạy hoặc sẵn sàng
S	Ngủ, đánh thức được
D	Ngủ không ngắt được, thường do chờ đĩa
T, Z	Bị dừng, zombie

Trên Linux, PCB là cấu trúc task_struct trong mã nguồn kernel.

12 Chuyển ngữ cảnh

Muốn chạy tiến trình khác, kernel lưu trạng thái cái cũ vào PCB rồi nạp trạng thái cái mới.




```python
import time

def switches():
    for line in open("/proc/self/status"):
        if line.startswith("voluntary"):
            return int(line.split()[1])

before = switches()
for _ in range(5):
    time.sleep(0.01)                # tự nhường CPU
print("Số lần tự nhường CPU:", switches() - before)
```

KẾT QUẢ CHẠY TRÊN LINUX

```
Số lần tự nhường CPU: 5
```

CHI PHÍ TRỰC TIẾP

Lưu, nạp thanh ghi và chạy mã lập lịch.

CHI PHÍ GIÁN TIẾP

Cache và TLB nguội dần, tiến trình mới chạy chậm một lúc.

Lúc chuyển ngữ cảnh, CPU không làm được việc có ích. Giả sử mỗi lần chuyển mất 0,01 ms: lượt 0,1 ms thì tốn 9% CPU, lượt 10 ms chỉ tốn 0,1%.

13 fork, exec, wait

fork nhân đôi tiến trình, exec thay bằng chương trình mới, wait đợi con kết thúc. fork trả về 0 ở con, PID của con ở cha, -1 khi lỗi.

```python
import os

print("Bắt đầu")
pid = os.fork()                          # từ đây có hai tiến trình
if pid == 0:                             # con nhận giá trị 0
    os.execvp("echo", ["echo", "Con đã thành chương trình echo"])

_, status = os.waitpid(pid, 0)           # cha nhận PID của con
print("Con thoát với mã", os.waitstatus_to_exitcode(status))
```

KẾT QUẢ CHẠY TRÊN LINUX

```
Bắt đầu
Con đã thành chương trình echo
Con thoát với mã 0
```

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();
    if (pid == 0) {                      // tiến trình con
        execlp("ls", "ls", "-l", NULL);
        return 1;                        // chỉ tới đây khi exec lỗi
    }
    wait(NULL);                          // tiến trình cha
    printf("Con đã xong\n");
    return 0;
}
```

exec không tạo tiến trình mới: PID giữ nguyên, nội dung bộ nhớ được thay hoàn toàn. Windows gộp hai việc này trong CreateProcess.

14 Bài toán đếm fork

Dạng bài rất hay ra. Mỗi fork làm số tiến trình đang chạy tới đó nhân đôi, nên n lần fork liên tiếp cho 2^n tiến trình.

```
python3 -c 'import os
for i in range(3):
    os.fork()
print("x")' | wc -l

python3 -c 'import os
os.fork() and os.fork()
print("x")' | wc -l

python3 -c 'import os
print("A", end="")
os.fork()
print("B")' | cat
```

KẾT QUẢ CHẠY TRÊN LINUX

```
8
3
AB
AB
```

Đoạn C	Số dòng in	Vì sao
fork(); fork();	4	2 mũ 2
fork() && fork();	3	Con của fork đầu nhận 0 nên bỏ qua fork sau
fork() || fork();	3	Cha nhận số dương nên bỏ qua fork sau

Dòng AB in hai lần vì chữ A còn nằm trong bộ đệm khi fork, và con được chép cả bộ đệm. Muốn tránh thì flush trước khi fork.

15 Zombie và orphan

Con thoát mà cha chưa wait thì thành zombie. Cha thoát trước thì con mồ côi.

```python
import os, time

pid = os.fork()
if pid == 0:
    os._exit(0)                          # con thoát ngay
time.sleep(0.2)                          # cha chưa gọi wait
stat = open(f"/proc/{pid}/stat").read()
print("Trạng thái của con:", stat.split()[2])

os.waitpid(pid, 0)                       # cha thu dọn con
print("Con còn tồn tại?", os.path.exists(f"/proc/{pid}"))
```

KẾT QUẢ CHẠY TRÊN LINUX

```
Trạng thái của con: Z
Con còn tồn tại? False
```

	Zombie	Orphan
Xảy ra khi	Con đã thoát, cha chưa wait	Cha thoát khi con còn chạy
Còn chạy không	Không, chỉ còn một dòng trong bảng	Có
Ai dọn	Cha gọi wait	init hoặc subreaper nhận nuôi
Nguy hiểm	Nhiều zombie làm cạn PID	Thường vô hại

KILL KHÔNG DIỆT ĐƯỢC ZOMBIE
Nó đã chết rồi. Phải sửa cha, hoặc kết thúc cha để init dọn thay.

TÌM ZOMBIE

```
ps -eo pid,ppid,stat,comm |
grep ' Z'
```

16 Luồng

Luồng là đơn vị chạy bên trong tiến trình. Các luồng dùng chung bộ nhớ, nên tạo nhanh và trao đổi dễ, nhưng dễ lỗi.

Dùng chung giữa các luồng	Riêng từng luồng
Code, biến toàn cục, heap	Stack
File đang mở	Thanh ghi, con trỏ lệnh
Không gian địa chỉ	Trạng thái lập lịch

```
import threading

shared = []                          # mọi luồng cùng thấy

def work(name):
    local = name * 2                 # riêng trên stack của luồng
    shared.append(local)

threads = [threading.Thread(target=work, args=(c,)) for c in "ab"]
for t in threads: t.start()
for t in threads: t.join()
print(sorted(shared))
```

KẾT QUẢ KHI CHẠY

```
['aa', 'bb']
```

Tiêu chí	Tiến trình	Luồng
Bộ nhớ	Riêng	Chung
Tạo mới	Tốn hơn	Nhẹ hơn
Trao đổi dữ liệu	Phải qua IPC	Đọc thẳng biến chung
Một cái lỗi nặng	Cái khác vẫn sống	Cả tiến trình sập

17 Mô hình luồng

Luồng người dùng được ánh xạ lên luồng kernel theo ba cách.

Mô hình	Đặc điểm	Ví dụ
N : 1	Nhanh, nhưng một luồng chặn là cả nhóm dừng, không dùng được nhiều lõi	Green thread đời đầu
1 : 1	Chạy song song thật, tốn tài nguyên kernel hơn	pthread trên Linux, luồng Windows
M : N	Nhiều luồng nhẹ chạy trên ít luồng kernel	Goroutine, virtual thread của Java

ĐỒNG THỜI
Nhiều việc cùng tiến triển, có thể chỉ một lõi luân phiên.

SONG SONG
Nhiều việc chạy cùng một thời điểm, cần nhiều lõi.

```c
#include <pthread.h>

void *work(void *arg) { return NULL; }

int main(void) {
    pthread_t t;
    pthread_create(&t, NULL, work, NULL);   // tạo luồng
    pthread_join(t, NULL);                  // đợi luồng xong
    return 0;
}
```

Trên Linux, luồng và tiến trình đều là task trong kernel. Khác nhau ở chỗ task dùng chung những gì với nhau.

18 CPU-bound và I/O-bound

Chọn đúng mô hình phụ thuộc vào việc chương trình chờ CPU hay chờ I/O.

```
import threading, time

def io_task():
    time.sleep(0.3)                 # giả lập chờ mạng hoặc đĩa

start = time.perf_counter()
for _ in range(5):
    io_task()
print("Tuần tự:", round(time.perf_counter() - start, 1), "giây")

start = time.perf_counter()
threads = [threading.Thread(target=io_task) for _ in range(5)]
for t in threads: t.start()
for t in threads: t.join()
print("5 luồng:", round(time.perf_counter() - start, 1), "giây")
```

KẾT QUẢ KHI CHẠY

```
Tuần tự: 1.5 giây
5 luồng: 0.3 giây
```

Loại việc	Ví dụ	Nên dùng
I/O-bound	Gọi API, truy vấn CSDL	Luồng, async
CPU-bound	Xử lý ảnh, huấn luyện AI	Nhiều tiến trình, luồng thật

Bản Python thường có GIL: mỗi lúc chỉ một luồng chạy mã Python. Việc nặng CPU nên dùng multiprocessing.

Python 3.14 có bản free-threaded không dùng GIL, nhưng bản cài mặc định vẫn còn GIL. Nhiều thư viện như NumPy nhả GIL trong lúc tính toán.

19 Giao tiếp giữa tiến trình

Tiến trình không đọc được bộ nhớ của nhau, nên phải trao đổi qua cơ chế do kernel cung cấp.

Cơ chế	Đặc điểm	Hợp với
Pipe	Một chiều, dòng byte	Cha con, lệnh shell
Bộ nhớ chung	Nhanh nhất, phải tự đồng bộ	Dữ liệu lớn
Hàng đợi thông điệp	Gửi theo từng gói	Công việc rời rạc
Socket	Hai chiều, qua được mạng	Client và server
Tín hiệu	Chỉ báo sự kiện, gần như không có dữ liệu	Dừng, tải lại cấu hình

```python
import os

r, w = os.pipe()                    # hai đầu của ống
pid = os.fork()
if pid == 0:                        # con ghi
    os.close(r)
    os.write(w, "Xin chào từ tiến trình con".encode())
    os._exit(0)

os.close(w)                         # cha đọc
print(os.read(r, 100).decode())
os.waitpid(pid, 0)
```

KẾT QUẢ CHẠY TRÊN LINUX

```
Xin chào từ tiến trình con
```

Quên đóng đầu ghi thì bên đọc không biết đã hết dữ liệu và treo mãi. Đây là lỗi hay gặp nhất khi làm lab pipe.

20 Sau fork, cái gì còn dùng chung?

Sau fork, con có bản sao riêng của bộ nhớ. Muốn dùng chung thật thì phải tạo vùng nhớ chung từ trước.

```
import mmap, os

normal = [0]                          # biến thường
shared = mmap.mmap(-1, 1)             # vùng nhớ chung 1 byte

pid = os.fork()
if pid == 0:
    normal[0] = 1                     # con sửa bản sao của mình
    shared[0] = 1                     # con sửa vùng dùng chung
    os._exit(0)

os.waitpid(pid, 0)
print("Biến thường:", normal[0])
print("Bộ nhớ chung:", shared[0])
```

KẾT QUẢ CHẠY TRÊN LINUX

```
Biến thường: 0
Bộ nhớ chung: 1
```

Thứ	Sau fork
Biến, heap, stack	Bản sao riêng, sửa không ảnh hưởng nhau
File đang mở	Dùng chung vị trí đọc ghi
Vùng mmap kiểu chung	Dùng chung thật

Copy-on-write: fork không chép RAM ngay. Hai bên dùng chung trang, ai ghi thì trang đó mới được chép, nên fork rất nhanh.

21 Tín hiệu

Thông báo ngắn gửi tới tiến trình. Ctrl+C, lệnh kill và lỗi segfault đều dùng tín hiệu.

Tín hiệu	Số	Khi nào	Mặc định
SIGINT	2	Bấm Ctrl+C	Kết thúc
SIGTERM	15	Lệnh kill thường	Kết thúc
SIGKILL	9	kill -9	Kết thúc, không chặn được
SIGSEGV	11	Truy cập sai bộ nhớ	Kết thúc, core dump
SIGTSTP	20	Bấm Ctrl+Z	Tạm dừng

```python
import os, signal

def bye(num, frame):
    print(f"Nhận {signal.Signals(num).name}, dọn dẹp rồi thoát")

signal.signal(signal.SIGTERM, bye)
os.kill(os.getpid(), signal.SIGTERM)

try:
    signal.signal(signal.SIGKILL, bye)
except OSError as e:
    print("Bắt SIGKILL:", e.strerror)
```

KẾT QUẢ CHẠY TRÊN LINUX

```
Nhận SIGTERM, dọn dẹp rồi thoát
Bắt SIGKILL: Invalid argument
```

Dùng kill thường trước để chương trình kịp lưu dữ liệu. Chưa thoát mới dùng -9.

22 Lập lịch CPU là gì?

Nhiều tiến trình sẵn sàng mà số lõi có hạn. Bộ lập lịch quyết định ai được chạy tiếp theo, trong bao lâu.

Cấp lập lịch	Quyết định	Tần suất
Dài hạn	Cho công việc nào vào hệ thống	Thưa
Trung hạn	Đưa tiến trình tạm ra đĩa, nạp lại	Vừa
Ngắn hạn	Tiến trình nào nhận CPU ngay bây giờ	Rất dày

KHÔNG CHIẾM QUYỀN
Non-preemptive. Tiến trình giữ CPU tới khi xong hoặc tự chờ I/O.

CHIẾM QUYỀN
Preemptive. Kernel lấy lại CPU giữa chừng nhờ ngắt timer.

Tiêu chí	Công thức
Thời gian hoàn thành	Lúc kết thúc - lúc đến
Thời gian chờ	Thời gian hoàn thành - thời gian chạy
Thời gian đáp ứng	Lần đầu được chạy - lúc đến
Thông lượng	Số tiến trình xong trên một đơn vị thời gian

Các trang sau dùng chung một đề: P1 đến lúc 0 chạy 7, P2 đến lúc 2 chạy 4, P3 đến lúc 4 chạy 1, P4 đến lúc 5 chạy 4.

23 FCFS

Đến trước chạy trước, không chiếm quyền. Dễ cài đặt nhất, nhưng việc ngắn có thể phải chờ việc dài.




```python
jobs = [("P1", 0, 7), ("P2", 2, 4), ("P3", 4, 1), ("P4", 5, 4)]

def fcfs(jobs):
    t, gantt = 0, []
    for name, arrive, burst in sorted(jobs, key=lambda j: j[1]):
        t = max(t, arrive)                  # CPU rảnh thì chờ việc đến
        gantt.append((name, t, t + burst))
        t += burst
    return gantt

def report(jobs, gantt):
    print("Gantt:", ", ".join(f"{n} {a}-{b}" for n, a, b in gantt))
    end = {n: b for n, a, b in gantt}       # đoạn cuối của mỗi việc
    waits = [end[n] - a - b for n, a, b in jobs]
    print("Chờ:", waits, "| trung bình:", sum(waits) / len(waits))

report(jobs, fcfs(jobs))
```

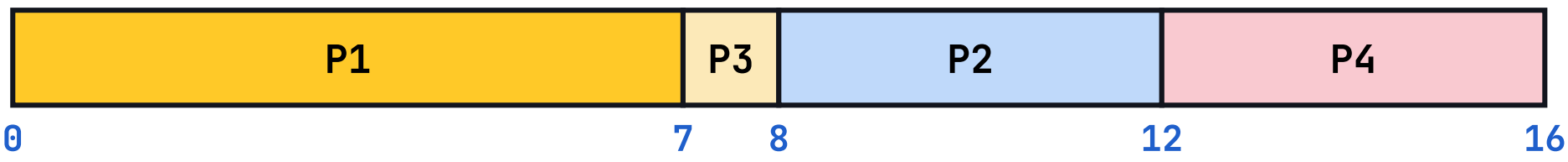
KẾT QUẢ KHI CHẠY

```
Chờ: [0, 5, 7, 7] | trung bình: 4.75
```

Hiệu ứng đoàn xe: ba việc dài 20, 2, 2 đến cùng lúc thì chờ trung bình 14. Đổi thứ tự thành 2, 2, 20 thì chỉ còn 2.

24 SJF

Trong các việc đã đến, chọn việc ngắn nhất. Không chiếm quyền. Khi mọi việc đến cùng lúc, thời gian chờ trung bình là nhỏ nhất.




```python
jobs = [("P1", 0, 7), ("P2", 2, 4), ("P3", 4, 1), ("P4", 5, 4)]

def sjf(jobs):
    left, t, gantt = list(jobs), 0, []
    while left:
        ready = [j for j in left if j[1] <= t]
        if not ready:
            t = min(j[1] for j in left)
            continue
        job = min(ready, key=lambda j: (j[2], j[1]))   # ngắn nhất
        gantt.append((job[0], t, t + job[2]))
        t += job[2]
        left.remove(job)
    return gantt

def report(jobs, gantt):
    print("Gantt:", ", ".join(f"{n} {a}-{b}" for n, a, b in gantt))
    end = {n: b for n, a, b in gantt}      # đoạn cuối của mỗi việc
    waits = [end[n] - a - b for n, a, b in jobs]
    print("Chờ:", waits, "| trung bình:", sum(waits) / len(waits))

report(jobs, sjf(jobs))
```

KẾT QUẢ KHI CHẠY

```
Chờ: [0, 6, 3, 7] | trung bình: 4.0
```

- Tại thời điểm 7 có P2, P3, P4 đang chờ, P3 ngắn nhất nên chạy trước.
- P2 và P4 cùng dài 4, xét việc đến trước. Đề thi thường nói rõ cách phá hòa.
- Nhược điểm: phải biết trước thời gian chạy, và việc dài có thể bị đói.

25 SRTF

SJF có chiếm quyền. Việc mới đến ngắn hơn phần còn lại của việc đang chạy thì giành CPU ngay.




```python
jobs = [("P1", 0, 7), ("P2", 2, 4), ("P3", 4, 1), ("P4", 5, 4)]

def srtf(jobs):
    left = {n: b for n, a, b in jobs}
    arrive = {n: a for n, a, b in jobs}
    t, gantt = 0, []
    while any(left.values()):
        ready = [n for n in left if left[n] and arrive[n] <= t]
        if ready:
            n = min(ready, key=lambda x: (left[x], arrive[x]))
            if gantt and gantt[-1][0] == n and gantt[-1][2] == t:
                gantt[-1] = (n, gantt[-1][1], t + 1)   # nối đoạn cũ
            else:
                gantt.append((n, t, t + 1))
            left[n] -= 1
        t += 1
    return gantt

def report(jobs, gantt):
    print("Gantt:", ", ".join(f"{n} {a}-{b}" for n, a, b in gantt))
    end = {n: b for n, a, b in gantt}      # đoạn cuối của mỗi việc
    waits = [end[n] - a - b for n, a, b in jobs]
    print("Chờ:", waits, "| trung bình:", sum(waits) / len(waits))

report(jobs, srtf(jobs))
```

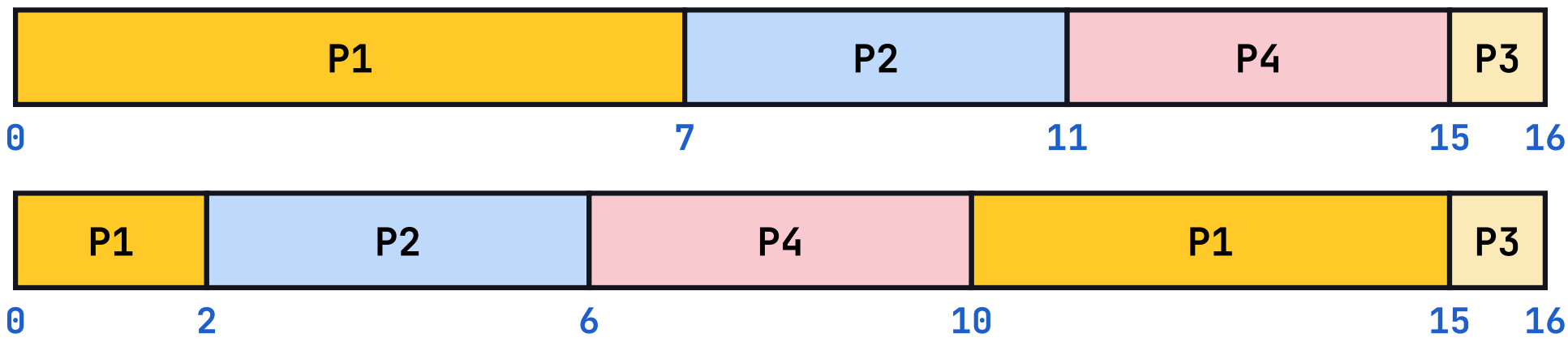
KẾT QUẢ KHI CHẠY

```
Chờ: [9, 1, 0, 2] | trung bình: 3.0
```

Mẹo: chỉ cần xét lại khi có việc mới đến hoặc việc vừa xong.

26 Lập lịch theo độ ưu tiên

Số cuối là độ ưu tiên, số nhỏ là ưu tiên cao. Đọc kỹ quy ước của đề.



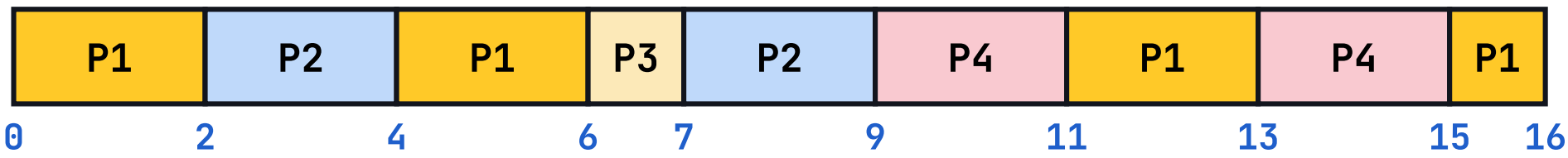

```
jobs = [("P1",0,7,3), ("P2",2,4,1), ("P3",4,1,4), ("P4",5,4,2)]

def priority(jobs, preempt):
    left = {j[0]: j[2] for j in jobs}
    info = {j[0]: j for j in jobs}
    t, cur, gantt = 0, None, []
    while any(left.values()):
        ready = [n for n in left if left[n] and info[n][1] <= t]
        if not ready:
            t += 1
            continue
        if preempt or cur is None or not left[cur]:
            cur = min(ready, key=lambda n: info[n][3])
        if gantt and gantt[-1][0] == cur and gantt[-1][2] == t:
            gantt[-1] = (cur, gantt[-1][1], t + 1)
        else:
            gantt.append((cur, t, t + 1))
        left[cur] -= 1
        t += 1
    return gantt
for flag in (False, True):
    g = priority(jobs, flag)
    print("Gantt:", ", ".join(f"{n} {a}-{b}" for n, a, b in g))
```

Hình trên không chiếm quyền, hình dưới có chiếm quyền. Việc ưu tiên thấp có thể bị đói, sửa bằng lão hóa (aging): tăng ưu tiên theo thời gian chờ.

27 Round Robin

Mỗi tiến trình chạy tối đa một lượt q rồi xếp xuống cuối hàng. Đây là nền tảng của hệ chia sẻ thời gian.




```python
from collections import deque
jobs = [("P1", 0, 7), ("P2", 2, 4), ("P3", 4, 1), ("P4", 5, 4)]

def rr(jobs, q):
    todo = sorted(jobs, key=lambda j: j[1])
    left = {n: b for n, a, b in jobs}
    queue, t, gantt = deque(), 0, []
    while todo or queue:
        while todo and todo[0][1] <= t:      # nhận các việc đã đến
            queue.append(todo.pop(0)[0])
        if not queue:
            t = todo[0][1]                   # CPU rảnh, chờ việc đến
            continue
        n = queue.popleft()
        run = min(q, left[n])
        gantt.append((n, t, t + run))
        t += run
        left[n] -= run
        while todo and todo[0][1] <= t:      # việc mới đến xếp trước
            queue.append(todo.pop(0)[0])
        if left[n]:
            queue.append(n)                  # việc hết lượt xếp sau
    return gantt

g = rr(jobs, 2)
print("Gantt:", ", ".join(f"{n} {a}-{b}" for n, a, b in g))
```

Quy ước hay gặp: việc đến đúng lúc hết lượt được xếp trước việc vừa bị ngắt. Một số giáo trình làm ngược lại, hãy theo đề.

28 Chọn lượt q bao nhiêu?

q quá nhỏ thì tốn chi phí chuyển ngữ cảnh. q quá lớn thì Round Robin trở thành FCFS.

KẾT QUẢ KHI CHẠY

```
q   | chờ TB | hoàn thành TB | số lần chuyển
1   |   5.50 |          9.50 | 14
2   |   5.00 |          9.00 | 8
3   |   7.00 |         11.00 | 7
4   |   4.50 |          8.50 | 4
8   |   4.75 |          8.75 | 3
```

q	Điều xảy ra
Rất nhỏ	Phản hồi mượt, nhưng CPU tốn thời gian chuyển ngữ cảnh
Vừa	Đa số việc ngắn xong trong một lượt
Rất lớn	Không ai bị ngắt, giống hệt FCFS

- ✓ Quy tắc kinh nghiệm: khoảng 80% lần dùng CPU nên ngắn hơn q.
- ✓ q càng nhỏ không có nghĩa thời gian chờ trung bình càng nhỏ, như bảng trên cho thấy.
- ✓ Round Robin không gây đói, vì ai cũng tới lượt sau tối đa (n - 1) × q.

Bảng trên do code chạy với đề chung, không tính chi phí chuyển ngữ cảnh.

29 So sánh các thuật toán

Cùng một đề, mỗi thuật toán tối ưu một thứ khác nhau. Không có thuật toán tốt nhất cho mọi trường hợp.

KẾT QUẢ KHI CHẠY

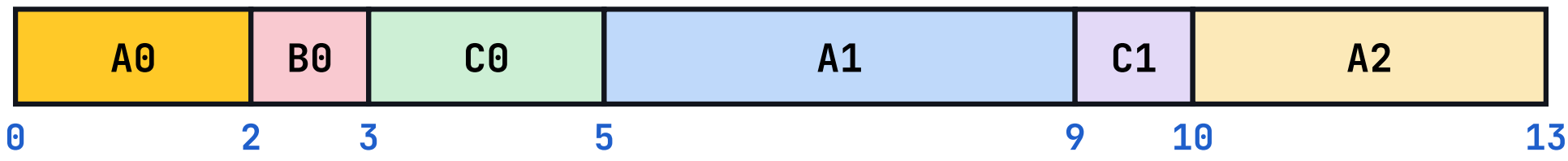
```
Thuật toán | chờ TB | hoàn thành TB | đáp ứng TB
FCFS       |   4.75 |          8.75 |   4.75
SJF        |   4.00 |          8.00 |   4.00
SRTF       |   3.00 |          7.00 |   0.50
RR q=2     |   5.00 |          9.00 |   1.50
```

Thuật toán	Chiếm quyền	Điểm mạnh	Điểm yếu
FCFS	Không	Đơn giản, công bằng theo thứ tự	Hiệu ứng đoàn xe
SJF	Không	Chờ trung bình thấp	Phải đoán thời gian chạy, gây đói
SRTF	Có	Chờ trung bình thấp nhất	Nhiều lần chuyển, gây đói
Ưu tiên	Cả hai	Việc quan trọng chạy trước	Gây đói nếu không lão hóa
RR	Có	Đáp ứng nhanh, công bằng	Hoàn thành thường chậm hơn

Máy tính cá nhân cần đáp ứng nhanh. Máy tính toán theo lô cần thông lượng cao. Mục tiêu khác nhau thì cách lập lịch khác nhau.

30 Hàng đợi đa cấp

MLFQ tự học hành vi: việc dùng hết lượt bị hạ cấp, việc ngắn hoặc tương tác giữ ưu tiên cao mà không cần biết trước thời gian chạy.




```python
from collections import deque

def mlfq(jobs, quanta=(2, 4, 8)):
    left = dict(jobs)
    levels = [deque(jobs), deque(), deque()]
    t, gantt = 0, []
    while any(levels):
        k = next(i for i, q in enumerate(levels) if q)  # cấp cao
        n = levels[k].popleft()
        run = min(quanta[k], left[n])
        gantt.append((f"{n}{k}", t, t + run))
        t += run
        left[n] -= run
        if left[n]:
            levels[min(k + 1, 2)].append(n)   # hạ cấp
    return gantt

g = mlfq({"A": 9, "B": 1, "C": 3})
print("Gantt:", ", ".join(f"{n} {a}-{b}" for n, a, b in g))
```

Kiểu	Đặc điểm
Nhiều hàng cố định	Tiến trình ở yên một hàng: hệ thống, tương tác, nền
Có phản hồi (MLFQ)	Tiến trình chuyển hàng theo hành vi thực tế

- ✓ Tên trên hình gồm tên việc và cấp, ví dụ A0 là A chạy ở cấp 0.
- ✓ Thực tế cần định kỳ đẩy mọi việc lên cấp cao, tránh việc dài bị đói.

31 Dự đoán lần chạy tiếp theo

SJF cần biết trước thời gian chạy. Hệ điều hành chỉ đoán được dựa trên quá khứ, bằng trung bình mũ.

Công thức: $\tau(n+1) = \alpha \cdot t(n) + (1 - \alpha) \cdot \tau(n)$. Trong đó t là thời gian chạy thật, τ là giá trị dự đoán.

```
def predict(bursts, guess, alpha=0.5):
    out = [guess]
    for t in bursts:
        guess = alpha * t + (1 - alpha) * guess
        out.append(guess)
    return out

print(predict([6, 4, 8, 12, 12], guess=10))
```

KẾT QUẢ KHI CHẠY

```
[10, 8.0, 6.0, 7.0, 9.5, 10.75]
```

α	Nghĩa
0	Bỏ qua dữ liệu mới, dự đoán không đổi
1	Chỉ tin vào lần chạy gần nhất
0,5	Cân bằng, thường dùng trong đề thi

Đây cũng là cách TCP ước lượng thời gian khứ hồi. Cùng một ý tưởng xuất hiện ở nhiều môn.

32 Lập lịch trong hệ thống thật

Giáo trình dạy thuật toán nền tảng, còn hệ thống thật kết hợp nhiều chính sách.

Hệ thống	Cách làm
Linux, việc thường	EEVDF, thay dần CFS từ kernel 6.6
Linux, thời gian thực	SCHED_FIFO, SCHED_RR, SCHED_DEADLINE
Linux, mở rộng	sched_ext từ 6.12: viết chính sách bằng BPF
Windows	Độ ưu tiên 0 đến 31, cùng mức thì chạy vòng tròn

```
nice -n 10 python3 -c 'import os; print("nice:", os.nice(0))'
taskset -c 0 python3 -c 'import os; print(os.sched_getaffinity(0))'
```

KẾT QUẢ CHẠY TRÊN LINUX

```
nice: 10
{0}
```

NICE
Từ -20 đến 19, số càng lớn càng nhường CPU. Hạ ưu tiên thì ai cũng làm được, tăng thì cần quyền root.

GẮN LÕI
Giữ tiến trình trên một lõi để cache còn ấm. Kernel vẫn cân bằng tải giữa các lõi.

- CPU mới có lõi hiệu năng và lõi tiết kiệm, nên kernel phải biết chọn lõi phù hợp.
- Windows tăng tạm ưu tiên cho luồng tương tác để giao diện đỡ giật.
- Đặt ưu tiên Realtime trên Windows có thể làm chuột, bàn phím bị đơ.

33 Lập lịch thời gian thực

Quan tâm tới hạn chót, không phải tốc độ trung bình. Trễ một lần có thể là thất bại.

THỜI GIAN THỰC CỨNG
Trễ hạn là lỗi nghiêm trọng: phanh ABS, túi khí.

THỜI GIAN THỰC MỀM
Trễ thì giảm chất lượng: gọi video, game.

```
tasks = [(1, 4), (2, 6), (3, 10)]      # (thời gian chạy, chu kỳ)

n = len(tasks)
u = sum(c / t for c, t in tasks)
bound = n * (2 ** (1 / n) - 1)
print(f"Mức dùng CPU U = {u:.3f}")
print(f"Ngưỡng đủ của RM = {bound:.3f}")
print("EDF chạy được:", u <= 1)
print("RM chắc chắn chạy được:", u <= bound)
```

KẾT QUẢ KHI CHẠY

```
Mức dùng CPU U = 0.883
Ngưỡng đủ của RM = 0.780
EDF chạy được: True
RM chắc chắn chạy được: False
```

Thuật toán	Quy tắc	Điều kiện
Rate Monotonic	Chu kỳ ngắn ưu tiên cao, cố định	U ≤ n(2^(1/n) - 1) là đủ
EDF	Hạn chót gần nhất chạy trước, thay đổi	U ≤ 1 là đủ và cần

U vượt ngưỡng của RM chưa có nghĩa là trễ hạn, chỉ là phải kiểm tra chi tiết hơn. Công thức EDF trên áp dụng khi hạn chót bằng chu kỳ.

34 Race condition

Lệnh x = x + 1 thật ra là ba bước: đọc, cộng, ghi. Hai luồng xen vào nhau thì mất một lần cộng.

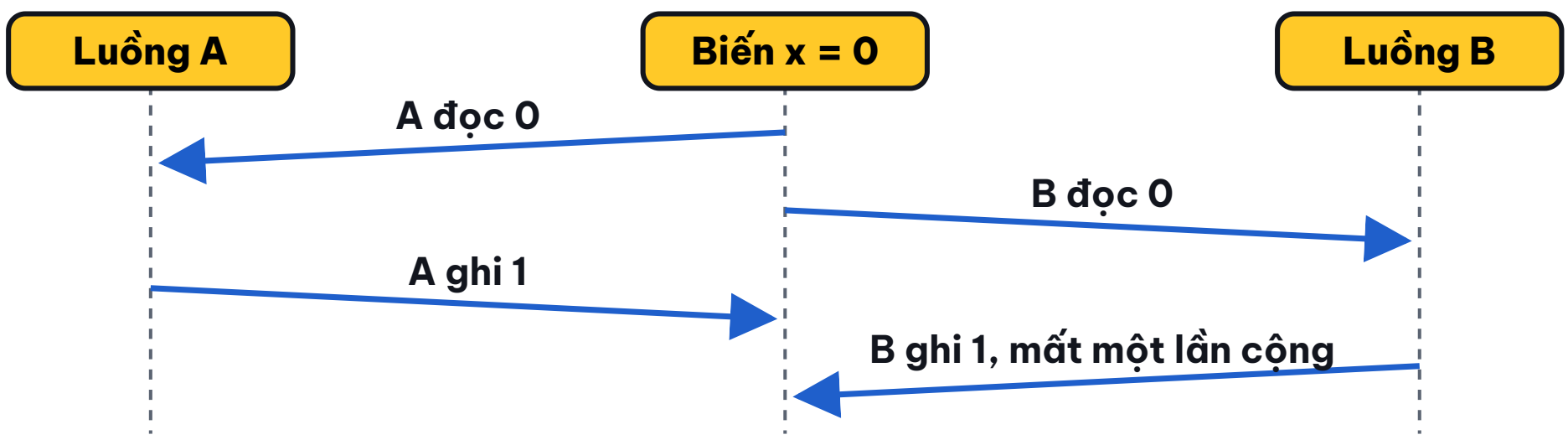
```
from itertools import combinations

def run(order):                          # order: thứ tự xen kẽ của A và B
    x, reg, step = 0, {}, {"A": 0, "B": 0}
    for who in order:
        if step[who] == 0:
            reg[who] = x                 # đọc
        elif step[who] == 1:
            reg[who] += 1                # cộng
        else:
            x = reg[who]                 # ghi
        step[who] += 1
    return x


count = {}
for pos in combinations(range(6), 3):
    order = "".join("A" if i in pos else "B" for i in range(6))
    count.setdefault(run(order), []).append(order)
print({k: len(v) for k, v in sorted(count.items())})
print("Ví dụ mất một lần cộng:", count[1][0])
```

KẾT QUẢ KHI CHẠY

```
{1: 18, 2: 2}
Ví dụ mất một lần cộng: AABABB
```



35 Race condition trong C

Chương trình thật: 2 luồng, mỗi luồng cộng 1.000.000 lần vào biến chung, không khóa. Kết quả đúng phải là 2.000.000.

```c
#include <stdio.h>
#include <pthread.h>

long x = 0;

void *add(void *arg) {
    for (int i = 0; i < 1000000; i++) x++;
    return NULL;
}

int main(void) {
    pthread_t a, b;
    pthread_create(&a, NULL, add, NULL);
    pthread_create(&b, NULL, add, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("%ld\n", x);
    return 0;
}
```

```
$ gcc -O0 -pthread race.c -o race
$ ./race     1015321
$ ./race     1000000
$ ./race     1411777
$ ./race     1015057
```

Chạy đúng 99 lần không có nghĩa là không có lỗi. Race condition phụ thuộc thời điểm, nên rất khó tái hiện và debug.

- ✓ In thêm log để debug có thể làm lỗi biến mất, vì thời điểm chạy đã đổi.
- ✓ Công cụ tìm lỗi: gcc -fsanitize=thread, hoặc Helgrind của Valgrind.

Bốn số trên ghi từ lần chạy thật. Máy khác sẽ ra số khác.

36 Đoạn găng

Đoạn code truy cập dữ liệu chung. Mọi lời giải phải đảm bảo chỉ một luồng ở trong đó tại một thời điểm.

Đoạn vào: xin phép

Đoạn găng: đọc, ghi dữ liệu chung

Đoạn ra: trả quyền

Phần còn lại: việc riêng

Yêu cầu	Ý nghĩa
Loại trừ tương hỗ	Không có hai luồng cùng trong đoạn găng
Tiến triển	Đoạn găng trống thì phải cho ai đó vào, không chờ mãi
Chờ có giới hạn	Không luồng nào phải chờ vô tận

- ✓ Giữ đoạn găng càng ngắn càng tốt, đừng gọi mạng hay đọc file trong đó.
- ✓ Tắt ngắt chỉ dùng được trên một lõi, và không bao giờ giao cho ứng dụng.
- ✓ Bài tập hay yêu cầu chỉ ra một lời giải vi phạm yêu cầu nào trong ba yêu cầu trên.

37 Thuật toán Peterson

Lời giải bằng phần mềm cho hai tiến trình i và j. Cờ cho biết ai muốn vào, biến lượt phân xử khi cả hai cùng muốn.

```
flag[i] = true;                          // tôi muốn vào
turn = j;                                // nhưng nhường lượt cho bạn
while (flag[j] && turn == j) ;           // chờ
/* đoạn găng */
flag[i] = false;                         // xong việc
```

Yêu cầu	Vì sao đạt
Loại trừ tương hỗ	turn chỉ mang một giá trị, nên chỉ một bên qua được vòng chờ
Tiến triển	Bên kia không muốn vào thì flag[j] tắt, đi thẳng vào
Chờ có giới hạn	Ra rồi muốn vào lại thì phải nhường lượt, nên bên kia chờ tối đa một lượt

38 Kiểm chứng bằng máy

Duyệt mọi cách xen kẽ của hai tiến trình, so Peterson với cách kiểm tra cờ trước rồi mới bật cờ.

```
def peterson(s, i):                  # s = (pc0, pc1, cờ0, cờ1, lượt)
    pc, flag, turn, j = list(s[:2]), list(s[2:4]), s[4], 1 - i
    if pc[i] == 2 and flag[j] and turn == j:
        return s                                  # chờ
    flag[i] = {0: 1, 3: 0}.get(pc[i], flag[i])
    turn = j if pc[i] == 1 else turn
    pc[i] = (pc[i] + 1) % 4                  # pc = 3 là trong đoạn găng
    return (*pc, *flag, turn)
def naive(s, i):                       # kiểm tra trước, bật cờ sau
    pc, flag, j = list(s[:2]), list(s[2:4]), 1 - i
    if pc[i] == 0 and flag[j]:
        return s
    flag[i] = {1: 1, 3: 0}.get(pc[i], flag[i])
    pc[i] = (pc[i] + 1) % 4
    return (*pc, *flag, 0)
def explore(step):
    start = (0, 0, 0, 0, 0)
    seen, todo = {start}, [start]
    while todo:
        s = todo.pop()
        if s[0] == s[1] == 3:
            return "hai bên cùng vào đoạn găng"
        for i in (0, 1):
            n = step(s, i)
            if n not in seen:
                seen.add(n)
                todo.append(n)
    return f"an toàn sau {len(seen)} trạng thái"
print("Kiểm tra rồi bật cờ:", explore(naive))
print("Peterson:", explore(peterson))
```

KẾT QUẢ KHI CHẠY

```
Kiểm tra rồi bật cờ: hai bên cùng vào đoạn găng
Peterson: an toàn sau 20 trạng thái
```

39 Lệnh nguyên tử của phần cứng

Lệnh đọc và ghi trong một bước không thể chen ngang. Mọi loại khóa xây dựng trên đó.

```c
#include <stdatomic.h>
#include <stdio.h>
#include <pthread.h>

atomic_flag lock = ATOMIC_FLAG_INIT;
long x = 0;

void *add(void *arg) {
    for (int i = 0; i < 1000000; i++) {
        while (atomic_flag_test_and_set(&lock)) ;   // spin
        x++;
        atomic_flag_clear(&lock);
    }
    return NULL;
}

int main(void) {
    pthread_t t[2];
    for (int i = 0; i < 2; i++) pthread_create(&t[i], NULL, add, NULL);
    for (int i = 0; i < 2; i++) pthread_join(t[i], NULL);
    printf("%ld\n", x);
    return 0;
}
```

KẾT QUẢ BIÊN DỊCH VÀ CHẠY

```
2000000
```

Lệnh	Làm gì
Test-and-set	Đặt cờ bằng 1 và trả về giá trị cũ, trong một bước
Compare-and-swap	Chỉ ghi nếu giá trị hiện tại đúng như mong đợi

40 Mutex và spinlock

Mutex cho luồng ngủ khi phải chờ. Spinlock cho luồng quay vòng liên tục.

```c
#include <stdio.h>
#include <pthread.h>

pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
long x = 0;
void *add(void *arg) {
    for (int i = 0; i < 1000000; i++) {
        pthread_mutex_lock(&m);       // vào đoạn găng
        x++;
        pthread_mutex_unlock(&m);     // ra đoạn găng
    }
    return NULL;
}

int main(void) {
    pthread_t t[2];
    for (int i = 0; i < 2; i++) pthread_create(&t[i], NULL, add, NULL);
    for (int i = 0; i < 2; i++) pthread_join(t[i], NULL);
    printf("%ld\n", x);
    return 0;
}
```

KẾT QUẢ BIÊN DỊCH VÀ CHẠY

```
2000000
```

Tiêu chí	Mutex	Spinlock
Khi chờ	Ngủ, nhường CPU	Quay vòng, chiếm CPU
Hợp với	Đoạn găng dài	Đoạn rất ngắn, trên nhiều lõi

Linux cài mutex bằng futex: không ai tranh chấp thì không cần vào kernel, nên khóa rất nhanh.

41 Semaphore

Một biến đếm nguyên với hai thao tác nguyên tử. wait giảm, nếu không còn thì ngủ. signal tăng và đánh thức.

```
import threading, time

slots = threading.Semaphore(2)        # tối đa 2 lượt tải cùng lúc
lock = threading.Lock()
now = peak = 0

def download():
    global now, peak
    with slots:                       # wait khi vào, signal khi ra
        with lock:
            now += 1
            peak = max(peak, now)
        time.sleep(0.05)
        with lock:
            now -= 1

threads = [threading.Thread(target=download) for _ in range(6)]
for t in threads: t.start()
for t in threads: t.join()
print("Nhiều nhất cùng lúc:", peak)
```

KẾT QUẢ KHI CHẠY

```
Nhiều nhất cùng lúc: 2
```

Giá trị đầu	Dùng để
1	Loại trừ tương hỗ, giống khóa
0	Báo hiệu thứ tự: A xong thì B mới chạy
N	Giới hạn N luồng dùng tài nguyên

Mutex có chủ: ai khóa thì người đó mở. Semaphore không có chủ: luồng này wait, luồng khác signal được. Sách cũ viết wait là P, signal là V.

42 Bài toán sản xuất, tiêu thụ

Dùng ba semaphore: empty đếm chỗ trống, full đếm phần tử, mutex bảo vệ bộ đệm.

```
import threading
from collections import deque

N, buffer, seen = 3, deque(), []
empty, full = threading.Semaphore(N), threading.Semaphore(0)
mutex = threading.Lock()
def producer():
    for item in range(20):
        empty.acquire()                 # chờ có chỗ trống
        with mutex:
            buffer.append(item)
        full.release()                  # báo có hàng
def consumer():
    for _ in range(20):
        full.acquire()                  # chờ có hàng
        with mutex:
            seen.append(buffer.popleft())
        empty.release()                 # báo có chỗ trống
ts = [threading.Thread(target=f) for f in (producer, consumer)]
for t in ts: t.start()
for t in ts: t.join()
print("Nhận đủ, đúng thứ tự:", seen == list(range(20)))
```

KẾT QUẢ KHI CHẠY

```
Nhận đủ, đúng thứ tự: True
```

Bên	Thứ tự trong bài thi
Sản xuất	wait(empty), wait(mutex), thêm, signal(mutex), signal(full)
Tiêu thụ	wait(full), wait(mutex), lấy, signal(mutex), signal(empty)

Đảo wait(mutex) lên trước wait(empty) sẽ gây bế tắc: bên sản xuất giữ khóa rồi ngủ, bên tiêu thụ không vào được.

43 Bài toán đọc, ghi

Nhiều người đọc cùng lúc được, người ghi phải ở một mình.

```
import threading
class RWLock:
    def __init__(self):
        self.readers = 0
        self.guard = threading.Lock()      # bảo vệ biến đếm
        self.room = threading.Lock()       # khóa phòng dữ liệu

    def start_read(self):
        with self.guard:
            self.readers += 1
            if self.readers == 1:          # người đọc đầu tiên
                self.room.acquire()
    def end_read(self):
        with self.guard:
            self.readers -= 1
            if self.readers == 0:          # người đọc cuối cùng
                self.room.release()

rw = RWLock()
rw.start_read()
rw.start_read()
print("2 người đọc, ghi được không:", rw.room.acquire(False))
rw.end_read()
rw.end_read()
print("0 người đọc, ghi được không:", rw.room.acquire(False))
```

KẾT QUẢ KHI CHẠY

```
2 người đọc, ghi được không: False
0 người đọc, ghi được không: True
```

Biến thể	Ưu tiên	Rủi ro
Thứ nhất	Người đọc	Người ghi có thể bị đói
Thứ hai	Người ghi	Người đọc có thể bị đói

44 Bữa ăn của triết gia

5 người, 5 đũa, cần 2 đũa để ăn. Code duyệt mọi trạng thái để tìm bế tắc.

```
N = 5
def find_deadlock(pick):                # pick(i): thứ tự lấy đũa của i
    start = (0,) * N                    # số đũa mỗi người đang cầm
    seen, todo = {start}, [start]
    while todo:
        s = todo.pop()
        held = {pick(i)[k] for i in range(N) for k in range(s[i])}
        moves = []
        for i in range(N):
            if s[i] == 2:               # ăn xong, trả đũa
                moves.append(s[:i] + (0,) + s[i + 1:])
            elif pick(i)[s[i]] not in held:
                moves.append(s[:i] + (s[i] + 1,) + s[i + 1:])
        if not moves:
            return s
        for m in moves:
            if m not in seen:
                seen.add(m)
                todo.append(m)
    return None
left_first = lambda i: (i, (i + 1) % N)
small_first = lambda i: tuple(sorted((i, (i + 1) % N)))
print("Ai cũng lấy đũa trái trước:", find_deadlock(left_first))
print("Lấy đũa số nhỏ trước:", find_deadlock(small_first))
```

KẾT QUẢ KHI CHẠY

```
Ai cũng lấy đũa trái trước: (1, 1, 1, 1, 1)
Lấy đũa số nhỏ trước: None
```

Cách tránh bế tắc	Phá điều kiện nào
Đánh số đũa, lấy số nhỏ trước	Chờ vòng tròn
Lấy cả hai đũa cùng lúc	Giữ và chờ

45 Monitor và biến điều kiện

Monitor gói dữ liệu và khóa vào một chỗ, nên khó dùng sai hơn semaphore. Biến điều kiện cho phép ngủ chờ một điều kiện.

```
import threading

class Box:                                  # một monitor nhỏ
    def __init__(self):
        self.cond = threading.Condition()
        self.item = None

    def put(self, x):
        with self.cond:
            while self.item is not None:    # dùng while, đừng dùng if
                self.cond.wait()
            self.item = x
            self.cond.notify_all()

    def take(self):
        with self.cond:
            while self.item is None:
                self.cond.wait()
            x, self.item = self.item, None
            self.cond.notify_all()
            return x
box = Box()
w = threading.Thread(target=lambda: [box.put(c) for c in "abc"])
w.start()
print([box.take() for _ in "abc"])
w.join()
```

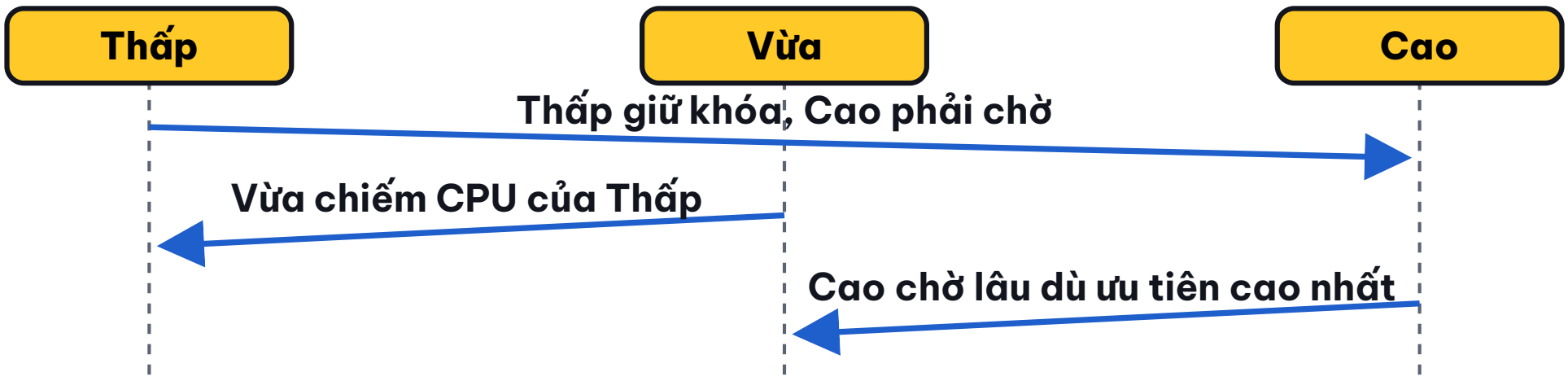
KẾT QUẢ KHI CHẠY

```
['a', 'b', 'c']
```

Dùng while vì khi tỉnh dậy, điều kiện có thể đã bị luồng khác đổi. Gặp ở synchronized của Java, Condition của Python, condition_variable của C++.

46 Đảo ngược ưu tiên

Việc ưu tiên cao phải chờ việc ưu tiên thấp đang giữ khóa, trong khi việc ưu tiên vừa chen vào chiếm CPU.



Năm 1997, tàu Mars Pathfinder trên Sao Hỏa liên tục tự khởi động lại vì lỗi này: tác vụ ưu tiên thấp giữ mutex, tác vụ vừa chen vào, tác vụ cao trễ hạn.

CÁCH SỬA
Kế thừa ưu tiên: ai giữ khóa thì tạm mang độ ưu tiên của người đang chờ khóa.

TRÊN TÀU
Đội JPL tái hiện lỗi dưới mặt đất, rồi bật cờ kế thừa ưu tiên từ xa.

Lỗi đồng bộ	Biểu hiện	Cách tránh
Race condition	Kết quả lúc đúng lúc sai	Khóa, kiểu nguyên tử
Deadlock	Chương trình đứng im	Khóa theo thứ tự cố định
Livelock	Chạy mãi mà không tiến triển	Thử lại sau thời gian ngẫu nhiên
Starvation	Một luồng không tới lượt	Hàng đợi công bằng
Lost wakeup	Ngủ mãi dù sự kiện đã xảy ra	Kiểm tra điều kiện trong khóa

47 Deadlock là gì?

Một nhóm tiến trình chờ nhau mãi mãi. Bế tắc xảy ra khi và chỉ khi cả bốn điều kiện Coffman cùng đúng.

```python
import threading

a, b = threading.Lock(), threading.Lock()
gate = threading.Barrier(2)

def work(first, second, name, out):
    with first:
        gate.wait()                          # cả hai đã giữ khóa đầu
        ok = second.acquire(timeout=0.2)     # để demo không treo
        out.append(f"{name}: {'được' if ok else 'kẹt ở khóa thứ hai'}")
        gate.wait()                          # chờ nhau thử xong

result = []
ts = [threading.Thread(target=work, args=(a, b, "T1", result)),
      threading.Thread(target=work, args=(b, a, "T2", result))]
for t in ts: t.start()
for t in ts: t.join()
print(sorted(result))
```

KẾT QUẢ KHI CHẠY

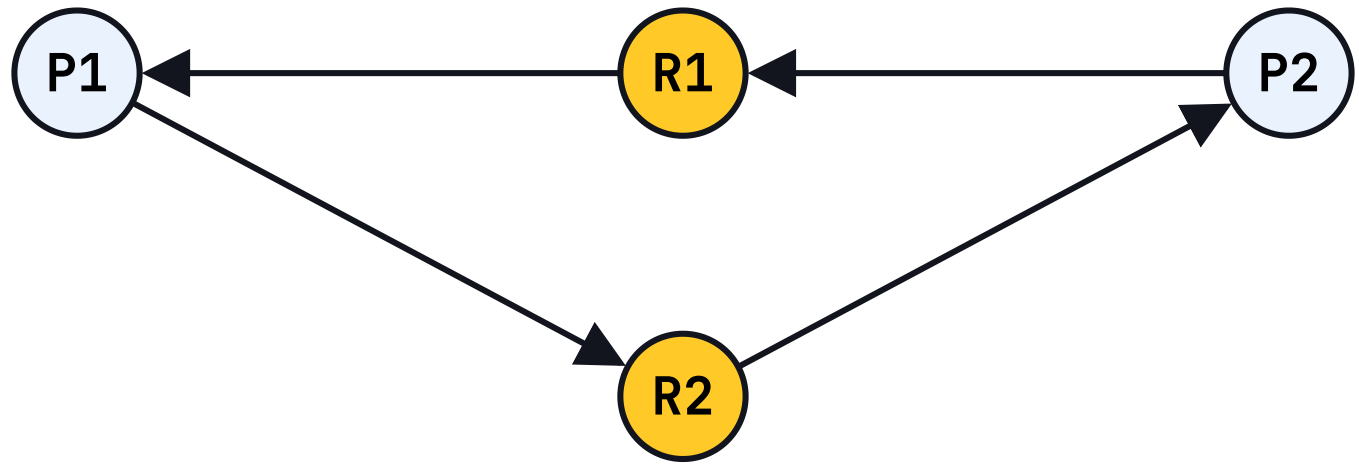
```
['T1: kẹt ở khóa thứ hai', 'T2: kẹt ở khóa thứ hai']
```

Điều kiện	Ý nghĩa
Loại trừ tương hỗ	Tài nguyên chỉ một người dùng tại một thời điểm
Giữ và chờ	Đang giữ cái này, chờ cái khác
Không chiếm đoạt	Không ai giật được tài nguyên đang bị giữ
Chờ vòng tròn	P1 chờ P2, P2 chờ P3, ..., Pn chờ P1

Đoạn code trên dùng timeout để demo kết thúc được. Không có timeout thì cả hai luồng treo vĩnh viễn.

48 Đồ thị cấp phát tài nguyên

Tài nguyên trỏ tới tiến trình là đang giữ. Tiến trình trỏ tới tài nguyên là đang xin.




```python
def has_cycle(graph):
    state = {}                                  # 1 là đang duyệt, 2 là xong
    def visit(v):
        state[v] = 1
        for w in graph.get(v, []):
            if state.get(w) == 1 or (w not in state and visit(w)):
                return True
        state[v] = 2
        return False
    return any(visit(v) for v in graph if v not in state)

print(has_cycle({"P1": ["P2"], "P2": ["P1"]}))
```

KẾT QUẢ KHI CHẠY

```
True
```

Đồ thị	Kết luận
Không có chu trình	Không có bế tắc
Có chu trình, mỗi tài nguyên 1 thực thể	Chắc chắn bế tắc
Có chu trình, tài nguyên nhiều thực thể	Có thể bế tắc, cần xét thêm

49 Các chiến lược xử lý

Có bốn hướng. Hệ điều hành phổ biến thường chọn bỏ qua và để ứng dụng tự lo.

Chiến lược	Cách làm
Ngăn chặn	Thiết kế sao cho một điều kiện Coffman luôn sai
Tránh	Xét từng yêu cầu, chỉ cấp khi vẫn an toàn (Banker)
Phát hiện, phục hồi	Cho xảy ra, kiểm tra định kỳ rồi xử lý
Bỏ qua	Coi như hiếm khi xảy ra, người dùng tự khởi động lại

Phá điều kiện	Cách làm	Nhược điểm
Loại trừ tương hỗ	Dùng spool cho máy in	Nhiều tài nguyên không chia sẻ được
Giữ và chờ	Xin tất cả một lần	Lãng phí, khó biết trước
Không chiếm đoạt	Xin không được thì trả hết	Không dùng được cho mọi tài nguyên
Chờ vòng tròn	Đánh số, xin theo thứ tự tăng	Phải tuân thủ trong toàn bộ code

Cách thực tế nhất khi lập trình: luôn khóa theo một thứ tự cố định. Chuyển tiền giữa hai tài khoản thì khóa tài khoản có mã nhỏ hơn trước.

50 Thuật toán Banker: kiểm tra an toàn

Trạng thái an toàn là khi tồn tại một thứ tự để mọi tiến trình đều chạy xong. Need = Max - Allocation.

```
def safe(avail, alloc, need):
    work, done, order = list(avail), [False] * len(alloc), []
    while True:
        for i, ok in enumerate(done):
            if not ok and all(n <= w for n, w in zip(need[i], work)):
                work = [w + a for w, a in zip(work, alloc[i])]
                done[i] = True
                order.append(f"P{i}")
                print(f"P{i} chạy xong, Work = {work}")
                break
        else:
            return all(done), order
alloc = [[1, 0, 1], [2, 1, 0], [1, 1, 2], [0, 1, 1]]
maxim = [[4, 2, 3], [3, 2, 2], [5, 3, 4], [1, 2, 2]]
need = [[m - a for m, a in zip(x, y)] for x, y in zip(maxim, alloc)]
print("Need =", need)
print(safe([1, 1, 1], alloc, need))
```

KẾT QUẢ KHI CHẠY

```
Need = [[3, 2, 2], [1, 1, 2], [4, 2, 2], [1, 1, 1]]
P3 chạy xong, Work = [1, 2, 2]
P1 chạy xong, Work = [3, 3, 2]
P0 chạy xong, Work = [4, 3, 3]
P2 chạy xong, Work = [5, 4, 5]
(True, ['P3', 'P1', 'P0', 'P2'])
```

- Tìm tiến trình có Need ≤ Work. Khi nó xong, cộng Allocation của nó vào Work.
- Lặp lại đến khi xong hết thì an toàn. Không tìm được ai thì không an toàn.
- Không an toàn chưa có nghĩa là đã bế tắc, chỉ là có nguy cơ.

51 Thuật toán Banker: xét yêu cầu

Có yêu cầu mới thì thử cấp trên giấy, an toàn mới cấp thật.

```
def safe(avail, alloc, need):
    work, done = list(avail), [False] * len(alloc)
    while True:
        for i, ok in enumerate(done):
            if not ok and all(n <= w for n, w in zip(need[i], work)):
                work = [w + a for w, a in zip(work, alloc[i])]
                done[i] = True
                break
        else:
            return all(done)
def request(i, req, avail, alloc, need):
    if any(r > n for r, n in zip(req, need[i])):
        return "Lỗi: xin vượt quá khai báo"
    if any(r > a for r, a in zip(req, avail)):
        return "Chờ: chưa đủ tài nguyên"
    sub = lambda x, y: [a - b for a, b in zip(x, y)]
    alloc2, need2 = [r[:] for r in alloc], [r[:] for r in need]
    alloc2[i] = sub(alloc[i], [-r for r in req])     # thử cấp
    need2[i] = sub(need[i], req)
    ok = safe(sub(avail, req), alloc2, need2)
    return "Cấp" if ok else "Từ chối: không an toàn"
avail = [1, 1, 1]
alloc = [[1, 0, 1], [2, 1, 0], [1, 1, 2], [0, 1, 1]]
need = [[3, 2, 2], [1, 1, 2], [4, 2, 2], [1, 1, 1]]
print("P3 xin 1 0 0:", request(3, [1, 0, 0], avail, alloc, need))
print("P1 xin 0 0 1:", request(1, [0, 0, 1], avail, alloc, need))
print("P0 xin 2 0 0:", request(0, [2, 0, 0], avail, alloc, need))
```

KẾT QUẢ KHI CHẠY

```
P3 xin 1 0 0: Cấp
P1 xin 0 0 1: Từ chối: không an toàn
P0 xin 2 0 0: Chờ: chưa đủ tài nguyên
```

52 Phát hiện và phục hồi

Giống Banker nhưng dùng yêu cầu hiện tại thay cho Need. Tiến trình không kết thúc được là tiến trình đang bế tắc.

```python
def deadlocked(avail, alloc, req):
    work = list(avail)
    done = [not any(row) for row in alloc]    # không giữ gì
    changed = True
    while changed:
        changed = False
        for i, ok in enumerate(done):
            if not ok and all(r <= w for r, w in zip(req[i], work)):
                work = [w + a for w, a in zip(work, alloc[i])]
                done[i] = changed = True
    return [f"P{i}" for i, x in enumerate(done) if not x]

alloc = [[0, 1], [1, 0], [1, 1]]
print(deadlocked([0, 0], alloc, [[0, 0], [1, 1], [1, 0]]))
print(deadlocked([0, 0], alloc, [[1, 0], [0, 1], [0, 0]]))
```

KẾT QUẢ KHI CHẠY

```
['P1', 'P2']
[]
```

Cách phục hồi	Cân nhắc
Kết thúc tất cả	Nhanh, nhưng mất nhiều việc đã làm
Kết thúc từng cái	Chọn cái ít thiệt hại, kiểm tra lại sau mỗi lần
Thu hồi tài nguyên	Cần quay lui về trạng thái an toàn

Cơ sở dữ liệu làm đúng như vậy. PostgreSQL phát hiện bế tắc rồi hủy một giao dịch, ứng dụng nhận lỗi và phải thử lại.

53 Địa chỉ logic và địa chỉ vật lý

Chương trình dùng địa chỉ logic. Phần cứng MMU dịch sang địa chỉ vật lý trong RAM ở mọi lần truy cập.




```
def mmu(logical, base, limit):
    if logical >= limit:                # ra ngoài vùng được cấp
        return "Lỗi truy cập, báo cho kernel"
    return base + logical

print(mmu(100, base=14000, limit=3000))
print(mmu(3500, base=14000, limit=3000))
```

KẾT QUẢ KHI CHẠY

```
14100
Lỗi truy cập, báo cho kernel
```

Gắn địa chỉ lúc	Đặc điểm
Biên dịch	Địa chỉ cố định, phải biết trước vị trí nạp
Nạp chương trình	Tính lại địa chỉ khi nạp, sau đó không đổi
Thực thi	Dịch ở mọi lần truy cập, chuyển chỗ được, cần MMU

VÌ SAO CẦN

Hai chương trình cùng dùng địa chỉ 0x1000 mà không đụng nhau, vì chúng nằm ở chỗ khác nhau trong RAM.

BẢO VỆ

Vượt giới hạn thì phần cứng báo lỗi ngay. Hệ điều hành hiện đại dùng phân trang thay cho cặp base, limit.

54 Cấp phát liên tục

Mỗi tiến trình nằm trong một khối liền nhau. Khi có nhiều lỗ trống, chọn lỗ nào?

```python
def allocate(holes, reqs, pick):
    holes, result = list(holes), []
    for size in reqs:
        fit = [i for i, h in enumerate(holes) if h >= size]
        if not fit:
            result.append("chờ")
            continue
        i = pick(fit, holes)
        result.append(holes[i])       # lỗ được chọn
        holes[i] -= size
    return result
first = lambda fit, h: fit[0]
best = lambda fit, h: min(fit, key=lambda i: h[i])
worst = lambda fit, h: max(fit, key=lambda i: h[i])
holes, reqs = [120, 480, 200, 320, 600], [210, 400, 110, 430]
for name, f in (("First", first), ("Best", best), ("Worst", worst)):
    print(f"{name:5}", *allocate(holes, reqs, f))
```

KẾT QUẢ KHI CHẠY

```
First 480 600 120 chờ
Best  320 480 110 600
Worst 600 480 390 chờ
```

Chiến lược	Chọn lỗ	Nhận xét
First fit	Đầu tiên đủ lớn	Nhanh, thường đủ tốt
Best fit	Nhỏ nhất đủ chứa	Để lại nhiều mảnh rất nhỏ
Worst fit	Lớn nhất	Giữ mảnh còn lại lớn, nhưng thường kém

PHÂN MẢNH NGOÀI
Tổng trống đủ nhưng rải rác. Sửa bằng dồn bộ nhớ, phân trang.

PHÂN MẢNH TRONG
Khối cấp lớn hơn nhu cầu, phần dư bị bỏ phí.

55 Buddy và slab

Kernel Linux cấp RAM theo hai lớp: buddy cho các khối trang, slab cho các đối tượng nhỏ hay dùng.

```
def buddy_size(need, page=4096):
    size = page
    while size < need:
        size *= 2                    # luôn là lũy thừa của 2
    return size

for need in (3000, 5000, 70000):
    got = buddy_size(need)
    print(f"Xin {need}: cấp {got}, phí {got - need} byte")
```

KẾT QUẢ KHI CHẠY

```
Xin 3000: cấp 4096, phí 1096 byte
Xin 5000: cấp 8192, phí 3192 byte
Xin 70000: cấp 131072, phí 61072 byte
```

Cơ chế	Cách làm	Được và mất
Buddy	Chia đôi khối tới khi vừa, trả về thì gộp với khối kề	Gộp nhanh, nhưng có phân mảnh trong
Slab	Giữ sẵn kho đối tượng cùng cỡ, như task_struct	Cấp rất nhanh, ít phí

Ở ứng dụng, malloc của C và new của Java xin một vùng lớn từ kernel, rồi tự chia nhỏ. Không phải lần malloc nào cũng là system call.

Xem bằng cat /proc/buddyinfo, và sudo cat /proc/slabinfo trên Linux.

56 Phân trang

Chia bộ nhớ logic thành trang, RAM thành khung cùng kích thước. Trang nằm ở khung nào cũng được, nên hết phân mảnh ngoài.




```
PAGE_BITS = 12                          # trang 4 KB
table = {0: 5, 1: 2, 2: 7, 3: 1}        # trang đến khung

def translate(addr):
    page = addr >> PAGE_BITS
    offset = addr & ((1 << PAGE_BITS) - 1)
    if page not in table:
        return f"{addr}: lỗi trang"
    physical = table[page] << PAGE_BITS | offset
    return f"{addr}: trang {page}, độ dời {offset} -> {physical}"

for a in (100, 4100, 12300, 20000):
    print(translate(a))
```

KẾT QUẢ KHI CHẠY

```
100: trang 0, độ dời 100 -> 20580
4100: trang 1, độ dời 4 -> 8196
12300: trang 3, độ dời 12 -> 4108
20000: lỗi trang
```

```
getconf PAGESIZE
```

KẾT QUẢ CHẠY TRÊN LINUX

```
4096
```

- ✓ Độ dời giữ nguyên, chỉ số trang được thay bằng số khung.
- ✓ Trang là lũy thừa của 2 nên tách bằng phép dịch bit, không cần chia.
- ✓ Vẫn có phân mảnh trong ở trang cuối của mỗi tiến trình.
- ✓ Mỗi tiến trình có bảng trang riêng, kernel đổi bảng khi chuyển ngữ cảnh.

57 Tính toán phân trang

Dạng bài chắc chắn có. Đổi hết về lũy thừa của 2 rồi trừ số mũ.

```python
from math import log2

def paging(addr_bits, page_size, entry, ram):
    d = int(log2(page_size))
    p = addr_bits - d
    print("Độ dời:", d, "bit | số trang: 2 mũ", p, "bit")
    print("Số khung RAM:", ram // page_size)
    print("Bảng trang:", (2 ** p * entry) // 2 ** 20, "MB")

paging(addr_bits=32, page_size=4096, entry=4, ram=2 ** 30)
```

KẾT QUẢ KHI CHẠY

```
Độ dời: 12 bit | số trang: 2 mũ 20 bit
Số khung RAM: 262144
Bảng trang: 4 MB
```

Đại lượng	Công thức
Số bit độ dời	$\log_2$(kích thước trang)
Số trang	2^(số bit địa chỉ logic - số bit độ dời)
Số khung	RAM / kích thước trang
Cỡ bảng trang	Số trang × cỡ một mục
Phân mảnh trong tối đa	Gần một trang, trung bình nửa trang

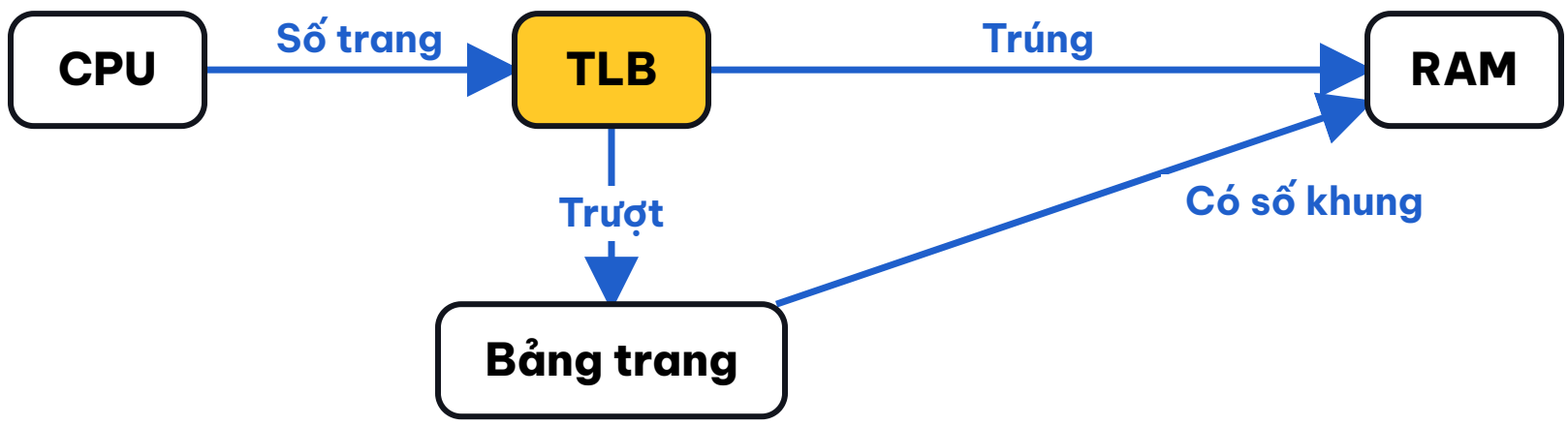
TRANG LỚN
Bảng trang nhỏ, đọc đĩa hiệu quả, nhưng phân mảnh trong nhiều hơn.

TRANG NHỎ
Ít lãng phí, nhưng bảng trang lớn. Thực tế dùng 4 KB, cùng các huge page 2 MB, 1 GB.

Mỗi tiến trình cần một bảng 4 MB thì quá phí. Vì thế mới có bảng trang nhiều cấp.

58 TLB

Bảng trang nằm trong RAM, nên mỗi lần đọc phải vào RAM hai lần. TLB là cache nhỏ trong CPU giúp bỏ bớt một lần.




```
def eat(hit, tlb, mem):
    return hit * (tlb + mem) + (1 - hit) * (tlb + 2 * mem)

for h in (0.8, 0.9, 0.98, 0.99):
    print(f"Tỉ lệ trúng {h:.0%}: {eat(h, tlb=10, mem=100):.0f} ns")
```

KẾT QUẢ KHI CHẠY

```
Tỉ lệ trúng 80%: 130 ns
Tỉ lệ trúng 90%: 120 ns
Tỉ lệ trúng 98%: 112 ns
Tỉ lệ trúng 99%: 111 ns
```

EAT = h × (t + m) + (1 - h) × (t + 2m). Trong đó h là tỉ lệ trúng, t là thời gian tra TLB, m là một lần truy cập RAM.

- ✓ Một số đề bỏ qua t. Đọc kỹ đề có tính thời gian tra TLB hay không.
- ✓ Chuyển sang tiến trình khác thì mục TLB cũ không còn đúng, phải xóa hoặc gắn nhãn ASID.
- ✓ Chương trình truy cập bộ nhớ gần nhau thì tỉ lệ trúng cao, nhờ tính cục bộ.

59 Bảng trang nhiều cấp

Phần lớn không gian địa chỉ không dùng tới. Phân cấp bảng trang giúp không phải cấp bảng cho vùng trống.






```python
def split(addr, widths):
    parts = []
    for w in reversed(widths):
        parts.append(addr & ((1 << w) - 1))
        addr >>= w
    return parts[::-1]

print(split(0x00403004, [10, 10, 12]))
print(split(0x7FFF12345678, [9, 9, 9, 9, 12]))

flat = 2 ** 20 * 4                          # bảng phẳng
used = (1 + 3) * 4096                       # 1 cấp 1, 3 cấp 2
print("Phẳng:", flat // 1024, "KB | 2 cấp:", used // 1024, "KB")
```

KẾT QUẢ KHI CHẠY

```
[1, 3, 4]
[255, 508, 145, 325, 1656]
Phẳng: 4096 KB | 2 cấp: 16 KB
```

Cấu trúc	Ý tưởng	Đánh đổi
Nhiều cấp	Chỉ cấp bảng con khi cần	Mỗi cấp thêm một lần đọc RAM
Băm	Băm số trang để tìm mục	Phải xử lý trùng
Đảo ngược	Một mục cho mỗi khung RAM	Tiết kiệm RAM, tìm chậm hơn

x86-64 thường dùng 4 cấp cho 48 bit địa chỉ, và có chế độ 5 cấp cho 57 bit.

60 Phân đoạn

Chia chương trình theo ý nghĩa: code, dữ liệu, stack. Mỗi đoạn có kích thước riêng, địa chỉ logic gồm số đoạn và độ dời.

Đoạn	Base	Limit
0: code	1400	1000
1: dữ liệu	6300	400
2: stack	4300	500

```
table = {0: (1400, 1000), 1: (6300, 400), 2: (4300, 500)}

def translate(seg, offset):
    base, limit = table[seg]
    if offset >= limit:
        return f"({seg}, {offset}): lỗi, vượt giới hạn"
    return f"({seg}, {offset}): địa chỉ {base + offset}"

for s, d in ((0, 53), (1, 399), (1, 400), (2, 499)):
    print(translate(s, d))
```

KẾT QUẢ KHI CHẠY

```
(0, 53): địa chỉ 1453
(1, 399): địa chỉ 6699
(1, 400): lỗi, vượt giới hạn
(2, 499): địa chỉ 4799
```

Tiêu chí	Phân trang	Phân đoạn
Kích thước	Cố định	Thay đổi
Phân mảnh	Trong	Ngoài
Người lập trình	Không thấy	Thấy, gắn với cấu trúc chương trình

Trên x86-64, phân trang mới là cơ chế chính.

61 Bộ nhớ ảo

Trang chỉ được nạp khi cần tới. Nhờ đó chương trình lớn hơn RAM vẫn chạy được, và cấp phát rất nhanh.

```python
import mmap, resource

def faults():
    return resource.getrusage(resource.RUSAGE_SELF).ru_minflt

size = 64 * 2 ** 20                          # 64 MB
before = faults()
m = mmap.mmap(-1, size)                      # xin 64 MB
after_alloc = faults()
for i in range(0, size, 4096):               # chạm vào từng trang
    m[i] = 1
after_touch = faults()
print("Lỗi trang khi xin dưới 100:", after_alloc - before < 100)
print("Lỗi trang khi chạm:", round(after_touch - after_alloc, -2))
```

KẾT QUẢ CHẠY TRÊN LINUX

```
Lỗi trang khi xin dưới 100: True
Lỗi trang khi chạm: 16400
```

1. **Truy cập trang chưa có trong RAM**
   Phần cứng thấy bit hợp lệ tắt, báo lỗi trang cho kernel.

2. **Kernel xử lý**
   Địa chỉ sai thì báo segfault. Hợp lệ thì tìm khung, đọc từ đĩa nếu cần.

3. **Chạy lại lệnh**
   Cập nhật bảng trang, thực hiện lại đúng lệnh vừa bị lỗi.

64 MB gồm 16384 trang. Xin chưa chắc đã chiếm RAM: kernel chỉ cấp khung thật khi trang được dùng.

62 Chi phí của lỗi trang

Một lỗi trang phải đọc đĩa chậm hơn truy cập RAM hàng chục nghìn lần, nên tỉ lệ lỗi phải cực nhỏ.

```
mem = 100                       # ns cho một lần truy cập RAM
fault = 8_000_000               # ns để xử lý lỗi trang, đĩa 8 ms

def eat(p):
    return (1 - p) * mem + p * fault

for p in (0, 1 / 1_000_000, 1 / 100_000, 1 / 1000):
    t = eat(p)
    print(f"Tỉ lệ {p:.6f}: {t:8.1f} ns, chậm {t / mem:.1f} lần")

p = 0.1 * mem / (fault - mem)  # chỉ cho phép chậm 10%
print(f"Chậm 10%: tối đa 1 lỗi trên {1 / p:,.0f} lần truy cập")
```

KẾT QUẢ KHI CHẠY

```
Tỉ lệ 0.000000:    100.0 ns, chậm 1.0 lần
Tỉ lệ 0.000001:    108.0 ns, chậm 1.1 lần
Tỉ lệ 0.000010:    180.0 ns, chậm 1.8 lần
Tỉ lệ 0.001000:   8099.9 ns, chậm 81.0 lần
Chậm 10%: tối đa 1 lỗi trên 799,990 lần truy cập
```

EAT = (1 - p) × m + p × thời gian xử lý lỗi trang.

Loại lỗi trang	Xảy ra khi	Chi phí
Nhỏ (minor)	Trang đã có trong RAM, chỉ cần gắn vào bảng	Rẻ
Lớn (major)	Phải đọc từ đĩa	Rất đắt
Không hợp lệ	Địa chỉ sai	Segfault

Tính cục bộ là lý do bộ nhớ ảo dùng được: chương trình thường chỉ truy cập một nhóm trang trong một khoảng thời gian.

63 mmap: dùng file như bộ nhớ

Ánh xạ file vào không gian địa chỉ. Đọc ghi file trở thành đọc ghi mảng, kernel lo nạp và ghi trang.

```
import mmap, os, tempfile

path = os.path.join(tempfile.mkdtemp(), "data.bin")
with open(path, "wb") as f:
    f.write(b"hello world")
with open(path, "r+b") as f:
    m = mmap.mmap(f.fileno(), 0)      # cả file
    print(m[:5])                      # đọc như mảng
    m[:5] = b"HELLO"                  # ghi như mảng
    m.flush()
    m.close()

print(open(path, "rb").read())
```

KẾT QUẢ CHẠY TRÊN LINUX

```
b'hello'
b'HELLO world'
```

Ứng dụng	Vì sao dùng
Nạp chương trình	File thực thi và thư viện được ánh xạ, các tiến trình dùng chung
Cơ sở dữ liệu	LMDB, SQLite có chế độ mmap
Xử lý file lớn	Không phải đọc hết vào RAM

malloc cỡ lớn cũng dùng mmap ẩn danh, tức là xin một vùng không gắn với file nào.

64 Thay trang: FIFO

Khi RAM hết khung trống, phải chọn một trang để đuổi ra. FIFO đuổi trang vào sớm nhất.

	5	0	2	1	0	3	0	2	4	3	0	3	2	1
K1	5	5	5	1	1	1	1	2	2	2	0	0	0	0
K2		0	0	0	0	3	3	3	4	4	4	4	2	2
K3			2	2	2	2	0	0	0	3	3	3	3	1
	F	F	F	F		F	F	F	F	F	F		F	F

```python
def fifo(refs, n):
    mem, faults = [], 0
    for p in refs:
        hit = p in mem
        if not hit:
            faults += 1
            mem = (mem + [p])[-n:]    # giữ n trang mới nhất
        print(p, mem, "" if hit else "*")
    return faults

refs = [5, 0, 2, 1, 0, 3, 0, 2, 4, 3, 0, 3, 2, 1]
print("Số lỗi trang:", fifo(refs, 3))
```

KẾT QUẢ KHI CHẠY

```
Số lỗi trang: 12
```

- Mỗi cột là một lần truy cập. K1 đến K3 là các khung, F là lỗi trang, ô vàng là trang vừa nạp.
- Ba lần đầu luôn là lỗi, vì khung còn trống (lỗi bắt buộc).
- Điểm yếu: trang cũ có thể là trang dùng liên tục, như trang 0 ở trên.

Làm bài: viết chuỗi thành hàng ngang, mỗi cột là các khung sau truy cập, đánh dấu lỗi rồi đếm.

65 Nghịch lý Belady

Đây là lời giải câu đố ở bìa. Với FIFO, thêm khung trang có thể làm tăng số lỗi trang.

	1	2	3	4	1	2	5	1	2	3	4	5
K1	1	1	1	4	4	4	5	5	5	5	5	5
K2		2	2	2	1	1	1	1	1	3	3	3
K3			3	3	3	2	2	2	2	2	4	4
	F	F	F	F	F	F	F			F	F	

	1	2	3	4	1	2	5	1	2	3	4	5
K1	1	1	1	1	1	1	5	5	5	5	4	4
K2		2	2	2	2	2	2	1	1	1	1	5
K3			3	3	3	3	3	3	2	2	2	2
K4				4	4	4	4	4	4	3	3	3
	F	F	F	F			F	F	F	F	F	F

KẾT QUẢ KHI CHẠY

```
3 khung: 9 lỗi | 4 khung: 10 lỗi
```

Với 4 khung, trang 1 và 2 bị đuổi đúng trước khi được dùng lại. FIFO chỉ nhìn tuổi của trang, không nhìn mức độ sử dụng.

LRU và OPT là thuật toán dạng stack: tập trang với n khung luôn nằm trong tập trang với n + 1 khung, nên không bị nghịch lý này.

66 Thay trang: OPT

Đuổi trang lâu được dùng lại nhất. Số lỗi là ít nhất có thể, nhưng cần biết trước tương lai.

	5	0	2	1	0	3	0	2	4	3	0	3	2	1
K1	5	5	5	1	1	3	3	3	3	3	3	3	3	3
K2		0	0	0	0	0	0	0	0	0	0	0	0	0
K3			2	2	2	2	2	2	4	4	4	4	2	1
	F	F	F	F		F			F				F	F

```python
def opt(refs, n):
    mem, faults = [], 0
    for i, p in enumerate(refs):
        hit = p in mem
        if not hit:
            faults += 1
            if len(mem) == n:
                rest = refs[i + 1:] + mem
                mem.remove(max(mem, key=rest.index))
            mem.append(p)
        print(p, mem, "" if hit else "*")
    return faults

refs = [5, 0, 2, 1, 0, 3, 0, 2, 4, 3, 0, 3, 2, 1]
print("Số lỗi trang:", opt(refs, 3))
```

KẾT QUẢ KHI CHẠY

```
Số lỗi trang: 8
```

Hệ điều hành không biết tương lai, nên OPT chỉ dùng làm thước đo. Thuật toán khác càng gần OPT càng tốt.

Nhiều trang đều không dùng lại thì chọn trang nào cũng cùng số lỗi.

67 Thay trang: LRU

Dùng quá khứ đoán tương lai: đuổi trang lâu nhất chưa được dùng. Kết quả thường gần với OPT.

	5	0	2	1	0	3	0	2	4	3	0	3	2	1
K1	5	5	5	1	1	1	1	2	2	2	0	0	0	1
K2		0	0	0	0	0	0	0	0	3	3	3	3	3
K3			2	2	2	3	3	3	4	4	4	4	2	2
	F	F	F	F		F		F	F	F	F		F	F

```python
def lru(refs, n):
    mem, faults = [], 0                    # cuối danh sách là mới dùng
    for p in refs:
        hit = p in mem
        if hit:
            mem.remove(p)
        else:
            faults += 1
            if len(mem) == n:
                mem.pop(0)                 # đuổi trang lâu nhất
        mem.append(p)
        print(p, sorted(mem), "" if hit else "*")
    return faults

refs = [5, 0, 2, 1, 0, 3, 0, 2, 4, 3, 0, 3, 2, 1]
print("Số lỗi trang:", lru(refs, 3))
```

KẾT QUẢ KHI CHẠY

```
Số lỗi trang: 11
```

Cài LRU chính xác phải cập nhật ở mọi lần truy cập, quá tốn kém, nên hệ thống thật dùng thuật toán xấp xỉ. Cache của ứng dụng cũng hay dùng LRU.

68 Thay trang: Clock

Kim quay qua các khung: bit 1 thì cho cơ hội thứ hai, bit 0 thì đuổi. Trang mới nạp có bit 1.

	5	0	2	1	0	3	0	2	4	3	0	3	2	1
K1	5	5	5	1	1	1	1	2	2	2	2	3	3	3
K2		0	0	0	0	0	0	0	4	4	4	4	2	2
K3			2	2	2	3	3	3	3	3	0	0	0	1
	F	F	F	F		F		F	F		F	F	F	F

```python
def clock(refs, n):
    frames, bit, hand, faults = [None] * n, [0] * n, 0, 0
    for p in refs:
        hit = p in frames
        if hit:
            bit[frames.index(p)] = 1
        else:
            faults += 1
            while bit[hand]:                # cơ hội thứ hai
                bit[hand] = 0
                hand = (hand + 1) % n
            frames[hand], bit[hand] = p, 1
            hand = (hand + 1) % n
        shown = [f for f in frames if f is not None]
        print(p, shown, "" if hit else "*")
    return faults
refs = [5, 0, 2, 1, 0, 3, 0, 2, 4, 3, 0, 3, 2, 1]
print("Số lỗi trang:", clock(refs, 3))
```

KẾT QUẢ KHI CHẠY

```
Số lỗi trang: 11
```

✓ Phần cứng tự bật bit khi truy cập, nên rất rẻ. Bit dirty ưu tiên đuổi trang chưa sửa.

69 So sánh các thuật toán thay trang

Số lỗi trang theo số khung, trên cùng một chuỗi tham chiếu. Cả bốn thuật toán chạy cùng dữ liệu.

KẾT QUẢ KHI CHẠY

```
Khung | FIFO | LRU | Clock | OPT
    1 |   14 |  14 |    14 |  14
    2 |   12 |  12 |    12 |  10
    3 |   12 |  11 |    11 |   8
    4 |    9 |   7 |     7 |   7
    5 |    6 |   6 |     6 |   6
```

Thuật toán	Dựa vào	Belady	Thực tế
FIFO	Thời điểm nạp	Có thể	Đơn giản, kém
OPT	Tương lai	Không	Không cài được, dùng làm chuẩn
LRU	Lần dùng gần nhất	Không	Tốt, cài chính xác rất đắt
Clock	Bit tham chiếu	Có thể	Rẻ, được dùng rộng rãi

THAY CỤC BỘ
Chỉ đuổi trang của chính tiến trình đó. Dễ dự đoán, có thể lãng phí.

THAY TOÀN CỤC
Đuổi trang của bất kỳ tiến trình nào. Hiệu quả hơn, được dùng phổ biến.

Kết quả chỉ đúng với chuỗi tham chiếu này. Thuật toán chuẩn để so sánh luôn là OPT, không thuật toán nào có ít lỗi hơn nó.

70 Thrashing và tập làm việc

Tổng nhu cầu vượt RAM thì hệ thống chỉ lo đổi trang. CPU rảnh mà máy gần như đứng im.

```
def working_set(refs, t, window):
    return set(refs[max(0, t - window + 1):t + 1])

refs = [1, 2, 1, 3, 1, 2, 7, 8, 7, 9, 7, 8]
for t in (5, 11):
    print(f"Thời điểm {t}: {sorted(working_set(refs, t, 6))}")

need = sum([3, 3, 4])                    # 3 tập làm việc
print("Cần", need, "khung, có 8:", "thiếu" if need > 8 else "đủ")
```

KẾT QUẢ KHI CHẠY

```
Thời điểm 5: [1, 2, 3]
Thời điểm 11: [7, 8, 9]
Cần 10 khung, có 8: thiếu
```

```
def proportional(sizes, frames):
    total = sum(sizes)
    return [round(s / total * frames) for s in sizes]

print(proportional([10, 127], 62))
```

KẾT QUẢ KHI CHẠY

```
[5, 57]
```

Dấu hiệu	Xử lý
Đèn ổ đĩa sáng liên tục, máy rất chậm	Đóng bớt chương trình, thêm RAM
Tỉ lệ lỗi trang cao, CPU thấp	Giảm số tiến trình chạy cùng lúc

Sai lầm kinh điển: thấy CPU rảnh nên tăng thêm tiến trình.

71 Đọc đúng lượng RAM trên Linux

RAM trống là RAM lãng phí. Linux dùng RAM dư làm cache, nên cột free nhỏ chưa phải là dấu hiệu xấu.

```
$ free -h
               total        used        free      shared  buff/cache   available
Mem:           3.7Gi       532Mi       2.5Gi       3.5Mi       727Mi       3.2Gi
Swap:          1.0Gi          0B       1.0Gi
```

Cột	Nghĩa
free	Hoàn toàn chưa dùng
buff/cache	Cache file, thu hồi được khi cần
available	Ước tính RAM còn cho chương trình mới. Hãy nhìn cột này
Swap used	Trang đã bị đẩy ra đĩa

OOM KILLER
Hết cả RAM lẫn swap, kernel chọn một tiến trình để kết thúc. Chương trình chỉ in Killed, mã thoát 137.

OVERCOMMIT
Linux cho xin nhiều hơn RAM thật, vì đa số vùng xin không dùng hết.

Trên Windows	Nghĩa
In use	Đang được dùng
Committed	Tổng đã hứa cấp, gồm cả page file
Cached	Cache, thu hồi được khi cần

Tìm kẻ ăn RAM: ps aux --sort=-rss | head. Kiểm tra OOM: sudo dmesg | grep -i oom. Trên Windows: Task Manager, sắp xếp cột Memory.

72 Stack, heap và lỗi bộ nhớ

Phần lớn lỗi crash của C và C++ nằm ở đây. Hiểu bộ nhớ ảo giúp đọc được thông báo lỗi.

```
import ctypes, os, signal

pid = os.fork()
if pid == 0:
    ctypes.string_at(0)              # đọc địa chỉ 0

_, status = os.waitpid(pid, 0)
print("Bị kết thúc bởi:", signal.Signals(os.WTERMSIG(status)).name)
```

KẾT QUẢ CHẠY TRÊN LINUX

```
Bị kết thúc bởi: SIGSEGV
```

	Stack	Heap
Cấp phát	Tự động theo hàm	malloc, new
Kích thước	Nhỏ, thường vài MB	Lớn
Lỗi đặc trưng	Stack overflow	Rò rỉ, dùng sau khi free

Lỗi	Nguyên nhân
Segmentation fault	Truy cập trang không hợp lệ, con trỏ NULL
Stack overflow	Đệ quy quá sâu, mảng cục bộ quá lớn
Memory leak	Cấp mà không trả, RAM tăng dần
Use after free	Dùng vùng nhớ đã trả
Buffer overflow	Ghi quá giới hạn mảng

Công cụ nên biết: gdb để xem vị trí crash, valgrind hoặc gcc -fsanitize=address để bắt lỗi bộ nhớ.

73 File descriptor

Với kernel, file đang mở chỉ là một con số. Số 0, 1, 2 là đầu vào, đầu ra và lỗi chuẩn. Dấu > của shell chỉ là đổi đích của số 1.

```
import os, tempfile

path = os.path.join(tempfile.mkdtemp(), "log.txt")
fd = os.open(path, os.O_CREAT | os.O_WRONLY)

saved = os.dup(1)                    # giữ lại đầu ra cũ
os.dup2(fd, 1)                       # số 1 giờ trỏ vào file
print("Dòng này đi vào file")
os.dup2(saved, 1)                    # trả lại như cũ

print("Dòng này ra màn hình")
print("Trong file có:", open(path).read().strip())
```

KẾT QUẢ CHẠY TRÊN LINUX

```
Dòng này ra màn hình
Trong file có: Dòng này đi vào file
```



Số	Tên	Ví dụ shell
0	stdin	`sort < a.txt`
1	stdout	`ls > out.txt`
2	stderr	`gcc a.c 2> err.txt`

Unix coi gần như mọi thứ là file: thiết bị, pipe, socket đều dùng read, write, close. Vì thế một lệnh có thể làm việc với rất nhiều nguồn dữ liệu.

74 Inode và thư mục

Inode lưu mọi thông tin về file, trừ tên. Tên nằm trong thư mục, và thư mục chỉ là bảng từ tên đến số inode.

```
echo "noi dung" > a.txt
before=$(stat -c %i a.txt)

mv a.txt b.txt
after=$(stat -c %i b.txt)
[ "$before" = "$after" ] && echo "Đổi tên: inode giữ nguyên"

stat -c 'Số link: %h | cỡ: %s byte | kiểu: %F' b.txt
```

KẾT QUẢ CHẠY TRÊN LINUX

```
Đổi tên: inode giữ nguyên
Số link: 1 | cỡ: 9 byte | kiểu: regular file
```

Trong inode	Không có trong inode
Kiểu file, quyền, chủ sở hữu	**Tên file**
Kích thước, các mốc thời gian	**Đường dẫn**
Số link, vị trí các khối dữ liệu	

MV NHANH
Đổi tên trong cùng phân vùng chỉ sửa thư mục, không chép dữ liệu, nên file 100 GB vẫn xong ngay.

HẾT INODE
Quá nhiều file nhỏ thì báo No space left on device dù ổ còn trống. Kiểm tra bằng df -i.

Mở /home/an/bai.txt: bắt đầu từ thư mục gốc, tìm home, rồi an, rồi bai.txt, mỗi bước đọc một thư mục. Có inode rồi mới kiểm tra quyền và cấp file descriptor.

75 Hard link và soft link

Hard link là thêm một tên cho cùng inode. Soft link là một file nhỏ chứa đường dẫn tới file khác.

```
echo "bai tap" > a.txt
ln a.txt hard.txt                # hard link
ln -s a.txt soft.txt             # soft link
stat -c '%h link: %n' a.txt hard.txt

rm a.txt
cat hard.txt                     # vẫn đọc được
cat soft.txt                     # hỏng
```

KẾT QUẢ CHẠY TRÊN LINUX

```
2 link: a.txt
2 link: hard.txt
bai tap
cat: soft.txt: No such file or directory
```

Tiêu chí	Hard link	Soft link
Có inode riêng	Không	Có
Xóa file gốc	Vẫn dùng được	Trỏ vào hư không
Khác phân vùng	Không được	Được
Trỏ tới thư mục	Thường bị cấm	Được

Lệnh rm không xóa dữ liệu ngay. Nó xóa một tên và giảm số link. Khi số link bằng 0 và không ai còn mở file, khối dữ liệu mới được thu hồi.

Trên Windows: mklink /H tạo hard link, mklink tạo symbolic link. Node dùng link rất nhiều ở node_modules/.bin.

76 Xóa file rồi mà ổ vẫn đầy

Lỗi rất hay gặp trên server. Xóa file log đang được mở thì tên mất, nhưng dung lượng vẫn bị giữ tới khi đóng file.

```python
import os, tempfile

path = os.path.join(tempfile.mkdtemp(), "app.log")
f = open(path, "w+")
f.write("Log quan trọng\n")
f.flush()

os.unlink(path)                          # giống lệnh rm
print("Còn thấy tên file?", os.path.exists(path))
print("Số link:", os.fstat(f.fileno()).st_nlink)

f.seek(0)
print("Vẫn đọc được:", f.read().strip())
f.close()                                # giờ mới thu hồi khối
```

KẾT QUẢ CHẠY TRÊN LINUX

```
Còn thấy tên file? False
Số link: 0
Vẫn đọc được: Log quan trọng
```

Việc	Lệnh
Tìm file đã xóa còn mở	`sudo lsof +L1`
Tìm thư mục ngốn đĩa	`du -sh /var/* | sort -h`
Làm rỗng log đúng cách	`: > app.log`

Làm rỗng file thay vì xóa, hoặc khởi động lại dịch vụ. Về lâu dài hãy cấu hình logrotate.

77 Cấp phát file trên đĩa

Đĩa chia thành các khối. Có ba cách ghi nhớ khối nào thuộc về file nào.

Cách	Ý tưởng	Được	Mất
Liên tục	Các khối liền nhau	Đọc rất nhanh	Phân mảnh ngoài, khó nới file
Liên kết	Mỗi khối trỏ tới khối sau	Không phân mảnh ngoài	Truy cập ngẫu nhiên chậm
Chỉ mục	Một khối chứa danh sách khối	Truy cập ngẫu nhiên tốt	Tốn khối chỉ mục

FAT là cách liên kết, nhưng gom mọi con trỏ vào một bảng để đọc nhanh hơn:




```
fat = {4: 7, 7: 2, 2: 10, 10: None}   # khối đến khối kế tiếp
def blocks(start):
    while start is not None:
        yield start
        start = fat[start]

print(list(blocks(4)))
```

KẾT QUẢ KHI CHẠY

```
[4, 7, 2, 10]
```

EXTENT

ext4 và NTFS ghi theo đoạn liền nhau: bắt đầu từ khối nào, dài bao nhiêu. Gọn hơn danh sách từng khối.

CHỐNG PHÂN MẢNH

Có ích với HDD, không cần với SSD.

78 Kích thước file tối đa của inode

Dạng bài kinh điển: inode có 12 con trỏ trực tiếp, thêm một con trỏ gián tiếp đơn, đôi và ba.

```
def max_size(block=4096, ptr=4, direct=12):
    n = block // ptr                          # số con trỏ trong một khối
    return (direct + n + n ** 2 + n ** 3) * block

def locate(offset, block=4096, ptr=4, direct=12):
    n, i = block // ptr, offset // block
    if i < direct:
        return f"trực tiếp số {i}"
    i -= direct
    if i < n:
        return f"gián tiếp đơn, mục {i}"
    i -= n
    if i < n * n:
        return f"gián tiếp đôi, mục {i // n} rồi {i % n}"
    return "gián tiếp ba"

print(f"Tối đa: {max_size() / 2 ** 40:.2f} TiB")
for off in (20_000, 100_000, 5_000_000, 2 ** 33):
    print(off, "->", locate(off))
```

KẾT QUẢ KHI CHẠY

```
Tối đa: 4.00 TiB
20000 -> trực tiếp số 4
100000 -> gián tiếp đơn, mục 12
5000000 -> gián tiếp đôi, mục 0 rồi 184
8589934592 -> gián tiếp ba
```

Đây là cấu trúc kiểu ext2, ext3. ext4 dùng extent và hỗ trợ file tới 16 TiB với khối 4 KiB. Đọc một byte ở vùng gián tiếp đôi cần đọc thêm 2 khối chỉ mục.

79 Quản lý vùng trống

Hệ thống file cần biết khối nào còn trống. Bitmap là cách phổ biến nhất: mỗi khối một bit.

```
disk = 2 ** 40                          # 1 TiB
block = 4096
bitmap_mb = disk // block // 8 / 2 ** 20
print("Bitmap cần:", bitmap_mb, "MB")

bits = 0b11110111                       # 1 là đã dùng
free = next(i for i in range(8) if not bits >> i & 1)
print("Khối trống đầu tiên:", free)
```

KẾT QUẢ KHI CHẠY

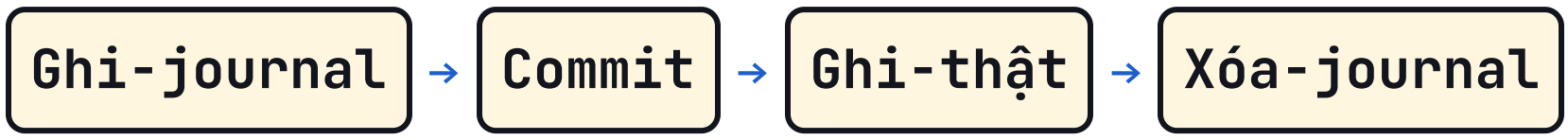
```
Bitmap cần: 32.0 MB
Khối trống đầu tiên: 3
```

Cách	Đặc điểm
Bitmap	Dễ tìm các khối liền nhau, tốn cố định
Danh sách liên kết	Không tốn thêm chỗ, nhưng duyệt chậm
Nhóm	Một khối chứa nhiều địa chỉ khối trống
Đếm	Lưu vị trí đầu và độ dài từng đoạn trống

Xóa file chỉ đánh dấu khối là trống, không ghi đè. Đó là lý do phần mềm phục hồi dữ liệu làm được, và vì sao bán máy cũ phải xóa an toàn.

80 Mất điện giữa lúc ghi

Mất điện giữa chừng làm hệ file không nhất quán. Journal ghi ý định trước, làm thật sau.



Chế độ ext4	Ghi vào journal	Đánh đổi
journal	Cả dữ liệu và metadata	An toàn nhất, chậm nhất
ordered	Metadata, dữ liệu ghi xuống trước	Mặc định, cân bằng
writeback	Chỉ metadata, không ràng buộc thứ tự	Nhanh, file có thể chứa dữ liệu cũ

Lưu file an toàn khi lập trình: mất điện lúc nào thì file vẫn là bản cũ hoặc bản mới.

```
import os, tempfile

def save(path, text):
    tmp = path + ".tmp"
    with open(tmp, "w", encoding="utf-8") as f:
        f.write(text)
        f.flush()                  # từ Python xuống kernel
        os.fsync(f.fileno())       # từ kernel xuống đĩa
    os.replace(tmp, path)          # đổi tên là thao tác nguyên tử
path = os.path.join(tempfile.mkdtemp(), "config.json")
save(path, '{"phien_ban": 2}')
print(open(path).read())
```

KẾT QUẢ KHI CHẠY

```
{"phien_ban": 2}
```

81 Chọn hệ thống file nào?

Hay gặp nhất: copy phim 5 GB vào USB thì báo file quá lớn, dù USB còn trống 30 GB. Đó là giới hạn của FAT32.

Hệ file	File tối đa	Dùng khi
FAT32	4 GiB	Thiết bị cũ, phân vùng khởi động EFI
exFAT	Rất lớn	USB, thẻ nhớ dùng chung Windows và Mac
NTFS	Rất lớn	Ổ Windows, có quyền và journal
ext4	16 TiB	Mặc định trên nhiều bản Linux
XFS	Rất lớn	Server, file lớn
Btrfs, ZFS	Rất lớn	Snapshot, kiểm tra toàn vẹn dữ liệu
APFS	Rất lớn	Máy Apple, tối ưu cho SSD

USB DÙNG CHUNG
Format exFAT. Lưu ý format sẽ xóa dữ liệu, hãy sao lưu trước.

Ổ NGOÀI CHO BACKUP
Dùng hệ file gốc của máy: NTFS cho Windows, APFS cho Mac.

Lệnh Linux	Công dụng
`df -T`	Xem kiểu hệ file đang gắn
`lsblk -f`	Xem ổ, phân vùng, hệ file
`mount, umount`	Gắn, gỡ một hệ file vào cây thư mục

Linux chỉ có một cây thư mục, ổ mới được gắn vào đâu đó trong cây. Windows thì tách theo ký tự C:, D:. Luôn rút USB an toàn để cache kịp ghi xuống.

82 Quyền truy cập file

Mỗi file có ba nhóm: chủ sở hữu, nhóm và người khác. Mỗi nhóm có ba quyền: đọc, ghi, thực thi.

```
echo 'echo "Đang chạy"' > a.sh
./a.sh
chmod +x a.sh
./a.sh
stat -c '%A = %a' a.sh
chmod 600 a.sh
stat -c '%A = %a' a.sh
stat -c '%A %n' /usr/bin/passwd
```

KẾT QUẢ CHẠY TRÊN LINUX

```
bash: line 3: ./a.sh: Permission denied
Đang chạy
-rwxr-xr-x = 755
-rw------- = 600
-rwsr-xr-x /usr/bin/passwd
```

Số	Quyền	Dùng cho
644	rw-r--r--	File thường
755	rwxr-xr-x	Script, thư mục
600	rw-------	Khóa SSH, file bí mật
777	rwxrwxrwx	Tránh dùng, ai cũng sửa được

QUYỀN TRÊN THƯ MỤC

r là liệt kê, w là tạo và xóa file, x là được đi vào thư mục.

BIT SUID

Chữ s ở passwd: chạy với quyền của chủ file.

Gặp Permission denied, đừng vội chmod 777. Xem chủ file bằng ls -l rồi sửa đúng chỗ.

83 Ổ cứng HDD đọc dữ liệu thế nào?

Thời gian đọc gồm ba phần. Tìm rãnh và chờ đĩa quay chiếm gần hết, nên phải giảm di chuyển đầu đọc.

```python
def access_ms(rpm, seek_ms, mb_per_s, kb):
    rotation = 60_000 / rpm / 2          # chờ nửa vòng
    transfer = kb / 1024 / mb_per_s * 1000
    return seek_ms, rotation, transfer

s, r, t = access_ms(rpm=7200, seek_ms=9, mb_per_s=150, kb=4)
print(f"Tìm rãnh {s} ms + quay {r:.2f} ms + truyền {t:.3f} ms")
print(f"Tổng: {s + r + t:.2f} ms")
print(f"Khoảng {1000 / (s + r + t):.0f} lần đọc ngẫu nhiên/giây")
```

KẾT QUẢ KHI CHẠY

```
Tìm rãnh 9 ms + quay 4.17 ms + truyền 0.026 ms
Tổng: 13.19 ms
Khoảng 76 lần đọc ngẫu nhiên/giây
```

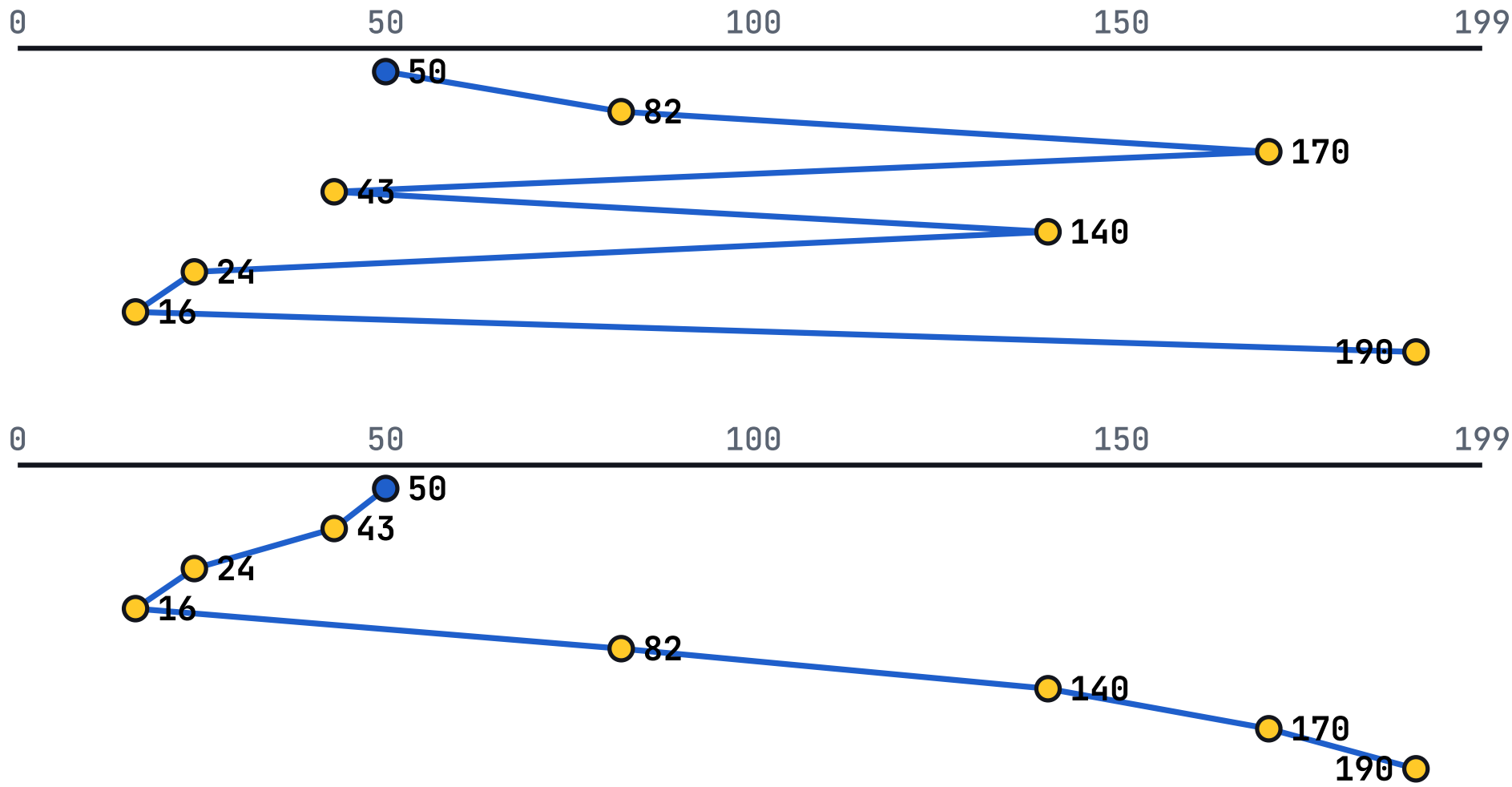
Thành phần	Ý nghĩa
Seek time	Đưa đầu đọc tới đúng rãnh
Rotational latency	Chờ sector quay tới dưới đầu đọc
Transfer time	Thật sự đọc dữ liệu

- ✓ Đọc tuần tự nhanh hơn đọc ngẫu nhiên rất nhiều trên HDD.
- ✓ Vì thế cơ sở dữ liệu và log cố ghi nối tiếp.
- ✓ Thông số trong code là giả định điển hình của ổ 7200 vòng mỗi phút.

Trang sau: khi có nhiều yêu cầu đọc cùng chờ, sắp xếp thứ tự hợp lý sẽ giảm quãng đường đầu đọc đi rất nhiều.

84 Điều phối đĩa: FCFS, SSTF

Đề chung: đầu đọc ở 50, các rãnh 0 đến 199, yêu cầu 82, 170, 43, 140, 24, 16, 190.



KẾT QUẢ KHI CHẠY

```
FCFS: 642 | SSTF: 208
```

FCFS

Công bằng, nhưng đầu đọc chạy qua chạy lại rất nhiều.

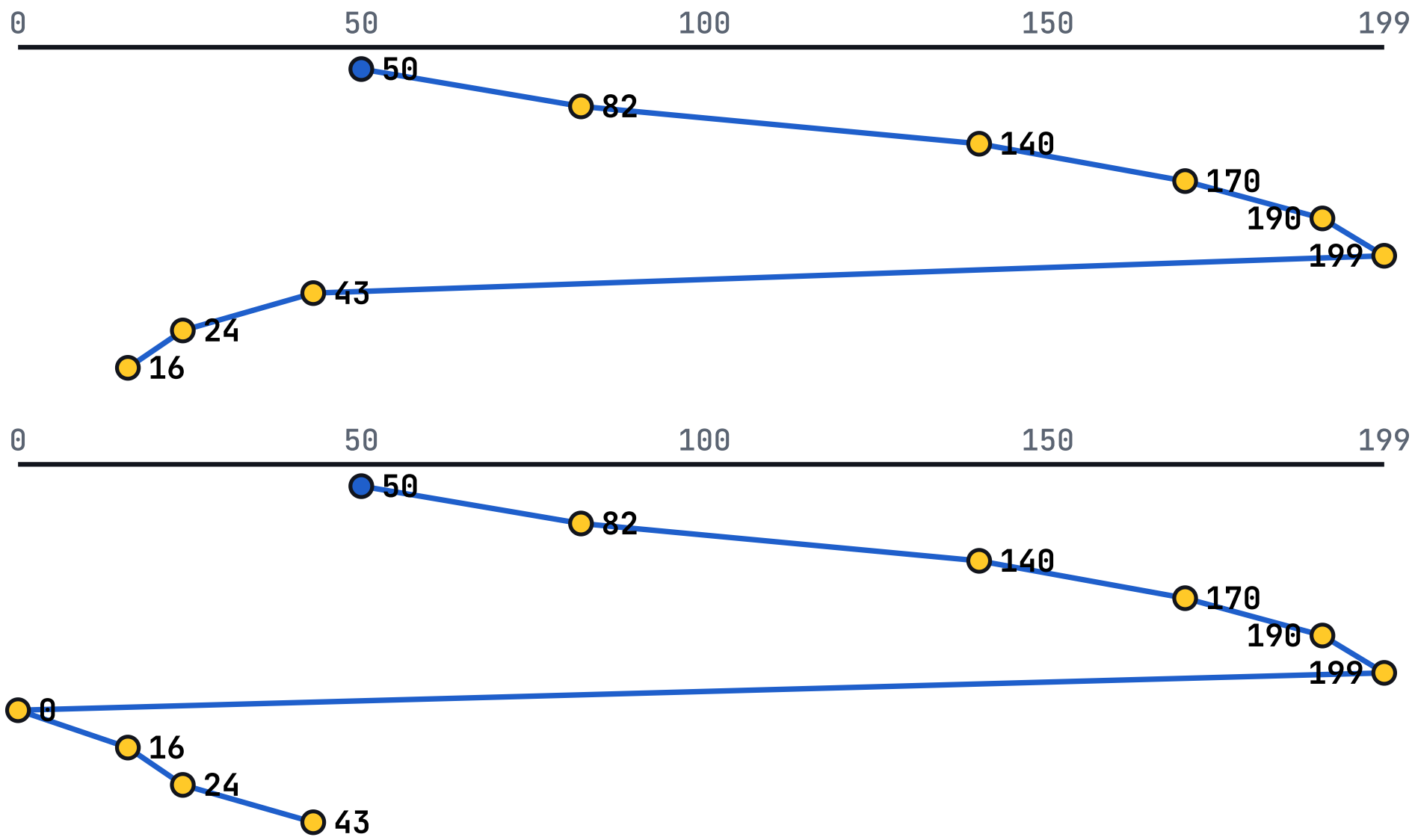
SSTF

Chọn yêu cầu gần nhất. Quãng đường ngắn, nhưng yêu cầu ở xa có thể bị đói.

Hình trên là FCFS, hình dưới là SSTF. Điểm xanh là vị trí bắt đầu.

85 Đĩa: SCAN và C-SCAN

Đầu đọc đi một chiều như thang máy. SCAN quay đầu ở mép đĩa, C-SCAN nhảy về đầu kia rồi đi tiếp cùng chiều.



KẾT QUẢ KHI CHẠY

```
SCAN: 332 | C-SCAN: 391
```

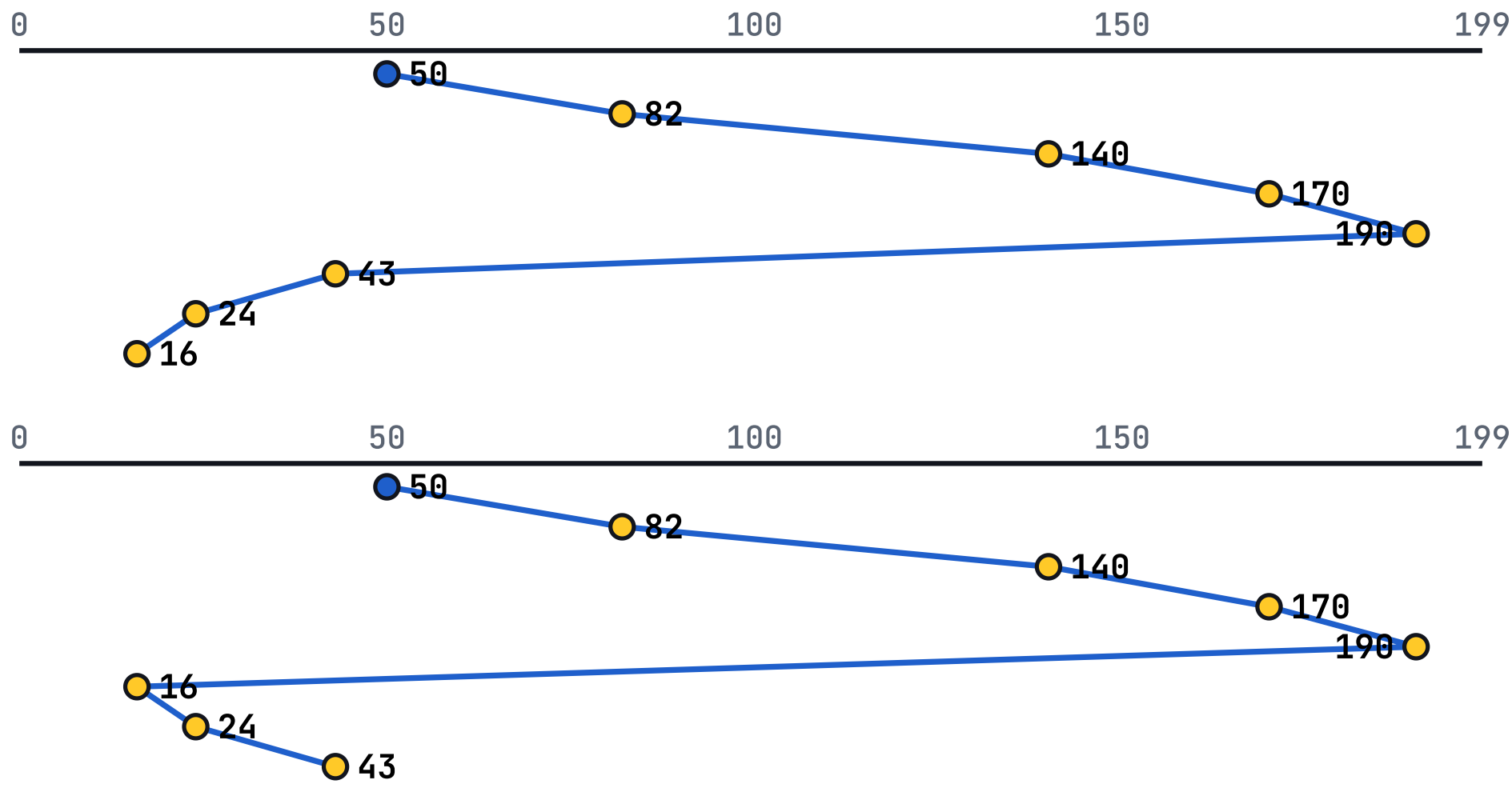
SCAN
Không gây đói. Yêu cầu ở giữa đĩa được phục vụ thường xuyên hơn hai đầu.

C-SCAN
Thời gian chờ đều hơn. Bước nhảy về đầu đĩa không phục vụ yêu cầu nào.

Hình trên giả sử đầu đọc đang đi về phía số lớn, và tính cả bước nhảy từ 199 về 0. Một số giáo trình không tính bước nhảy này, hãy theo đề.

86 Đĩa: LOOK và C-LOOK

Giống SCAN, C-SCAN nhưng chỉ đi tới yêu cầu xa nhất, không chạy thừa ra mép đĩa.



KẾT QUẢ KHI CHẠY

```
LOOK: 314 | C-LOOK: 341
```

Thuật toán	Đặc điểm chính	Đói
FCFS	Theo thứ tự đến	Không
SSTF	Gần nhất trước	Có thể
SCAN, LOOK	Quét hai chiều	Không
C-SCAN, C-LOOK	Quét một chiều, chờ đều hơn	Không

87 Code điều phối đĩa

Dùng để tự kiểm tra bài làm: đổi reqs, head, top theo đề của bạn.

```
def total(path):
    return sum(abs(a - b) for a, b in zip(path, path[1:]))

def sstf(head, reqs):
    left, path = list(reqs), [head]
    while left:
        near = min(left, key=lambda r: abs(r - path[-1]))
        left.remove(near)
        path.append(near)
    return path

def plans(head, reqs, top):
    up = sorted(r for r in reqs if r >= head)
    low = sorted(r for r in reqs if r < head)
    return {"FCFS": [head] + reqs,
            "SSTF": sstf(head, reqs),
            "SCAN": [head] + up + [top] + low[::-1],
            "C-SCAN": [head] + up + [top, 0] + low,
            "LOOK": [head] + up + low[::-1],
            "C-LOOK": [head] + up + low}
reqs, head, top = [82, 170, 43, 140, 24, 16, 190], 50, 199
for name, path in plans(head, reqs, top).items():
    print(f"{name:7}{total(path):4}  {path}")
```

KẾT QUẢ KHI CHẠY

```
FCFS    642  [50, 82, 170, 43, 140, 24, 16, 190]
SSTF    208  [50, 43, 24, 16, 82, 140, 170, 190]
SCAN    332  [50, 82, 140, 170, 190, 199, 43, 24, 16]
C-SCAN  391  [50, 82, 140, 170, 190, 199, 0, 16, 24, 43]
LOOK    314  [50, 82, 140, 170, 190, 43, 24, 16]
C-LOOK  341  [50, 82, 140, 170, 190, 16, 24, 43]
```

Code giả sử đầu đọc đang đi về phía số lớn. Đề cho đi về phía số nhỏ thì đảo chiều up và low.

88 SSD khác HDD thế nào?

SSD không có đầu đọc, nên đọc ngẫu nhiên rất nhanh và các thuật toán điều phối ở trên hầu như không cần nữa.

```
cat /sys/block/sd*/queue/scheduler | sort -u
```

KẾT QUẢ CHẠY TRÊN LINUX

```
[none] mq-deadline kyber
```

Tiêu chí	HDD	SSD
Cấu tạo	Đĩa quay, đầu đọc	Chip nhớ flash
Đọc ngẫu nhiên	Chậm	Rất nhanh
Ghi đè	Ghi thẳng lên chỗ cũ	Phải xóa theo khối lớn trước
Tuổi thọ	Hỏng cơ khí	Giới hạn số lần ghi xóa

Kỹ thuật SSD	Công dụng
FTL	Lớp dịch địa chỉ logic sang vị trí flash thật
Wear leveling	Rải đều lượt ghi cho mọi ô nhớ
TRIM	Hệ điều hành báo khối nào đã xóa, để SSD dọn trước
Garbage collection	Dồn dữ liệu còn dùng, xóa khối cũ

Dấu ngoặc [none] ở trên là bộ điều phối đang dùng. Với ổ rất nhanh, xếp lại thứ tự không còn đáng công, nên thường chọn none.

Dữ liệu trên SSD bị xóa khó khôi phục hơn HDD vì TRIM. Nhớ sao lưu những gì quan trọng.

89 RAID

Ghép nhiều ổ để tăng tốc độ, tăng độ an toàn hoặc cả hai. Parity bằng phép XOR giúp khôi phục một ổ hỏng.

```
def xor(*blocks):
    out = bytes(len(blocks[0]))
    for b in blocks:
        out = bytes(x ^ y for x, y in zip(out, b))
    return out

d1, d2, d3 = b"ANH", b"EMI", b"TOT"
parity = xor(d1, d2, d3)                # lưu ở ổ thứ tư

print("Ổ 2 hỏng, khôi phục:", xor(d1, d3, parity))
```

KẾT QUẢ KHI CHẠY

```
Ổ 2 hỏng, khôi phục: b'EMI'
```

Mức	Cách làm	Số ổ tối thiểu	Chịu hỏng	Dung lượng dùng được
RAID 0	Chia dải	2	0 ổ	n ổ
RAID 1	Nhân bản	2	n - 1 ổ	1 ổ
RAID 5	Parity rải đều	3	1 ổ	n - 1 ổ
RAID 6	Hai parity	4	2 ổ	n - 2 ổ
RAID 10	Nhân bản rồi chia dải	4	1 ổ mỗi cặp	n / 2 ổ

RAID không phải sao lưu. Xóa nhầm, mã độc tống tiền hay cháy máy đều làm hỏng mọi ổ cùng lúc. Vẫn cần bản sao ở nơi khác.

Ví dụ: 4 ổ 2 TB chạy RAID 5 dùng được 6 TB, chạy RAID 10 dùng được 4 TB.

90 Vào ra và driver

Thiết bị chậm và rất đa dạng. Kernel giấu sự khác biệt sau driver, và tránh để CPU ngồi chờ.

Cách	CPU làm gì	Hợp với
Polling	Liên tục hỏi thiết bị	Thiết bị cực nhanh
Ngắt	Làm việc khác, thiết bị báo khi xong	Bàn phím, chuột
DMA	Chỉ ra lệnh, thiết bị tự chép vào RAM	Đĩa, card mạng



Kỹ thuật	Công dụng
Buffering	Đệm dữ liệu giữa hai bên có tốc độ khác nhau
Caching	Giữ bản sao dữ liệu hay dùng trong RAM
Spooling	Xếp hàng công việc in cho thiết bị dùng riêng

Trên Linux, thiết bị xuất hiện trong /dev. Thiết bị khối như ổ đĩa đọc theo khối, thiết bị ký tự như bàn phím đọc theo dòng byte.

91 Một luồng phục vụ nghìn kết nối

Một luồng cho mỗi kết nối thì tốn kém. nginx và Node.js dùng một luồng theo dõi nhiều kết nối nhờ kernel báo sự kiện.

```
import os, selectors

sel = selectors.DefaultSelector()
a, b = os.pipe(), os.pipe()
sel.register(a[0], selectors.EVENT_READ, "ống A")
sel.register(b[0], selectors.EVENT_READ, "ống B")

os.write(b[1], b"xin chao")            # chỉ ống B có dữ liệu

for key, _ in sel.select(timeout=1):
    print(key.data, "có dữ liệu:", os.read(key.fd, 20))
print("Cơ chế bên dưới:", type(sel).__name__)
```

KẾT QUẢ CHẠY TRÊN LINUX

```
ống B có dữ liệu: b'xin chao'
Cơ chế bên dưới: EpollSelector
```

Kiểu I/O	Đặc điểm
Chặn	Gọi read rồi ngủ tới khi có dữ liệu. Dễ viết
Ghép kênh	Một lời gọi chờ nhiều nguồn: select, poll, epoll
Bất đồng bộ	Gửi yêu cầu, xong thì nhận thông báo

Cơ chế hiệu năng cao: Linux có epoll, io_uring. Windows có I/O Completion Ports. macOS, BSD có kqueue.

async, await chạy trên các cơ chế này. Một việc nặng CPU trong vòng lặp sự kiện sẽ chặn mọi kết nối khác.

92 Máy ảo

Hypervisor chia một máy thật thành nhiều máy ảo. Mỗi máy ảo có kernel riêng, nên có thể chạy Linux trên Windows.

Máy ảo 1, máy ảo 2

Hypervisor loại 1

Phần cứng

Máy ảo

Hypervisor loại 2

Hệ điều hành chủ

Phần cứng

Loại	Đặc điểm	Ví dụ
Loại 1	Chạy trực tiếp trên phần cứng, dùng cho server	VMware ESXi, Hyper-V, KVM
Loại 2	Chạy như một ứng dụng, dùng cho cá nhân	VirtualBox, VMware Workstation

- ✓ CPU có hỗ trợ ảo hóa (Intel VT-x, AMD-V) để máy ảo chạy gần tốc độ thật. Thường phải bật trong UEFI.
- ✓ Snapshot lưu trạng thái máy ảo, thử gì hỏng thì quay lại ngay.
- ✓ Cloud chạy phần lớn trên máy ảo: một máy chủ thật cho nhiều khách thuê.

Máy ảo cách ly mạnh, nhưng tốn RAM và đĩa vì mỗi máy có một hệ điều hành đầy đủ.

93 Container thật ra là gì?

Container không phải máy ảo. Nó chỉ là tiến trình bình thường, được kernel cho nhìn thế giới riêng và giới hạn tài nguyên.

```
old=$(hostname)
unshare --user --map-root-user --uts \
  bash -c 'hostname lab; hostname'
[ "$(hostname)" = "$old" ] && echo "Bên ngoài tên máy không đổi"

unshare --user --map-root-user --pid --fork --mount-proc \
  bash -c 'echo "PID của shell bên trong: $$"'

ls /proc/self/ns | grep -v children | tr '\n' ' '
```

KẾT QUẢ CHẠY TRÊN LINUX

```
lab
Bên ngoài tên máy không đổi
PID của shell bên trong: 1
cgroup ipc mnt net pid time user uts
```

Namespace	Cô lập
pid	Số hiệu tiến trình, nên bên trong có PID 1 riêng
net	Card mạng, cổng
mnt	Cây thư mục
uts	Tên máy
user	ID người dùng

Namespace quyết định nhìn thấy gì. Cgroup quyết định dùng được bao nhiêu CPU, RAM, I/O.

docker run --memory=256m đặt giới hạn cgroup. Vượt giới hạn thì tiến trình bị kết thúc, dù máy thật còn nhiều RAM.

94 Container hay máy ảo?

Câu phỏng vấn rất phổ biến. Khác biệt cốt lõi là có dùng chung kernel với máy chủ hay không.

Tiêu chí	Máy ảo	Container
Kernel	Riêng	Dùng chung
Khởi động	Chậm hơn	Rất nhanh
Kích thước	Lớn	Nhỏ
Cách ly	Mạnh hơn	Yếu hơn
Chạy khác hệ điều hành	Được	Không

- ✓ Container Linux cần kernel Linux. Vì thế Docker Desktop trên Windows, Mac phải có một máy ảo Linux bên dưới.
- ✓ Image gồm nhiều lớp chỉ đọc, chạy thì thêm một lớp ghi ở trên.
- ✓ Không nên chạy tiến trình trong container bằng root nếu không cần.
- ✓ Thực tế hay kết hợp: container chạy trong máy ảo trên cloud.

Một tiến trình trong container vẫn thấy được bằng ps trên máy chủ. Hiểu điều này là hiểu container không có phép màu gì.

95 WSL 2

Cách dễ nhất để có Linux trên Windows. WSL 2 chạy kernel Linux thật trong một máy ảo nhẹ do Windows quản lý.

```
wsl --install          # chạy trong PowerShell quyền Administrator
wsl -l -v              # xem các bản Linux và phiên bản WSL
wsl --shutdown         # tắt máy ảo WSL, giải phóng RAM
```

```
uname -sr | sed 's/-microsoft.*/ (của WSL 2)/'
python3 -c 'import os; print("Có fork:", hasattr(os, "fork"))'
```

KẾT QUẢ CHẠY TRÊN LINUX

```
Linux 6.18.33.1 (của WSL 2)
Có fork: True
```

Đặc điểm	WSL 1	WSL 2
Cách chạy	Dịch system call	Kernel Linux thật trong máy ảo
Tương thích system call	Một phần	Đầy đủ
systemd, Docker	Không	Có
File trên ổ Windows	Nhanh hơn	Chậm hơn

Mẹo tăng tốc lớn nhất: để project trong thư mục Linux như ~/code, đừng để ở /mnt/c. Đi qua lại giữa hai hệ file rất chậm.

- ✓ Mở thư mục Linux từ Windows: gõ \\wsl$ trong File Explorer.
- ✓ VS Code có tiện ích WSL: giao diện ở Windows, code chạy ở Linux.
- ✓ Toàn bộ ví dụ Linux trong cẩm nang này được chạy trên WSL 2.

Chọn môi trường học Linux

Không cần xóa Windows. Với đa số sinh viên, WSL 2 là đủ cho môn học và cho công việc lập trình hằng ngày.

Cách	Hợp với	Lưu ý
WSL 2	Học lệnh, lập trình, lab C	Dễ nhất, gần như không tốn cấu hình
VirtualBox	Thử nghiệm dễ hỏng, học cài đặt	Có snapshot, tốn RAM
Cài song song	Muốn dùng Linux thật hằng ngày	Sao lưu trước, chia phân vùng cẩn thận
Máy chủ cloud	Học triển khai dịch vụ	Tốn phí, phải bảo mật SSH

```
sudo apt update
sudo apt install -y build-essential gdb strace htop tree
gcc --version
mkdir -p ~/hdh && cd ~/hdh
```

1. **Máy ảo cần gì**
   Bật ảo hóa trong UEFI, RAM máy từ 8 GB trở lên là thoải mái.

2. **Chọn bản phân phối**
   Ubuntu LTS: tài liệu nhiều, ít gặp lỗi lạ.

3. **Giữ thói quen tốt**
   Dùng Git cho mọi bài lab, sao lưu trước khi thử lệnh lạ.

Không chạy lệnh sudo copy trên mạng khi chưa hiểu. Đặc biệt cẩn thận với rm -rf và dd, vì chúng không hỏi lại.

Lệnh Linux nên thuộc

Nắm các lệnh sau là tự tin làm lab, đọc log và vào server. Quên cú pháp thì dùng man hoặc --help.

Nhóm	Lệnh
Di chuyển	`pwd, cd, ls -la, tree`
File	`cp -r, mv, rm, mkdir -p, touch, ln -s`
Xem nội dung	`cat, less, head, tail -f, wc -l`
Tìm kiếm	`grep -rn, find . -name, which`
Xử lý văn bản	`sort, uniq -c, cut, sed, awk`
Quyền	`chmod, chown, sudo, id`
Tiến trình	`ps aux, top, kill, jobs, fg, bg`
Tài nguyên	`free -h, df -h, du -sh, lsblk`
Mạng	`ip a, ss -ltnp, ping, curl, ssh`
Gói phần mềm	`apt update, apt install, apt remove`
Nén	`tar czf, tar xzf, zip, unzip`
Dịch vụ	`systemctl status, journalctl -u`

Phím tắt terminal	Tác dụng
Tab	Tự điền tên lệnh, tên file
Ctrl+C, Ctrl+Z	Gửi SIGINT, tạm dừng tiến trình
Ctrl+R	Tìm trong lịch sử lệnh
Ctrl+D	Báo hết dữ liệu vào, thoát shell

Pipe và chuyển hướng

Sức mạnh của Unix: nhiều lệnh nhỏ nối với nhau. Dấu | chính là pipe, dấu > là đổi file descriptor.

```
printf '200\n404\n200\n500\n200\n404\n' > codes.txt

sort codes.txt | uniq -c | sed 's/^ *//' # đếm mỗi mã
grep -c 200 codes.txt                    # số dòng có 200
wc -l < codes.txt                        # lấy đầu vào từ file

ls khong_co 2> err.txt                   # lỗi đi vào file
cat err.txt
ls khong_co > /dev/null 2>&1 || echo "Lệnh trước đã lỗi"
```

KẾT QUẢ CHẠY TRÊN LINUX

```
3 200
2 404
1 500
3
6
ls: cannot access 'khong_co': No such file or directory
Lệnh trước đã lỗi
```

Ký hiệu	Ý nghĩa
a | b	Đầu ra của a thành đầu vào của b
a > f	Ghi đè vào file f
a >> f	Ghi nối vào cuối file f
a 2>&1	Gộp lỗi vào đầu ra chuẩn
a && b	a thành công mới chạy b
a || b	a thất bại mới chạy b
a &	Chạy a ở nền

Viết script Bash

Việc gì làm tới lần thứ ba thì nên viết thành script. Dòng set giúp script dừng ngay khi có lỗi.

```
cat > dem.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
if [ "$#" -eq 0 ]; then
    echo "Cách dùng: dem.sh thư_mục..." >&2
    exit 1
fi
for dir in "$@"; do
    count=$(find "$dir" -type f | wc -l)
    echo "$dir: $count file"
done
EOF
mkdir -p bai/lab1 bai/lab2
touch bai/lab1/a.c bai/lab1/b.c bai/lab2/c.c
bash dem.sh bai bai/lab1
bash dem.sh; echo "Mã thoát: $?"
```

KẾT QUẢ CHẠY TRÊN LINUX

```
bai: 3 file
bai/lab1: 2 file
Cách dùng: dem.sh thư_mục...
Mã thoát: 1
```

Quy tắc	Vì sao
`set -e`	Dừng ngay khi một lệnh lỗi
`set -u`	Báo lỗi khi dùng biến chưa đặt
`"$var"`	Luôn đặt biến trong nháy kép, tên file có dấu cách mới không vỡ

Theo dõi tiến trình

Dùng các lệnh này để nhìn thấy mọi thứ đã học: PID, cha con, trạng thái, độ ưu tiên.

```
$ ps -eo pid,ppid,stat,ni,comm --sort=pid | head -5
    PID    PPID STAT  NI COMMAND
      1       0 Ss     0 systemd
      2       1 Sl+    0 init-systemd(Ub
      7       2 Sl+    0 init
     56       1 S<s   -1 systemd-journal
```

Việc cần làm	Lệnh
Tìm theo tên	`pgrep -a python`
Xem cây cha con	`pstree -p`
Ăn CPU nhất	`ps aux --sort=-%cpu | head`
Ăn RAM nhất	`ps aux --sort=-rss | head`
Xem system call	`strace -c ls`
Kết thúc lịch sự	`kill PID`
Chạy nền, tiếp tục	`Ctrl+Z, bg, fg, jobs`

LOAD AVERAGE
Trung bình số việc đang chạy hoặc chờ trong 1, 5, 15 phút. Trên Linux có tính cả việc chờ đĩa ở trạng thái D.

ĐỌC THẾ NÀO
So với số lõi. Load 4 trên máy 8 lõi là nhàn, trên máy 2 lõi là quá tải.

Trong top: bấm P sắp theo CPU, M sắp theo RAM, 1 xem từng lõi, q để thoát. htop dễ nhìn hơn, cần cài thêm.

Theo dõi bộ nhớ và đĩa

vmstat cho thấy cả CPU, bộ nhớ, swap và I/O trên một dòng. Đọc được nó là biết máy đang nghẽn ở đâu.

```
$ vmstat 1 2
 r  b swpd    free  buff  cache  si  so   bi  bo  in  cs us sy  id wa
 0  1    0 2643380 49252 696108   0   0 6253 132 1166  0  2  4  93  1
 0  0    0 2646536 49252 696312   0   0    4   0  224 150 0  0 100  0
```

Cột	Nghĩa	Đáng lo khi
r	Việc đang chờ CPU	Lâu dài lớn hơn số lõi
b	Việc đang chờ I/O	Luôn lớn hơn 0
si, so	Đọc, ghi swap	Khác 0 liên tục: thiếu RAM
bi, bo	Đọc, ghi khối trên đĩa	Cao mà ứng dụng chậm
wa	CPU rảnh vì chờ I/O	Cao: đĩa đang nghẽn
cs	Số lần chuyển ngữ cảnh	Tăng vọt bất thường

Câu hỏi	Lệnh
Ổ nào còn bao nhiêu?	`df -h`
Còn inode không?	`df -i`
Thư mục nào lớn nhất?	`du -sh * | sort -h`
Ổ nào đang bận?	`iostat -x 1 (gói sysstat)`
Tiến trình nào ghi đĩa?	`sudo iotop`

Số liệu trên là kết quả thật trên máy thử, ngay sau khi khởi động WSL. Dòng đầu của vmstat là trung bình từ lúc khởi động.

Dịch vụ và lịch chạy

Server cần chạy chương trình khi khởi động, tự chạy lại khi lỗi, và chạy việc định kỳ.

```
sudo systemctl status nginx      # xem trạng thái
sudo systemctl restart nginx     # khởi động lại
sudo systemctl enable nginx      # tự chạy khi bật máy
journalctl -u nginx -n 50        # 50 dòng log mới nhất
journalctl -u nginx -f           # theo dõi log trực tiếp
```

Một dịch vụ tự viết chỉ cần một file trong /etc/systemd/system:

```
[Unit]
Description=API do an

[Service]
ExecStart=/usr/bin/python3 /opt/api/app.py
Restart=on-failure
User=api

[Install]
WantedBy=multi-user.target
```

cron chạy việc theo lịch. Năm trường là phút, giờ, ngày, tháng, thứ.

```
0 2 * * *    /home/an/backup.sh      # 2 giờ sáng mỗi ngày
*/15 * * * * /home/an/check.sh       # mỗi 15 phút
0 8 * * 1    /home/an/report.sh      # 8 giờ sáng thứ Hai
```

BẪY CỦA CRON
cron có ít biến môi trường hơn terminal. Hãy dùng đường dẫn đầy đủ và ghi log ra file.

WINDOWS
Services đảm nhận vai trò systemd, Task Scheduler đảm nhận vai trò cron.

Biến môi trường và PATH

Tiến trình con kế thừa biến môi trường của cha. PATH là danh sách thư mục mà shell tìm lệnh. Hiểu được là hết sợ command not found.

```
A=1
export B=2
bash -c 'echo "Tiến trình con thấy: A=$A B=$B"'

mkdir bin
printf '#!/bin/sh\necho "Xin chào"\n' > bin/chao
chmod +x bin/chao

chao
PATH="$PWD/bin:$PATH" chao
PATH="$PWD/bin:$PATH" command -v chao | sed "s|$PWD/||"
```

KẾT QUẢ CHẠY TRÊN LINUX

```
Tiến trình con thấy: A= B=2
bash: line 10: chao: command not found
Xin chào
bin/chao
```

Tình huống	Cách xử lý
Cài xong mà không tìm thấy lệnh	Thêm thư mục vào PATH, mở terminal mới
Có hai bản Python	`which -a python3` để xem thứ tự
Muốn đặt lâu dài	Thêm dòng export vào ~/.bashrc
Chạy file ở thư mục hiện tại	Gõ ./ten_file, vì dấu chấm không nằm trong PATH

Đừng để mật khẩu, khóa API trong mã nguồn. Truyền qua biến môi trường hoặc kho bí mật, và đưa file .env vào .gitignore.

Linux và Windows: lệnh tương đương

Cùng nguyên lý, khác công cụ. Sinh viên dùng Windows có thể quan sát phần lớn kiến thức ngay trên máy mình.

Việc	Linux	Windows PowerShell
Xem tiến trình	`ps aux`	`Get-Process`
Kết thúc tiến trình	`kill PID`	`Stop-Process -Id PID`
Xem dịch vụ	`systemctl`	`Get-Service`
Xem cổng	`ss -ltnp`	`netstat -ano`
Dung lượng ổ	`df -h`	`Get-Volume`
Tìm trong file	`grep`	`Select-String`
Biến môi trường	`env`	`Get-ChildItem Env:`

Công cụ giao diện	Xem được gì
Task Manager	CPU, RAM, đĩa, mạng theo tiến trình. Mở bằng Ctrl+Shift+Esc
Resource Monitor	Tiến trình nào đang đọc ghi file nào, chiếm cổng nào
Process Explorer	Cây cha con, luồng, handle. Thuộc bộ Sysinternals
Event Viewer	Log hệ thống, giống journalctl

Bài tập nhanh: mở Details trong Task Manager, thêm cột Threads, Handles, Page faults, rồi xem chúng thay đổi khi mở một trang web nặng.

Máy chậm, làm gì trong 5 phút?

Đừng đoán, hãy đo. Chỉ có bốn tài nguyên có thể nghẽn: CPU, RAM, đĩa và mạng.

Dấu hiệu	Nghi ngờ	Hướng xử lý
CPU gần 100%	Một tiến trình đang tính nặng	Tìm và tắt nó, kiểm tra quạt, nhiệt độ
RAM gần đầy, swap tăng	Thiếu RAM, có thể thrashing	Đóng tab, tìm rò rỉ, thêm RAM
Đĩa bận 100%	Đang cập nhật, quét, HDD quá chậm	Chờ xong, đổi sang SSD
Web chậm, app khác bình thường	Mạng hoặc DNS	ping, đổi DNS, thử mạng khác

1. **Mở công cụ đo**
   Task Manager hoặc top. Xem tài nguyên nào cao nhất.

2. **Tìm tiến trình gây ra**
   Sắp xếp theo cột đó, ghi lại tên và PID.

3. **Xử lý rồi đo lại**
   Sửa một thứ, đo một lần, để biết cái nào có tác dụng.

Đổi HDD sang SSD, nâng RAM 4 GB lên 8 GB hiệu quả hơn mọi phần mềm dọn rác.

12 lỗi gặp hằng ngày

Mỗi thông báo lỗi đều đến từ kernel. Biết nguồn gốc thì sửa nhanh hơn việc copy lên Google.

Thông báo	Thật ra là	Kiểm tra
Permission denied	Thiếu quyền, thiếu bit x	`ls -l, id`
No such file or directory	Sai đường dẫn, link hỏng	`pwd, ls -l`
command not found	Không có trong PATH	`which, echo $PATH`
No space left on device	Hết đĩa hoặc hết inode	`df -h, df -i`
Too many open files	Hết file descriptor, quên close	`ulimit -n, lsof`
Address already in use	Cổng đang có tiến trình giữ	`ss -ltnp`
Connection refused	Không ai lắng nghe ở cổng đó	`ss -ltn`
Killed, mã thoát 137	SIGKILL, thường do OOM killer	`dmesg | grep -i oom`
Segmentation fault, 139	SIGSEGV, truy cập sai bộ nhớ	`gdb, ASan`
Broken pipe	Bên đọc của pipe đã đóng	Xem bên nhận
Text file busy	Ghi đè file thực thi đang chạy	`lsof file`
Device or resource busy	Gỡ ổ khi vẫn có file đang mở	`lsof +D thư_mục`

Mã thoát lớn hơn 128 nghĩa là bị tín hiệu kết thúc: 137 = 128 + 9, còn 139 = 128 + 11.

Tái hiện lỗi để hiểu lỗi

Tự tạo được lỗi thì không còn sợ nó. Dưới đây là kết quả thật từ kernel Linux.

```
import errno, resource, socket
resource.setrlimit(resource.RLIMIT_NOFILE, (16, 16))
files = []
try:
    while True:
        files.append(open("/dev/null"))
except OSError as e:
    print(errno.errorcode[e.errno], "sau", len(files), "file")
for f in files:
    f.close()
a, b = socket.socket(), socket.socket()
a.bind(("127.0.0.1", 18080))
try:
    b.bind(("127.0.0.1", 18080))
except OSError as e:
    print(errno.errorcode[e.errno], e.strerror)
```

KẾT QUẢ CHẠY TRÊN LINUX

```
EMFILE sau 12 file
EADDRINUSE Address already in use
```

```
{ bash -c 'kill -9 $$'; } 2>/dev/null
echo "Mã thoát khi bị SIGKILL: $?"
{ python3 -c 'import ctypes; ctypes.string_at(0)'; } 2>/dev/null
echo "Mã thoát khi segfault: $?"
ulimit -v 200000
python3 -c 'x = bytearray(10 ** 9)' 2>&1 | tail -1
```

KẾT QUẢ CHẠY TRÊN LINUX

```
Mã thoát khi bị SIGKILL: 137
Mã thoát khi segfault: 139
MemoryError
```

Lab 1: Tự viết shell mini

Một shell chỉ là vòng lặp fork, exec, wait và pipe.

```
import os

def run(line):
    cmds = [c.split() for c in line.split("|")]
    prev, pids = 0, []                          # lệnh đầu đọc từ stdin
    for i, cmd in enumerate(cmds):
        r, w = os.pipe() if i < len(cmds) - 1 else (0, 1)
        pid = os.fork()
        if pid == 0:                            # tiến trình con
            os.dup2(prev, 0)                    # đọc từ lệnh trước
            os.dup2(w, 1)                       # ghi cho lệnh sau
            os.execvp(cmd[0], cmd)
        if prev != 0:                           # cha đóng các đầu không dùng
            os.close(prev)
        if w != 1:
            os.close(w)
        prev = r
        pids.append(pid)
    for pid in pids:
        os.waitpid(pid, 0)
for line in ("echo xin chao", "seq 1 20 | grep 1 | wc -l"):
    print("$", line)
    run(line)
```

KẾT QUẢ CHẠY TRÊN LINUX

```
$ echo xin chao
xin chao
$ seq 1 20 | grep 1 | wc -l
11
```

BÀI TẬP

Thêm vòng lặp đọc lệnh bằng input(), lệnh cd, chuyển hướng dấu > và chạy nền bằng dấu &.

cd phải chạy trong chính shell, vì con đổi thư mục thì cha không bị ảnh hưởng.

Lab 2: Thread pool

Web server không tạo luồng mới cho mỗi yêu cầu. Nó giữ sẵn vài luồng và một hàng đợi công việc.

```
import queue, threading

tasks, results = queue.Queue(), []
lock = threading.Lock()

def worker():
    while True:
        n = tasks.get()                  # ngủ khi hàng đợi rỗng
        if n is None:
            return
        with lock:
            results.append(n * n)

pool = [threading.Thread(target=worker) for _ in range(4)]
for t in pool: t.start()
for n in range(100):
    tasks.put(n)
for _ in pool:
    tasks.put(None)                      # mỗi luồng một lệnh dừng
for t in pool: t.join()
print(len(results), sum(results))
```

KẾT QUẢ KHI CHẠY

```
100 328350
```

BÀI TẬP

Giới hạn độ dài hàng đợi. Trả kết quả về qua một đối tượng Future. Đo thời gian với 1, 2, 4, 8 luồng cho việc chờ I/O và cho việc nặng CPU.

Trong dự án thật hãy dùng concurrent.futures.ThreadPoolExecutor. Bài này để hiểu bên trong nó có gì.

Lab 3: Hệ thống file mini

Mô phỏng khối, bitmap, inode và thư mục. Toàn bộ nằm trong một lớp Python ngắn.

```python
class MiniFS:
    def __init__(self, blocks=6, size=4):
        self.data, self.free = [b""] * blocks, [True] * blocks
        self.inodes, self.dir, self.size, self.next = {}, {}, size, 1

    def create(self, name, text):
        k = self.size
        parts = [text[i:i + k] for i in range(0, len(text), k)]
        slots = [i for i, ok in enumerate(self.free) if ok]
        slots = slots[:len(parts)]
        if len(slots) < len(parts):
            raise OSError("No space left on device")
        for i, part in zip(slots, parts):
            self.data[i], self.free[i] = part, False
        self.inodes[self.next] = {"blocks": slots, "links": 1}
        self.dir[name] = self.next
        self.next += 1

    def link(self, old, new):
        self.dir[new] = self.dir[old]
        self.inodes[self.dir[old]]["links"] += 1

    def read(self, name):
        node = self.inodes[self.dir[name]]
        return b"".join(self.data[i] for i in node["blocks"])

    def unlink(self, name):
        ino = self.dir.pop(name)
        self.inodes[ino]["links"] -= 1
        if self.inodes[ino]["links"] == 0:
            for i in self.inodes.pop(ino)["blocks"]:
                self.free[i] = True
```

data là các khối trên đĩa, free là bitmap, inodes là bảng inode, dir là thư mục.

Lab 3: Chạy thử

Đĩa có 6 khối, mỗi khối 4 byte. Xem hard link giữ dữ liệu, và khi nào thì hết chỗ.

KẾT QUẢ KHI CHẠY

```
b'xin chao ban' | khối trống: 3
No space left on device
Sau khi xóa link cuối, khối trống: 6
```

```python
fs = MiniFS()
fs.create("a.txt", b"xin chao ban")
fs.link("a.txt", "b.txt")
fs.unlink("a.txt")
print(fs.read("b.txt"), "| khối trống:", sum(fs.free))
try:
    fs.create("c.txt", b"qua dai " * 3)
except OSError as e:
    print(e)
fs.unlink("b.txt")
print("Sau khi xóa link cuối, khối trống:", sum(fs.free))
```

Trong code	Trong hệ thống thật
self.data	Các khối dữ liệu trên đĩa
self.free	Bitmap vùng trống
self.inodes	Bảng inode: vị trí khối, số link
self.dir	Thư mục: tên đến số inode
unlink	Lệnh rm: chỉ thu hồi khối khi số link bằng 0

BÀI TẬP

Thêm thư mục con, quyền đọc ghi, và ghi nối file. Giả lập mất điện giữa lúc ghi, rồi thêm journal để phục hồi.

Lab 4: Đo tính cục bộ

Vì sao cache và bộ nhớ ảo hiệu quả? So sánh truy cập ngẫu nhiên với truy cập có tính cục bộ.

```
import random
from collections import OrderedDict

def hit_rate(refs, n):                          # LRU với n khung
    cache, hits = OrderedDict(), 0
    for p in refs:
        if p in cache:
            hits += 1
            cache.move_to_end(p)
        else:
            cache[p] = True
            if len(cache) > n:
                cache.popitem(last=False)
    return hits / len(refs)
rng = random.Random(1)
uniform = [rng.randrange(1000) for _ in range(100_000)]
local, base = [], 0
for _ in range(100_000):
    if rng.random() < 0.01:
        base = rng.randrange(1000)              # thỉnh thoảng đổi vùng
    local.append((base + rng.randrange(20)) % 1000)
for n in (10, 30, 100):
    print(f"{n:<3} khung | ngẫu nhiên {hit_rate(uniform, n):6.1%}"
          f" | cục bộ {hit_rate(local, n):6.1%}")
```

KẾT QUẢ KHI CHẠY

```
10  khung | ngẫu nhiên   1.0% | cục bộ  46.8%
30  khung | ngẫu nhiên   3.0% | cục bộ  83.2%
100 khung | ngẫu nhiên  10.0% | cục bộ  84.4%
```

BÀI TẬP

Thay LRU bằng FIFO, Clock rồi vẽ biểu đồ. Tìm số khung mà tỉ lệ trúng không tăng thêm nhiều.

Ý tưởng đồ án môn học

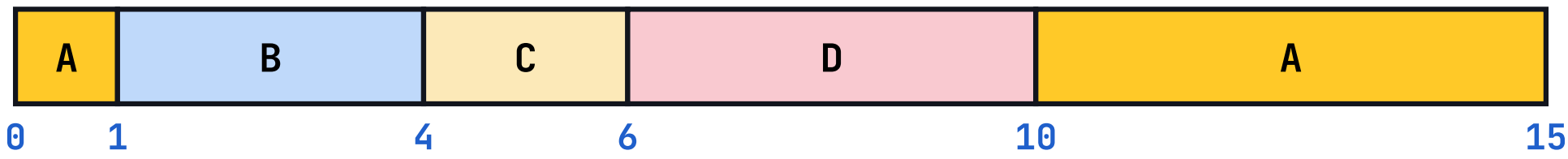
Đồ án tốt có phần chạy được, có số đo thật và có giải thích bằng kiến thức của môn.

Đề tài	Mức	Kiến thức dùng
Mô phỏng lập lịch có giao diện	Dễ	FCFS, SJF, RR, biểu đồ Gantt
Mô phỏng thay trang	Dễ	FIFO, LRU, Clock trên dữ liệu thật
Shell hoàn chỉnh	Vừa	fork, exec, pipe, tín hiệu, job control
Web server đa luồng	Vừa	Socket, thread pool, epoll, đo tải
Trình quản lý tác vụ	Vừa	Đọc /proc, hiển thị CPU, RAM
Tự viết malloc	Khó	sbrk, mmap, danh sách vùng trống
Container mini	Khó	Namespace, cgroup, chroot
Thêm system call vào xv6	Khó	Kernel, trap, bộ nhớ

- ✓ Phát biểu rõ giả thuyết, ví dụ: Round Robin q nào tốt nhất cho dữ liệu này?
- ✓ Có số đo trước và sau, ghi cấu hình máy, chạy nhiều lần.
- ✓ Dùng Git từ ngày đầu, mỗi thành viên có commit của mình.
- ✓ Được dùng AI hỗ trợ thì phải hiểu từng dòng, vì vấn đáp sẽ hỏi tới.

Bài mẫu: lập lịch

Đề. A đến lúc 0 chạy 6, B đến lúc 1 chạy 3, C đến lúc 3 chạy 2, D đến lúc 4 chạy 4. Dùng SRTF. Vẽ Gantt, tính thời gian chờ và hoàn thành trung bình.



KẾT QUẢ KHI CHẠY

```
A: kết thúc 15, hoàn thành 15, chờ 9
B: kết thúc 4, hoàn thành 3, chờ 0
C: kết thúc 6, hoàn thành 3, chờ 1
D: kết thúc 10, hoàn thành 6, chờ 2
Trung bình: chờ 3.0 | hoàn thành 6.75
```

Lúc 0. Chỉ có A, chạy A.

Lúc 1. B đến, cần 3, A còn 5. B ngắn hơn nên chiếm CPU.

Lúc 3. C đến, cần 2, B còn 1. B chạy tiếp.

Lúc 4. B xong, D đến. A còn 5, C cần 2, D cần 4. Chọn C.

Lúc 6. C xong. D cần 4, A còn 5. Chọn D. Sau D mới tới A.

Bài đầy đủ gồm: Gantt, bảng từng tiến trình, công thức, kết quả trung bình. Thiếu bảng có thể bị trừ điểm dù đáp số đúng.

Bài mẫu: Banker

Đề. 3 loại tài nguyên, tổng (7, 4, 5). Allocation: P0 (1, 0, 1), P1 (2, 1, 1), P2 (2, 1, 1). Max: P0 (3, 2, 2), P1 (4, 2, 3), P2 (5, 3, 5). Trạng thái có an toàn không? P0 xin (1, 0, 1) thì có cấp không?

KẾT QUẢ KHI CHẠY

```
Available = [2, 2, 2] | Need = [[2, 2, 1], [2, 1, 2], [3, 2, 4]]
P0: Need [2, 2, 1] ≤ Work, Work mới = [3, 2, 3]
P1: Need [2, 1, 2] ≤ Work, Work mới = [5, 3, 4]
P2: Need [3, 2, 4] ≤ Work, Work mới = [7, 4, 5]
An toàn: (True, ['P0', 'P1', 'P2'])
Sau khi thử cấp, Available = [1, 2, 1] | Need P0 = [1, 2, 0]
An toàn: (True, ['P0', 'P1', 'P2'])
```

Bước 1. Tính Available = Tổng - tổng theo cột của Allocation. Tính Need = Max - Allocation.

Bước 2. Chạy thuật toán an toàn, ghi Work sau từng tiến trình, kết luận thứ tự an toàn.

Bước 3. Với yêu cầu mới: kiểm tra Request ≤ Need và Request ≤ Available.

Bước 4. Thử cấp, chạy lại thuật toán an toàn. Còn thứ tự an toàn thì cấp.

Kết luận: sau khi thử cấp vẫn có thứ tự an toàn, nên cấp cho P0. Luôn ghi rõ bảng Need, đó là phần được tính điểm.

Bài mẫu: phân trang và TLB

Đề 1. Địa chỉ logic 16 bit, trang 1 KB. Bảng trang: trang 0 ở khung 3, trang 1 ở khung 7, trang 2 ở khung 5. Dịch địa chỉ 2500.

Lời giải. Độ dời 10 bit, số trang 6 bit. p = 2500 / 1024 = 2, d = 2500 - 2048 = 452. Khung 5, địa chỉ vật lý = 5 × 1024 + 452 = 5572.

Đề 2. Truy cập RAM 100 ns, tra TLB 20 ns, tỉ lệ trúng 90%. Tính EAT với bảng trang một cấp, rồi hai cấp.

Lời giải. Một cấp: 0,9 × 120 + 0,1 × 220 = 130 ns. Hai cấp: khi trượt phải đọc thêm hai lần, 0,9 × 120 + 0,1 × 320 = 140 ns.

Đề 3. Địa chỉ 48 bit, trang 16 KB, mỗi mục 8 byte. Bảng trang một cấp lớn bao nhiêu?

Lời giải. Độ dời 14 bit, còn 34 bit số trang. $2^{34} \times 8$ byte = 128 GiB. Vì vậy bắt buộc phải dùng nhiều cấp.

```
table = {0: 3, 1: 7, 2: 5}
p, d = divmod(2500, 1024)
print("Đề 1:", p, d, table[p] * 1024 + d)

h, t, m = 0.9, 20, 100
print("Đề 2:", round(h * (t + m) + (1 - h) * (t + 2 * m), 1),
      round(h * (t + m) + (1 - h) * (t + 3 * m), 1))

print("Đề 3:", 2 ** (48 - 14) * 8 // 2 ** 30, "GiB")
```

KẾT QUẢ KHI CHẠY

```
Đề 1: 2 452 5572
Đề 2: 130.0 140.0
Đề 3: 128 GiB
```

Bài mẫu: thay trang

Đề. Chuỗi 1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3 với 3 khung. Đếm lỗi trang bằng LRU, trình bày bảng.

	1	2	3	4	2	1	5	6	2	1	2	3
K1	1	1	1	4	4	4	5	5	5	1	1	1
K2		2	2	2	2	2	2	6	6	6	6	3
K3			3	3	3	1	1	1	2	2	2	2
	F	F	F	F		F	F	F	F	F		F

KẾT QUẢ KHI CHẠY

```
Tổng số lỗi trang: 10
```

Cách làm nhanh. Tại mỗi lỗi trang khi khung đã đầy, nhìn ngược chuỗi về trái. Trang trong khung được nhìn thấy sau cùng là trang bị đuổi.

Ví dụ ở vị trí số 7 (trang 5). Trong khung có 1, 2, 4. Nhìn ngược: 1, 2, rồi 4. Vậy đuổi 4.

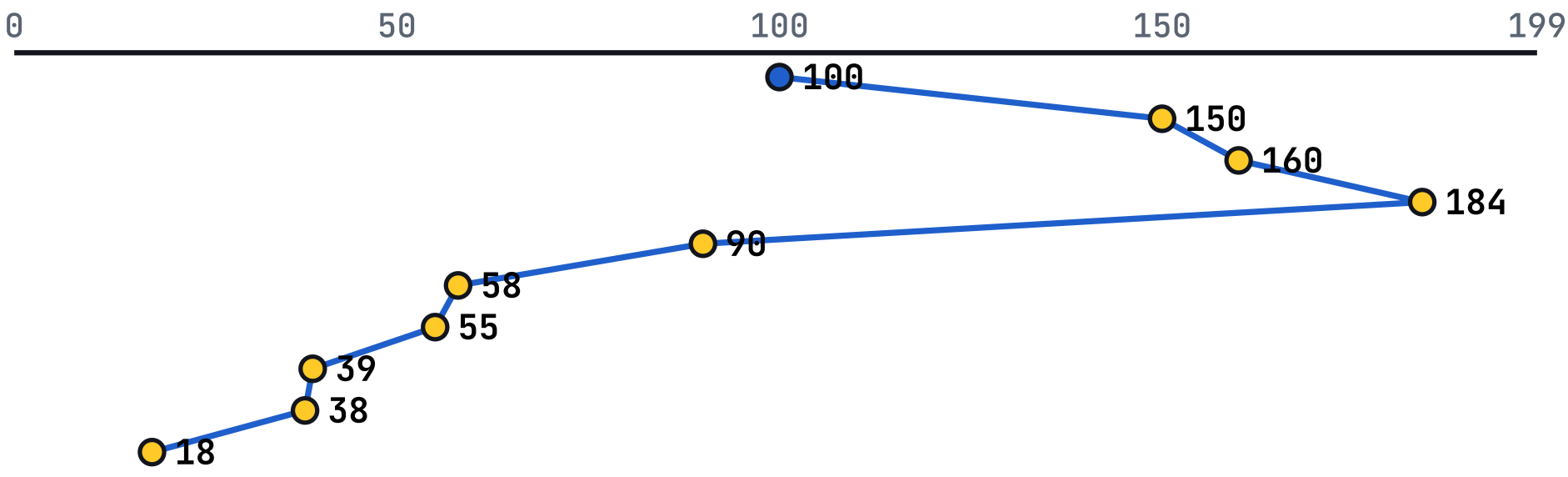
Với FIFO. Ghi thứ tự nạp của các trang, đuổi trang nạp sớm nhất. Truy cập trúng không đổi thứ tự.

Với OPT. Nhìn về phía phải, đuổi trang được dùng lại muộn nhất hoặc không bao giờ dùng lại.

Đừng quên đếm các lỗi đầu tiên khi khung còn trống, trừ khi đề nói bắt đầu với khung đã nạp sẵn.

Bài mẫu: đĩa và inode

Đề 1. Đầu đọc ở 100, đang đi lên, đĩa 0 đến 199, yêu cầu 55, 58, 39, 18, 90, 160, 150, 38, 184. Tính tổng quãng đường theo SSTF và LOOK.



KẾT QUẢ KHI CHẠY

```
SSTF: [90, 58, 55, 39, 38, 18, 150, 160, 184] 248
LOOK: 250
Đề 2: 262666 khối, 513 MiB
```

Đề 2. Khối 2 KB, con trỏ 4 byte, inode có 10 con trỏ trực tiếp, 1 gián tiếp đơn, 1 gián tiếp đôi. File lớn nhất là bao nhiêu?

Lời giải. Một khối chứa n = 512 con trỏ. Tổng khối = 10 + 512 + 512^2. Nhân với 2 KB để ra kích thước.

Hình trên là đường đi của LOOK. SSTF có thể hòa khi hai yêu cầu cách đều, hãy trình bày cách phá hòa của bạn.

Bảng công thức

Nội dung	Công thức
Hoàn thành	Kết thúc - Đến
Chờ	Hoàn thành - Chạy
Đáp ứng	Lần đầu chạy - Đến
Dự đoán burst	$\tau(n+1) = \alpha \cdot t(n) + (1 - \alpha) \cdot \tau(n)$
Đa chương	CPU bận = $1 - p^n$
Số tiến trình sau n fork	2^n
Banker	Need = Max - Allocation
Số bit độ dời	$\log_2$(cỡ trang)
Dịch địa chỉ	p = a / trang, d = a mod trang, vật lý = f × trang + d
Cỡ bảng trang	Số trang × cỡ mục
EAT có TLB	h(t + m) + (1 - h)(t + 2m)
EAT có lỗi trang	(1 - p)·m + p·thời gian xử lý lỗi
Cấp theo tỉ lệ	$a_i = s_i / S \times m$
Thời gian đĩa	Tìm rãnh + quay + truyền
Chờ quay trung bình	60 / RPM / 2 giây
Inode	(trực tiếp + n + n^2 + n^3) × cỡ khối, n = cỡ khối / cỡ con trỏ
RAID 5	Dùng được (n - 1) ổ
Thời gian thực	U = Σ C/T, RM: U ≤ n(2^(1/n) - 1), EDF: U ≤ 1

Chụp lại trang này để xem nhanh trước giờ thi. Một số trường cho mang một tờ tài liệu, hãy hỏi giảng viên.

Tự luyện

Tự làm trước, đáp án ở trang sau. Dùng chung đề: P1 (đến 0, chạy 5), P2 (1, 3), P3 (2, 8), P4 (3, 6).

Bài 1. Tính thời gian chờ trung bình với FCFS.

Bài 2. Tính thời gian chờ trung bình với SJF không chiếm quyền.

Bài 3. Vẽ Gantt với Round Robin, q = 3.

Bài 4. Có bao nhiêu tiến trình sau khi chạy for (i = 0; i < 4; i++) fork();

Bài 5. Available (2, 1), Allocation P0 (1, 0), P1 (1, 1), P2 (0, 1), Need P0 (2, 2), P1 (1, 1), P2 (3, 2). Có an toàn không?

Bài 6. Địa chỉ 40 bit, trang 8 KB, mỗi mục 8 byte. Tính số bit độ dời và cỡ bảng trang một cấp.

Bài 7. TLB trúng 95%, tra TLB 5 ns, RAM 80 ns. Tính EAT.

Bài 8. Đếm lỗi trang của LRU với 3 khung, chuỗi 1 2 3 4 1 2 5 1 2 3 4 5.

Bài 9. Đầu đọc ở 100, yêu cầu 23, 89, 132, 42, 187. Tính tổng quãng đường của SSTF.

Bài 10. Khối 1 KB, con trỏ 4 byte, 10 trực tiếp, 1 gián tiếp đơn, 1 đôi. Tính file lớn nhất theo KiB.

Đáp án tự luyện

Các đáp án dưới đây do code tính ra.

KẾT QUẢ KHI CHẠY

```
1. FCFS: 5.75
2. SJF: 5.25
3. RR: P1 P2 P3 P4 P1 P3 P4 P3 | xong lúc 22
4. Số tiến trình: 16
5. An toàn: True ['P1', 'P0', 'P2']
6. Độ dời 13 bit, bảng 1 GiB
7. EAT: 89.0 ns
8. Số lỗi trang: 10
9. Quãng đường: 273
10. Lớn nhất: 65802 KiB
```

Mẹo kiểm tra lại	Cách làm
Gantt	Tổng độ dài mỗi tiến trình phải bằng thời gian chạy
Banker	Cuối cùng Work phải bằng tổng tài nguyên
Dịch địa chỉ	Độ dời phải nhỏ hơn cỡ trang
Thay trang	Số lỗi không ít hơn số trang khác nhau

Trắc nghiệm (1/2)

1. Thành phần nào chạy ở kernel mode?
A. Trình duyệt B. Driver C. Trình soạn thảo D. Shell

2. Ứng dụng nhờ kernel làm việc bằng cách nào?
A. Gọi hàm thường B. Biến toàn cục C. System call D. Đọc thẳng RAM

3. Sau 3 lần fork liên tiếp có bao nhiêu tiến trình?
A. 3 B. 6 C. 7 D. 8

4. Zombie là gì?
A. Con đã thoát, cha chưa wait B. Cha đã thoát C. Tiến trình bị treo D. Tiến trình mã độc

5. Các luồng trong một tiến trình không dùng chung gì?
A. Heap B. Stack C. Code D. File đang mở

6. Mọi việc đến cùng lúc, thuật toán nào cho chờ trung bình nhỏ nhất?
A. FCFS B. Round Robin C. SJF D. Ưu tiên

7. Round Robin với q rất lớn giống thuật toán nào?
A. FCFS B. SJF C. SRTF D. Ưu tiên

8. Kỹ thuật nào chống đói trong lập lịch ưu tiên?
A. Phân trang B. Spooling C. Polling D. Lão hóa

9. Đâu không phải yêu cầu của bài toán đoạn găng?
A. Loại trừ tương hỗ B. Tiến triển C. Chờ có giới hạn D. Chiếm quyền

10. Semaphore khởi tạo bằng 0 thường dùng để làm gì?
A. Cấp RAM B. Báo hiệu thứ tự C. Đổi trang D. Chạy song song

Trắc nghiệm (2/2)

11. Đâu không phải điều kiện Coffman?
A. Giữ và chờ B. Chờ vòng tròn C. Không chiếm đoạt D. Đói tài nguyên

12. Thuật toán Banker thuộc chiến lược nào?
A. Ngăn chặn B. Tránh C. Phát hiện D. Bỏ qua

13. Địa chỉ 32 bit, trang 4 KB thì độ dời có bao nhiêu bit?
A. 12 B. 10 C. 20 D. 32

14. TLB dùng để làm gì?
A. Lưu file B. Lập lịch CPU C. Cache mục bảng trang D. Chống bế tắc

15. Phân trang loại bỏ được gì?
A. Phân mảnh trong B. Phân mảnh ngoài C. Lỗi trang D. TLB

16. Thuật toán nào có thể gặp nghịch lý Belady?
A. OPT B. LRU C. FIFO D. Cả ba

17. Thrashing xảy ra khi nào?
A. Thiếu khung cho tập làm việc B. CPU quá nhanh C. Mạng chậm D. Ổ cứng mới

18. Hard link có đặc điểm gì?
A. Có inode riêng B. Chứa đường dẫn C. Hỏng khi xóa tên gốc D. Cùng inode với file gốc

19. Thuật toán đĩa nào có thể gây đói?
A. FCFS B. SSTF C. SCAN D. C-LOOK

20. Container khác máy ảo ở điểm nào?
A. Không chạy được Linux B. Không có tiến trình C. Dùng chung kernel với máy chủ D. Không dùng RAM

Đáp án trắc nghiệm

Câu	Đáp án	Giải thích
1	B	Driver chạy cùng kernel
2	C	System call là cổng vào kernel
3	D	2 mũ 3
4	A	Chỉ còn một dòng trong bảng tiến trình
5	B	Mỗi luồng có stack riêng
6	C	Việc ngắn chạy trước
7	A	Không ai bị ngắt giữa chừng
8	D	Tăng ưu tiên theo thời gian chờ
9	D	Chiếm quyền là khái niệm của lập lịch
10	B	Bên chờ ngủ tới khi bên kia signal
11	D	Bốn điều kiện còn lại là loại trừ, giữ và chờ, không chiếm đoạt, chờ vòng
12	B	Chỉ cấp khi vẫn an toàn
13	A	$\log_2(4096) = 12$
14	C	Tránh đọc bảng trang trong RAM
15	B	Vẫn còn phân mảnh trong ở trang cuối
16	C	LRU và OPT là thuật toán dạng stack
17	A	Trang vừa đuổi lại cần ngay
18	D	Chỉ là thêm một tên
19	B	Yêu cầu ở xa có thể bị bỏ qua mãi
20	C	Chỉ dùng namespace và cgroup

20 câu vấn đáp (1/2)

1. Chương trình và tiến trình khác nhau thế nào?
Chương trình là file trên đĩa. Tiến trình là chương trình đang chạy, có bộ nhớ và trạng thái.

2. Tiến trình và luồng khác nhau thế nào?
Tiến trình có không gian địa chỉ riêng. Các luồng trong một tiến trình dùng chung bộ nhớ, mỗi luồng có stack riêng.

3. Điều gì xảy ra khi gõ một lệnh trong shell?
Shell tìm lệnh trong PATH, fork một con, con exec chương trình, shell wait tới khi con kết thúc.

4. Context switch là gì, tốn gì?
Lưu trạng thái tiến trình cũ, nạp tiến trình mới. Tốn thời gian trực tiếp, và làm cache, TLB nguội.

5. System call khác gọi hàm thường thế nào?
Nó chuyển từ user mode sang kernel mode qua một lệnh đặc biệt, nên tốn hơn gọi hàm.

6. Mutex khác semaphore thế nào?
Mutex có chủ và chỉ có hai trạng thái. Semaphore là bộ đếm, luồng khác có thể signal.

7. Race condition là gì, sửa thế nào?
Kết quả phụ thuộc thứ tự xen kẽ. Sửa bằng khóa, kiểu nguyên tử, hoặc tránh chia sẻ dữ liệu.

8. Bốn điều kiện của deadlock?
Loại trừ tương hỗ, giữ và chờ, không chiếm đoạt, chờ vòng tròn.

9. Tránh deadlock khi lập trình bằng cách nào?
Khóa theo thứ tự cố định, giữ khóa ngắn, dùng timeout, tránh gọi code lạ khi đang giữ khóa.

10. Bộ nhớ ảo là gì, có lợi gì?
Mỗi tiến trình có không gian địa chỉ riêng, được ánh xạ sang RAM theo trang. Cách ly, cho chạy chương trình lớn, cấp phát lười.

20 câu vấn đáp (2/2)

11. Lỗi trang xử lý thế nào?
Phần cứng báo kernel. Kernel kiểm tra địa chỉ, tìm khung, nạp trang, sửa bảng trang rồi chạy lại lệnh.

12. TLB để làm gì?
Lưu các ánh xạ gần đây trong CPU, tránh phải đọc bảng trang trong RAM ở mỗi lần truy cập.

13. Stack và heap khác nhau thế nào?
Stack tự quản lý theo lời gọi hàm, nhỏ và nhanh. Heap cấp phát động, lớn hơn, phải tự trả hoặc nhờ GC.

14. Thrashing là gì?
Không đủ khung cho tập làm việc, nên hệ thống dành gần hết thời gian để đổi trang.

15. Inode là gì?
Cấu trúc lưu thông tin và vị trí khối của file, trừ tên. Tên nằm trong thư mục.

16. Hard link khác soft link thế nào?
Hard link là một tên khác của cùng inode. Soft link là file riêng chứa đường dẫn.

17. kill và kill -9 khác nhau thế nào?
kill gửi SIGTERM, chương trình có thể dọn dẹp. SIGKILL do kernel kết thúc ngay, không bắt được.

18. fsync dùng để làm gì?
Buộc dữ liệu trong cache của kernel xuống thiết bị lưu trữ, cần cho các dữ liệu không được mất.

19. epoll giải quyết vấn đề gì?
Cho một luồng theo dõi nhiều kết nối hiệu quả, không cần một luồng cho mỗi kết nối.

20. Container khác máy ảo thế nào?
Container là tiến trình dùng chung kernel, cô lập bằng namespace và cgroup. Máy ảo có kernel riêng.

Lỗi hay mất điểm

Nhầm lẫn	Cách đúng
Chờ = lúc bắt đầu chạy	Chờ = hoàn thành - chạy, tính cả các lần bị ngắt
Quên thời gian đến	Hoàn thành tính từ lúc đến, không từ 0
RR xếp sai hàng đợi	Việc mới đến xếp trước việc vừa hết lượt, trừ khi đề nói khác
Có chu trình là deadlock	Chỉ chắc chắn khi mỗi tài nguyên có 1 thực thể
Không an toàn là deadlock	Chỉ mới có nguy cơ
Trộn KB và Kb, 1000 và 1024	Bài bộ nhớ dùng lũy thừa của 2
EAT quên lần đọc bảng trang	Trượt TLB phải đọc RAM hai lần
LRU tính theo thời điểm nạp	Tính theo lần dùng gần nhất
C-SCAN quên bước nhảy	Đọc đề xem có tính bước nhảy không

Đề luôn ghi quy ước riêng: ưu tiên số nhỏ hay số lớn, phá hòa thế nào, hướng đầu đọc. Khoanh các quy ước này trước khi làm.

Kế hoạch 15 tuần

Tuần	Nội dung	Làm
1	Tổng quan, system call	Cài WSL, học lệnh cơ bản
2	Tiến trình, fork, exec	Bài đếm fork, zombie
3	Luồng, IPC, tín hiệu	Bắt đầu Lab 1
4	Lập lịch cơ bản	Tự vẽ Gantt, viết code
5	RR, MLFQ, hệ thống thật	So sánh các thuật toán
6	Race condition, khóa	Đo lỗi race bằng C
7	Semaphore, bài toán kinh điển	Lab 2, ôn giữa kỳ
8	Deadlock, Banker	Bài mẫu Banker
9	Phân trang, TLB	Bài tính địa chỉ
10	Bộ nhớ ảo, thay trang	Lab 4
11	File, inode, link	Thử lệnh trên Linux
12	Cấp phát đĩa, journal, RAID	Lab 3
13	Điều phối đĩa, I/O	Làm bài mẫu
14	Ảo hóa, container	Chạy thử unshare, Docker
15	Ôn tập	Bài mẫu, trắc nghiệm, vấn đáp

Trình tự theo cẩm nang. Hãy chỉnh cho khớp đề cương và lịch của trường bạn.

Học gì tiếp theo?

Hệ điều hành là nền của backend, DevOps, nhúng và bảo mật. Chắc môn này thì học các hướng đó nhanh hơn nhiều.

Hướng	Cần đào sâu
Backend	Luồng, I/O, cache, hiệu năng, mạng
DevOps, SRE	Linux, container, giám sát, xử lý sự cố
Nhúng, IoT	RTOS, driver, ngắt, C
An toàn thông tin	Cơ chế bảo vệ, phân tích mã độc
Hệ thống, kernel	xv6, Linux kernel, C, Rust

1 **Đọc OSTEP**
Sách miễn phí, dễ đọc, có phần code và bài tập.

2 **Làm lab xv6**
Khóa MIT 6.1810 công khai tài liệu. xv6 là hệ điều hành nhỏ, đọc hiểu được.

3 **Đọc man page**
Dùng man 2 fork, man 7 signal thay vì chỉ xem bài blog.

4 **Quan sát hệ thống thật**
strace chương trình của bạn, đọc /proc, thử đo trước và sau khi tối ưu.

Nên học cùng cẩm nang Mạng máy tính, Cấu trúc dữ liệu và An toàn thông tin của Code4Life. Các môn này hỗ trợ nhau rất nhiều.

Tài liệu nên đọc

Nguồn	Vì sao nên đọc
Arpaci-Dusseau **Operating Systems: Three Easy Pieces**	Miễn phí, rất dễ hiểu
Silberschatz, Galvin, Gagne **Operating System Concepts**	Giáo trình phổ biến nhất
Tanenbaum, Bos **Modern Operating Systems**	Rộng, giải thích kỹ
Kerrisk **The Linux Programming Interface**	Lập trình hệ thống Linux
Cox, Kaashoek, Morris **xv6: a simple Unix-like teaching OS**	Miễn phí, đọc mã nguồn kernel
Từ Minh Phương **Giáo trình Hệ điều hành**	Tiếng Việt
Hà Quang Thụy **Giáo trình Nguyên lý các hệ điều hành**	Tiếng Việt
man7.org, docs.kernel.org	Tài liệu Linux chính thống
Microsoft Learn	Tài liệu Windows, WSL

Thấy chỗ nào chưa đúng? Anh em comment giúp để Code4Life sửa ở bản sau nhé.

Biên soạn: Tào Việt Đức, Code4Life Technology, với sự hỗ trợ của AI. Code Python chạy trên Python 3.14. Ví dụ Linux chạy trên Ubuntu trong WSL 2. Thông số phần cứng trong ví dụ là giả định để minh họa.